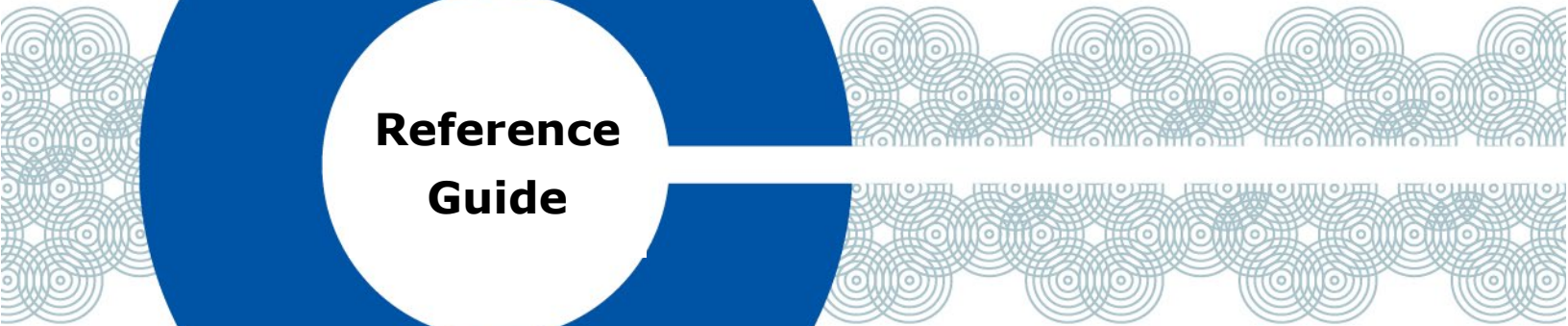
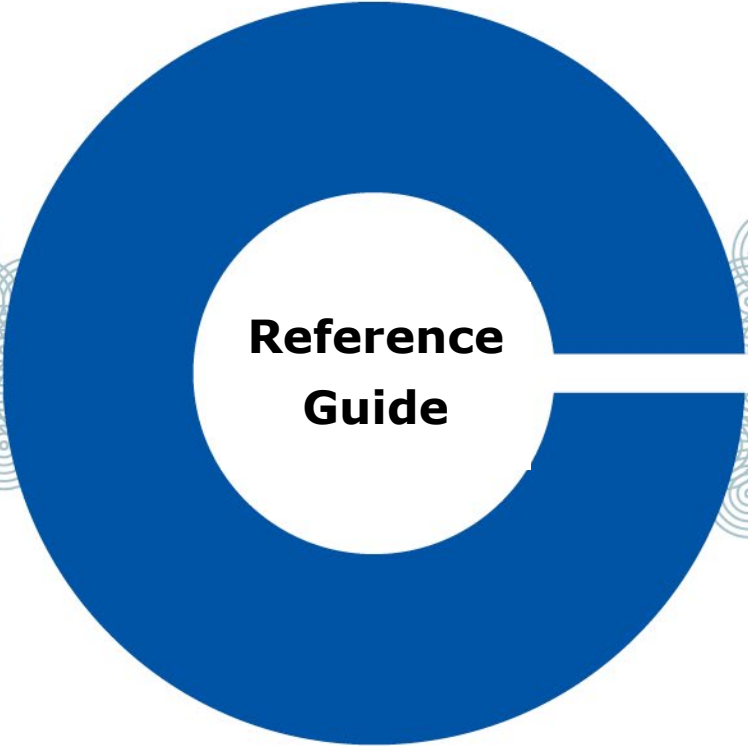




Reference Guide

HCI 2.0 Reference Guide

399G175 Rev F
20 Nov 2024

Document Reference

Clear-Com HCI Reference Guide

Part Number: 399G175 Rev F

Legal Disclaimers

Copyright © 2025 HME Clear-Com Ltd.

All rights reserved.

Clear-Com and the Clear-Com logo are registered trademarks of HM Electronics, Inc.

The software described in this document is furnished under a license agreement and may be used only in accordance with the terms of the agreement.

The product described in this document is distributed under licenses restricting its use, copying, distribution, and decompilation/reverse engineering. No part of this document may be reproduced in any form by any means without prior written authorization of Clear-Com, an HME Company.

Clear-Com Offices are located in California, USA; Cambridge, UK; Dubai, UAE; Montreal, Canada; and Beijing, China. Specific addresses and contact information can be found on Clear-Com's corporate website:

www.clearcom.com

Clear-Com Contacts

Americas and Asia-Pacific Headquarters

California, United States

Tel: +1.510.337.6600

Email: SalesSupportUS@clearcom.com

Europe, Middle East, and Africa Headquarters

Cambridge, United Kingdom

Tel: +44 1223 815000

Email: SalesSupportEMEA@clearcom.com

China Office

Beijing Representative Office

Beijing, P.R.China

Tel: +8610 59000198/59002608

Email: SalesSupportChina@clearcom.com

END USER LICENSE AGREEMENT

PLEASE READ THIS END USER LICENSE AGREEMENT ("Agreement"). BY DOWNLOADING OR INSTALLING SOFTWARE (DEFINED BELOW), OR BY USING HARDWARE (DEFINED BELOW), YOU AND ANY ENTITY YOU REPRESENT ("Licensee") AGREE TO BE BOUND BY THIS AGREEMENT WITH CLEAR-COM LLC ("Licensor"). IF LICENSEE DOES NOT AGREE TO ALL OF THE TERMS OF THIS AGREEMENT, THEN DO NOT DOWNLOAD, INSTALL OR USE THE SOFTWARE.

This Agreement governs Licensee's use of the Software unless the particular software is subject to a separate written agreement between Licensor and Licensee that is signed by Licensor.

1. Definitions.

1.1. "Authorized Modules" means those modules of the Software identified by Licensor in writing designated as authorized for Licensee's use and for which Licensee has paid the applicable fees.

1.2. "Documentation" means the then-current printed, online, and/or electronic documentation, if any, that is provided by Licensor to Licensee describing the use of the Software and/or Hardware. "Documentation" does not include any sales or marketing materials.

1.3. "Effective Date" means the date that Licensee installs or otherwise uses any Software or Hardware, as may be updated from time to time.

1.4. "Exclusions" is defined in Section 5.4 (Exclusions).

1.5. "Evaluation Software" means any Software provided to Licensee for evaluation purposes or otherwise designated by Licensor as evaluation software, as may be provided by Licensor from time to time in Licensor's sole discretion and as specified in a separate written agreement between Licensor and Licensee.

1.6. "Hardware" means the hardware sold or otherwise provided by Licensor to Licensee, which includes embedded Software.

1.7. "Improvements" means modifications, improvements, derivative works and the like.

1.8. "Intellectual Property Rights" means all intellectual property and proprietary rights throughout the world (including patents, trade secrets, copyrights and trademarks).

1.9. "Machine Data" means the data that is generated by any Hardware or Software in connection with Licensee's use of any Hardware.

1.10. "Personal Data" means any information relating to an identified or identifiable natural person.

1.11. "Software" means the software provided by Licensor for which Licensor references this Agreement (including software identified in Licensor's order form or quotation that references this Agreement and any updates Licensor provides to Licensee to such software for use in connection with it).

- 1.12. "Software Warranty" is defined in Section 5.1 (Software Warranty).
- 1.13. "Software Warranty Period" is defined in Section 5.1 (Software Warranty).
- 1.14. "Third Party Software" is defined in Section 2.7 (Third Party Software).
- 1.15. "Use Limitations" means the limitations on use of the Software provided by Licensor in writing on its website or in connection with Licensee's order and/or receipt of the Software (including in connection with download and/or as provided in connection with the order form provided by Licensor). Use Limitations includes, but is not limited to, "node-locked" Software licensed with respect to a specific or otherwise identifiable Hardware device and not to any other hardware device whatsoever, whether similar or dissimilar and prohibited from any transfer or assignment whatsoever without the prior written consent of Licensor.

2. License Grant and Intellectual Property Rights.

2.1. License. As of the Effective Date, subject to the terms and conditions of this Agreement (including payment and Section 2.3 (License Restrictions)) and irrespective of whether Hardware is sold to or rented by Licensee, Licensor grants Licensee a non-exclusive, non-transferable, non-assignable, non-sublicensable and limited license, only in object code form, to (a) the number of copies of Software, whether downloaded or installed Software or embedded on a component or system of Hardware, purchased by Licensee and (b) use such Software in accordance with the Documentation and Use Limitations solely for Licensee's internal business purposes. For the avoidance of doubt and notwithstanding anything to the contrary in this Agreement or anywhere else, Software is licensed not sold, and the license to Software is only the express, written license agreement for such software provided by Licensor to Licensee.

2.2. Evaluation License. All terms and provisions of this Agreement shall apply to Evaluation Software, subject to the following modifications: (a) the license of Section 2.1 (License) shall be limited to Licensee's evaluation of the Evaluation Software only; (b) Licensor shall have the right at any time in its sole discretion to terminate this Agreement and any license to Evaluation Software upon written notice to Licensee; and (c) the provision of Section 5 (Limited Warranty) shall not apply with respect to any Evaluation Software and all Evaluation Software is provided "as is" without any warranties whatsoever, express or implied.

2.3. License Restrictions. The licenses under Section 2 (License Grant and Intellectual Property Rights) are conditioned on Licensee's compliance with, and Licensee agrees to, the following: (a) Licensee shall not use the Software outside of the scope of the license granted hereunder; (b) Licensee shall not sublicense, distribute, disclose, market, rent, lease or transfer the Software, the Documentation or any portion thereof to any third party (including through any remote access, interactive, Internet-based, service bureau or other on line service); (c) Licensee shall not remove or cause to be removed from any copies of the Software, the Documentation or any portion thereof any copyright, licensed trademark, patent or other proprietary notices thereon; (d) Licensee shall not modify, alter, adapt, translate, reverse-engineer, decompile, disassemble or attempt to discover the source code, underlying ideas,

algorithms, file formats or programming interfaces of the Software, the Documentation or any portion thereof; (v) Licensee shall not export or re-export the Software, either directly or indirectly, without Licensor's prior written consent or in violation of the laws of the United States or other jurisdiction; and (vi) Licensee shall not use any aspects of the Software other than the Authorized Modules, and without limiting the foregoing, Licensee shall use the Software only in accordance with the applicable Documentation, license key and any Use Limitations and only within the scope for which Licensee is paying the required fees. Licensee agrees that the Software may use or contain a license key mechanism limiting its use, and Licensee shall not use or enable the use of the Software other than in accordance with the intended limitations of any license key provided by Licensor (in addition to any contractual limitations). Licensee agrees that, without limitation, any breach of this Section 2.3 (License Restrictions) or unauthorized use of the Software or Documentation is a material breach of this Agreement.

2.4. **Export, Import and Government Restrictions.** Licensee is advised that the Software may be subject to export laws and the laws of the country where it is delivered or used. Licensee shall abide by all applicable export laws and regulations. Under such laws and regulations, the Software may not be sold, leased or transferred to certain restricted countries, restricted end-users or for restricted end-uses.

2.5. **Ownership by Licensor.** Licensor retains all Intellectual Property Rights covering or embodied in the Software and any Improvements to any Software. The non-public aspects of the Software (including any source code of the Software) are Licensor's confidential information. Licensee shall maintain the confidentiality of Licensor's confidential information and not disclose such confidential information to any third party without Licensor's prior written consent. To the extent that Licensor grants Licensee use of or access to Hardware or Software outside of the scope of the license in Section 2.1 (License) (e.g. access to source code), the terms of such use or access shall be set forth in a separate written addendum to this Agreement.

2.6. **License Only.** The Software is licensed, not sold, to Licensee. This Agreement does not transfer any right, title, or interest in or to any such Software regardless of any use of the terms "purchase," "acquire," "sale" or similar language herein or in any agreement between Licensee and any third party.

2.7. **Third Party Software.** The Software may use or rely on software that is subject to open source or commercial license terms as identified by Licensor in writing on its website or connection with Licensee's order and/or receipt of the Software ("Third Party Software"). Third Party Software is provided "as is" without any warranty, express, implied or otherwise.

2.8. **No Other Rights.** All Intellectual Property Rights of Licensor not expressly granted to Licensee in this Agreement are expressly reserved by Licensor. Without limitation, Licensee receives no right or license, by implication, estoppel or otherwise, to any software, technology or Intellectual Property Rights not

embodied in the Software, even if such other software, technology or Intellectual Property Rights are useful or necessary in connection with the Software.

3. Data.

3.1. Ownership of Machine Data. Licensor retains all Intellectual Property Rights covering or embodied in the Machine Data and any Improvements to any Machine Data. In the event that Licensee acquires any rights in or to any Machine Data, Improvements thereto or Intellectual Property Rights covering or embodying the Machine Data or Improvements to the Machine Data, Licensee shall assign, and hereby assigns, all right, title and interest in such Improvements and Intellectual Property Rights to Licensor. Licensor grants Licensee a non-exclusive, non-transferable, non-assignable, non-sublicensable, royalty-free, fully paid-up and limited license to use such Machine Data for Licensee's internal business purposes.

3.2. Transfer and Access to Machine Data. Machine Data may be transferred automatically or as facilitated by Licensor or Licensee's representatives, from Hardware to Licensor, from time to time, and Licensee hereby acknowledges and agrees to such transfer of Machine Data. To the extent that any such Machine Data cannot be automatically transferred to Licensor or is not otherwise not conducive for such automatic transfer, Licensee shall transfer such Machine Data to Licensor, whether in-person or pursuant to a separate network transfer protocol or procedures, at a time, location or pursuant to procedures mutually acceptable to Licensee and Licensor.

3.3. Personal Data. Information on the Personal Data collected by Licensor from Licensee, which may be included in Machine Data, and how Licensor treats that Personal Data is set out in Licensor's privacy policy, which can be found at <https://www.clearcom.com/privacy-policy/>.

4. Payment. The terms of this Section 4 (Payment) shall apply to the extent that there are no conflicting provisions in a separate written agreement (e.g. terms of sale) between Licensor and Licensee.

4.1. Payment. All fees for the Software are due and payable in U.S. dollars by bank wire transfer in immediately available funds to a bank account designated by Licensor. If not otherwise provided in the respective schedule or other documentation from Licensor, then Licensee shall pay amounts due within thirty (30) days of invoice.

4.2. Records and Audit Rights. Licensee shall keep and maintain books and records in sufficient detail to permit the verification of Licensee's obligations hereunder. Licensor shall have the right to inspect or appoint an auditor to inspect Licensee's systems, books and records relevant to this Agreement and other materials as may be required to verify or determine Licensee's compliance with its obligations hereunder during regular business hours, wherever they are normally kept by Licensee, upon not less than one (1) week's prior written notice. The cost of any such audit shall be paid for by Licensor unless material non-compliance is found (in which case, Licensee shall pay Licensor for the costs associated with the audit).

4.3. Late Payment. All late payments shall be subject to interest calculated at the rate of one and one half percent (1.5%) per month or the maximum allowable by law, whichever is less. This Section 4.3 (Late Payment) shall in no way limit any other rights or remedies that may otherwise be available to Licensor. Licensee shall pay all expenses associated with collection (including reasonable attorneys' fees).

4.4. Taxes. All stated prices, fees and other stated charges are exclusive of applicable sales, excise, VAT, use or similar taxes. Licensee shall pay all such taxes, either directly or to Licensor, as required by applicable law or regulation. In the event that Licensee is required to withhold taxes, Licensee shall provide to Licensor official receipts issued by the appropriate taxing authority or other evidence to establish that such taxes have been paid.

4.5. Payment Terms. Licensor shall have the right to require advance payment and require adequate assurance of payment in the event that Licensee fails to make payment within the time limits required hereunder or, as determined in Licensor's sole discretion, it is advisable in view of Licensee's financial condition. The foregoing does not limit Licensor's other remedies hereunder (including the right to terminate this Agreement for breach).

5. Limited Warranty.

5.1. Hardware Warranty. Warranties for Hardware are set forth in a separate written agreement between Licensor and Licensee or as an addendum to this Agreement, as applicable.

5.2. Software Warranty. Unless otherwise set forth in a separate written agreement (e.g. terms of sale) between Licensor and Licensee, Licensor warrants to Licensee that, for a period of ninety (90) days following the delivery of the initial Software to Licensee (the "Software Warranty Period"), the Software will substantially conform to the Documentation when maintained and operated in accordance with the Documentation ("Software Warranty"). Following the Software Warranty Period, Licensor may provide maintenance, training and support of Software as set forth in a separate written agreement between Licensor and Licensee or as an addendum to this Agreement, as applicable.

5.3. Remedy. Licensee shall provide Licensor notice during the Software Warranty Period identifying any non-conformity to Software Warranty and shall provide detail as requested by Licensor to identify the nature of the non-conformity and conditions under which it appears. Licensor, at its sole option, shall use commercially reasonable efforts to repair or to deliver a replacement copy of the Software. All warranty repairs by Licensor shall be made at Licensor's premises, unless Licensor otherwise agrees. Licensee's sole remedy and Licensor's sole liability with respect to any warranty regarding the Software shall be for Licensor to use such commercially reasonable efforts to provide the above correction during the Software Warranty Period.

5.4. Exclusions. Licensor provides no warranty for the following or any liability or claim arising from any of the following (collectively, "Exclusions"): (a) software and equipment of Licensee or any third party (including any Third Party Software); (b) directions, designs, plans or specifications furnished by or on

behalf of Licensee; (c) unauthorized use or use of the Software in violation of this Agreement or other terms with Licensor (including any copies of the Software not made in accordance with the terms of this Agreement and the limitations of the licenses granted hereunder); (d) any data provided by Licensee or third party; (e) any activities of Licensee or third party through the use of the Software; (f) modification of the Software made other than by Licensor; (g) the combination, operation or use by Licensee of the Software with equipment, devices or software not supplied by Licensor; (h) failure of Licensee to use an updated or modified Software provided by Licensor (including those provided to avoid error or infringement); or (i) Licensee's use of the Software after termination of this Agreement.

5.5. **Warranty Disclaimer.** THE FOREGOING LIMITED WARRANTIES AND REMEDIES PROVIDED BY LICENSOR ARE THE SOLE AND EXCLUSIVE REMEDIES FOR ANY BREACH OF WARRANTY. EXCEPT TO THE EXTENT EXPRESSLY SET FORTH ABOVE, LICENSOR DISCLAIMS ALL WARRANTIES, CONDITIONS, AND REPRESENTATIONS (EXPRESS OR IMPLIED, ORAL OR WRITTEN) WITH RESPECT TO THE SOFTWARE OR ANY SUPPORT RELATED THERETO (INCLUDING ANY AND ALL WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON INFRINGEMENT, NON INTERFERENCE, ACCURACY OF DATA, OPERATION WITHOUT INTERRUPTION OR ERROR, FREEDOM FROM VULNERABILITY TO INTRUSION OR ATTACK AND INTEROPERABILITY WITH ANY HARDWARE, SOFTWARE, SYSTEMS OR DATA NOT PROVIDED BY LICENSOR, AS WELL AS ANY AND ALL WARRANTIES ARISING FROM A COURSE OF DEALING, AND ALL SUCH WARRANTIES ARE EXCLUDED TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW.

6. **Indemnification.** Licensee shall indemnify and hold harmless Licensor at Licensee's expense from any action brought against Licensor to the extent that it is based on a claim arising out of or relating to any of the Exclusions, and Licensee shall pay any costs and damages finally awarded against Licensor in any such actions which are attributable to such claim.

7. **Limitation of Liability.**

7.1. **Limitation of Liability.** LICENSOR SHALL NOT BE LIABLE FOR ANY SPECIAL, INDIRECT, INCIDENTAL, EXEMPLARY OR CONSEQUENTIAL DAMAGES (INCLUDING LOST PROFITS, BUSINESS INTERRUPTION OR LOSS OF INFORMATION) RELATING TO THIS AGREEMENT OR IN ANY WAY RELATED TO THIS AGREEMENT, REGARDLESS OF WHETHER LICENSOR WAS ADVISED OF THE POSSIBILITY OF ANY OF THE FOREGOING. IN NO EVENT SHALL THE TOTAL COLLECTIVE LIABILITY OF LICENSOR FOR ALL CLAIMS HEREUNDER OR IN ANY WAY RELATED TO THIS AGREEMENT EXCEED AMOUNTS PAID BY LICENSEE FOR THE APPLICABLE SOFTWARE GIVING RISE TO THE CLAIM IN THE LAST TWELVE (12) MONTHS UNDER THIS AGREEMENT BEFORE THE EARLIER OF THE MAKING OF THE CLAIM OR TERMINATION OR END OF THE TERM OF THIS AGREEMENT.

7.2. **Allocation of Risk.** The warranty disclaimer and limitations of liability set forth in this Agreement shall apply irrespective of any failure of the essential purpose of any limited remedy. Licensee and Licensor each acknowledge and agree that the limitation of liability provisions of this Agreement reflect an

informed, voluntary allocation between them of the risk associated with the exercise of Licensee's rights regarding the Software and, but for these provisions, Licensor would not have made the Software available to Licensee under the terms contemplated under this Agreement.

7.3. Applicable Law. The warranty disclaimer and limitations of liability set forth in this Agreement shall not apply to the extent prohibited by law, in which case the disclaimer or limitation shall be modified to disclaim and/or limit in accordance with applicable law. Without limiting the foregoing, to the extent required by law, the foregoing limitations shall not apply to claims due to fraud, bodily injury or death.

8. Term and Termination.

8.1. Term. Unless terminated earlier as set forth below, this Agreement shall become effective upon the Effective Date and shall continue for the license term stated by Licensor in writing on the Licensor website or in connection with Licensee's order and/or receipt of the Software.

8.2. Termination for Cause. Without prejudice to any other right or remedy which may be available to it, each party may terminate this Agreement upon written notice to the other upon the other party's material breach of this Agreement that is incurable or, if curable, remains uncured thirty (30) days after notice to the other party or, in the case of failure to make payment, remains uncured ten (10) days after notice to the other party. Licensee's failure to pay amounts when due hereunder shall be deemed a material breach.

8.3. Termination for Bankruptcy. Each party may terminate this Agreement immediately upon written notice to the other upon the other party (a) becoming insolvent or bankrupt; (b) dissolving or ceasing to conduct business in the ordinary course; (c) making an assignment for the benefit of its creditors; (d) commencing any insolvency, receivership, bankruptcy or other similar proceeding for the settlement of its debts; or (e) having commenced against it any insolvency, receivership, bankruptcy or other similar proceeding for the settlement of its debts that is not dismissed within thirty (30) days after notice of such proceeding.

8.4. Return or Destruction of Software. Within fifteen (15) days after termination of this Agreement, Licensee shall certify in writing to Licensor that all copies of the Software and Documentation in any form (including partial copies) have been destroyed or returned to Licensor.

8.5. Effect of Termination. If this Agreement is terminated for any reason, then all rights granted hereunder to Licensee shall terminate. The provisions of Sections 1 (Definitions), 2.3 (License Restrictions), 2.4 (Export, Import and Government Restrictions), 2.5 (Ownership by Licensor), 2.7 (Third Party Software), 3 (Data), 4 (Payment), 5.4 (Exclusions), 5.5 (Warranty Disclaimer), 6 (Indemnification), 7 (Limitation of Liability), 8 (Term and Termination) and 9 (Miscellaneous) shall survive any termination or expiration of this Agreement.

9. Miscellaneous.

9.1. Relationship of Parties. The parties to this Agreement are independent contractors and this Agreement shall not establish any relationship of partnership, joint venture, employment, franchise, or agency between the parties. Neither party shall have the power to bind the other or incur obligations on the other's behalf without the other's prior written consent.

9.2. Assignment. Licensee shall not have the right to assign, transfer or delegate this Agreement, in whole or in part, without Licensor's prior written consent and any attempt thereto shall be null and void.

9.3. Force Majeure. Nonperformance of any party (other than with respect to payment obligations) shall be excused to the extent that performance is rendered impossible by strike, fire, earthquake, flood, governmental acts or orders or restrictions, failure of suppliers or any other reason where failure to perform is beyond the reasonable control of the nonperforming party.

9.4. Federal Acquisition. This provision applies to all acquisitions of the Software by or for the Federal Government, whether by any prime contractor or subcontractor (at any tier) and whether under any procurement contract, grant, cooperative agreement, or other activity by or with the Federal Government. By accepting delivery of the Software, the Government agrees that the Software qualifies as "commercial items" as that term is defined at 48 C.F.R. 2.101 and that the software and documentation delivered by Licensor consists of "commercial computer software" and "commercial computer software documentation" as such terms are used in 48 C.F.R. 12.212. Consistent with 48 C.F.R. 12.212 and 48 C.F.R. 227.7202-1 through 227.7202-4, all Federal Government end users acquire the software and documentation with only those rights set forth herein. The terms and conditions of this Agreement shall pertain to the Federal Government's use and disclosure of the software, and shall supersede any conflicting contractual terms or conditions. Disputes with the Federal Government shall be subject to resolution pursuant to the Contract Disputes Act of 1978, as amended. If this Agreement fails to meet the Government's needs or is inconsistent in any respect with Federal law, then the Federal Government shall return the Software, unused, to Licensor.

9.5. Governing Law. This Agreement shall be governed by, and construed in accordance with, the laws of the State of California, USA, applicable to contracts made in and fully performed in the State of California, USA, without reference to conflict of law or choice of law principles that would cause the application of laws of any other jurisdiction. The United Nations Convention on Contracts for the International Sales of Goods shall not apply to this Agreement. The use of the words "include," "includes" and "including" followed by one or more examples is intended to be illustrative and does not limit the scope of the description or term for which the examples are provided. In any action or proceeding under which a party enforces its respective rights hereunder, the non-prevailing party agrees to pay all costs incurred by the prevailing party therein (including reasonable attorneys' fees).

9.6. Severability. If any provision of this Agreement or the application thereof is for any reason, and to any extent thereof, determined by a court of competent jurisdiction to be invalid or unenforceable under applicable law, then a valid

provision that most closely matches the intent of the original shall be substituted and the remaining provisions of this Agreement shall be interpreted so as best to reasonably effect its original intent.

9.7. Waiver. The failure by either party to enforce any provision of this Agreement shall not constitute a waiver of future enforcement of that or any other provision. No waiver by either party of any such provision shall be deemed to have been made unless expressed in writing and signed by such party.

9.8. Notices. All notices required or permitted under this Agreement shall be in writing and delivered by courier, overnight delivery service or by certified mail and in each instance shall be deemed given upon receipt. All notices shall to Licensor be sent to the addresses set forth below or to such other address as may be specified by Licensor on its website or otherwise in writing. Licensor may provide notices to Licensee at the address provided by Licensee in connection with its order or otherwise as changed by Licensee by notice to Licensor in accordance with this clause.

Notices to Licensor:

Clear Com LLC
2848 Whiptail Loop
Carlsbad, CA 92010 USA
Attention: General Counsel

9.9. Effect of Purchase Orders and Other Documents. This Agreement is intended by Licensor and Licensee to operate as a basic set of operating conditions regarding the license of the Software and related services. Purchase orders or other documents may be used for convenience to identify the order information and any other terms of purchase orders are hereby rejected. Unless otherwise expressly provided herein, this Agreement shall prevail over any conflicting or additional terms or conditions of any purchase order or other document exchanged in connection with the ordering of the Software and related services. Any such conflicting or additional terms of Licensee are hereby deemed to be material alterations and notice of objection to and rejection of them is hereby given.

9.10. Entire Agreement. This Agreement contains the complete understanding and agreement of the parties and supersedes all prior or contemporaneous agreements or understandings, oral or written, relating to the subject matter herein. Any modification or amendment of any provision of this Agreement shall be effective only if in writing and signed by duly authorized representatives of the parties. A facsimile or e-mail transmission of a signed version of this Agreement shall be legal and binding on the applicable party.

CONTENTS

Document Reference	i
1	INTRODUCTION TO HCI REFERENCE49
1.1	WHAT IS HCI? 49
1.2	WHO USES HCI? 49
1.3	WHAT HCI COMPRISES 49
1.4	TYPE OF MESSAGE 50
1.5	SCOPE AND LIMITATIONS 50
1.6	SCOPE OF THIS DOCUMENT..... 50
2	OVERVIEW52
2.1	START UP 52
2.2	THE RUNNING STATE..... 52
2.3	REPLIES AND REQUESTS..... 52
2.4	BROADCAST SYSTEM MESSAGES..... 53
2.5	SYSTEM FEATURES..... 53
2.6	CROSSPOINTS..... 53
2.7	CONFERENCES..... 54
2.8	GENERAL PURPOSE INPUTS AND OUTPUTS 55
2.9	AUDIO LEVEL CONTROL 55
2.10	KEY ASSIGNMENT 56
2.11	CONFIGURATION FILES..... 56
2.12	ACTION STATUS 56
2.13	SENDING MULTIPLE COMMANDS 56
3	TRANSPORT PROTOCOLS.....57
3.1	ETHERNET LINK 57
3.1.1	Packets Structure..... 57
3.1.2	Connection Ports..... 57
3.1.3	Ping Facility..... 57
3.1.4	Client Session Closure 58
4	MESSAGES.....59
4.1	INTRODUCTION 59

4.2	FLAGS	59
4.2.1	Host	59
4.2.2	CSU	60
4.3	Protocol identification.....	61
4.4	Slot numbering scheme.....	61
4.5	FIELD SIZE LIMITATIONS	62
4.6	HOW TO INTERPRET THE DATA MESSAGE DESCRIPTIONS	62
4.6.1	Dial code format	64
4.7	Broadcast System Message.....	66
4.7.1	Description.....	66
4.7.2	Structure	66
4.7.3	Usage	66
4.7.4	Scope	66
4.7.5	Performance.....	66
4.7.6	Example	66
4.7.7	Special notes.....	67
4.7.8	Related messages	67
4.7.9	See also.....	67
4.8	Request Actions Status	68
4.8.1	Description.....	68
4.8.2	Structure	68
4.8.3	Usage	68
4.8.4	Scope	69
4.8.5	Performance.....	69
4.8.6	Example	69
4.8.7	Special notes.....	69
4.8.8	Related messages	69
4.8.9	See also.....	69
4.9	Reply Actions Status.....	70
4.9.1	Description.....	70
4.9.2	Structure	70
4.9.3	Usage	70
4.9.4	Scope	70
4.9.5	Performance.....	70
4.9.6	Example	70

4.9.7	Special notes	70
4.9.8	Related messages	70
4.9.9	See also.....	70
4.10	Request Conference Actions	71
4.10.1	Description.....	71
4.10.2	Structure	71
4.10.3	Usage.....	72
4.10.4	Scope.....	72
4.10.5	Performance.....	72
4.10.6	Example	72
4.10.7	Special notes.....	72
4.10.8	Related messages	72
4.10.9	See also.....	72
4.11	Request Conference Status	73
4.11.1	Description.....	73
4.11.2	Structure	73
4.11.3	Usage.....	73
4.11.4	Scope.....	73
4.11.5	Performance.....	73
4.11.6	Example	73
4.11.7	Special notes.....	73
4.11.8	Related messages	74
4.11.9	See also.....	74
4.12	Reply Conference Status	75
4.12.1	Description.....	75
4.12.2	Structure	75
4.12.3	Usage.....	76
4.12.4	Scope.....	76
4.12.5	Performance.....	76
4.12.6	Example	76
4.12.7	Special notes.....	76
4.12.8	Related messages	76
4.12.9	See also.....	76
4.13	Request Crosspoint Actions	77
4.13.1	Description.....	77

4.13.2	Structure	77
4.13.3	Usage	78
4.13.4	Scope	78
4.13.5	Performance.....	79
4.13.6	Example	79
4.13.7	Special notes.....	79
4.13.8	Related messages	79
4.13.9	See also.....	79
4.14	Request Crosspoint Status	80
4.14.1	Description.....	80
4.14.2	Structure	80
4.14.3	Usage	80
4.14.4	Scope	80
4.14.5	Performance.....	80
4.14.6	Example	80
4.14.7	Special notes.....	81
4.14.8	Related messages	81
4.14.9	See also.....	81
4.15	Reply Crosspoint Status	82
4.15.1	Description.....	82
4.15.2	Structure	82
4.15.3	Usage	83
4.15.4	Scope	83
4.15.5	Performance.....	83
4.15.6	Example	83
4.15.7	Special notes.....	83
4.15.8	Related messages	83
4.15.9	See also.....	83
4.16	Request Crosspoint Level Actions	84
4.16.1	Description.....	84
4.16.2	Structure	84
4.16.3	Usage	84
4.16.4	Scope	84
4.16.5	Performance.....	85
4.16.6	Example	85

4.16.7	Special notes	85
4.16.8	Related messages	85
4.17	Request Crosspoint Level Status	86
4.17.1	Description	86
4.17.2	Structure	86
4.17.3	Usage	86
4.17.4	Scope	86
4.17.5	Performance	86
4.17.6	Example	86
4.17.7	Special notes	87
4.17.8	Related messages	87
4.17.9	See also	87
4.18	Reply Crosspoint Level Status	88
4.18.1	Description	88
4.18.2	Structure	88
4.18.3	Usage	88
4.18.4	Scope	88
4.18.5	Performance	89
4.18.6	Example	89
4.18.7	Special notes	89
4.18.8	Related messages	89
4.18.9	See also	89
4.19	Request CPU Reset	90
4.19.1	Description	90
4.19.2	Structure	90
4.19.3	Usage	90
4.19.4	Scope	91
4.19.5	Performance	91
4.19.6	Example	91
4.19.7	Special notes	91
4.19.8	Related messages	91
4.19.9	See also	91
4.20	Request Unicode Alias Add	92
4.20.1	Description	92
4.20.2	Structure	92

4.20.3	Usage	93
4.20.4	Scope	93
4.20.5	Performance.....	93
4.20.6	Example	93
4.20.7	Special notes.....	93
4.20.8	Related messages	93
4.20.9	See also.....	93
4.21	Reply Unicode Alias Status.....	94
4.21.1	Description.....	94
4.21.2	Structure	94
4.21.3	Usage.....	95
4.21.4	Scope	95
4.21.5	Performance.....	95
4.21.6	Example	95
4.21.7	Special notes.....	95
4.21.8	Related messages	95
4.21.9	See also.....	95
4.22	Request Unicode Alias List	96
4.22.1	Description.....	96
4.22.2	Structure	96
4.22.3	Usage.....	96
4.22.4	Scope	96
4.22.5	Performance.....	96
4.22.6	Example	96
4.22.7	Special notes.....	96
4.22.8	Related messages	96
4.22.9	See also.....	97
4.23	Request Alias Delete	98
4.23.1	Description.....	98
4.23.2	Structure	98
4.23.3	Usage.....	99
4.23.4	Scope	99
4.23.5	Performance.....	99
4.23.6	Example	99
4.23.7	Special notes.....	99

4.23.8	Related messages	99
4.23.9	See also.....	99
4.24	Reply Alias Delete	100
4.24.1	Description.....	100
4.24.2	Structure	100
4.24.3	Usage.....	101
4.24.4	Scope.....	101
4.24.5	Performance.....	101
4.24.6	Example	101
4.24.7	Special notes.....	101
4.24.8	Related messages	101
4.24.9	See also.....	101
4.25	Request EHX Control Actions.....	102
4.25.1	Description.....	102
4.25.2	Structure	102
4.25.3	Usage.....	103
4.25.4	Scope.....	103
4.25.5	Performance.....	104
4.25.6	Example	104
4.25.7	Special notes.....	104
4.25.8	Related messages	104
4.25.9	See also.....	104
4.26	Request EHX Control Card Status.....	105
4.26.1	Description.....	105
4.26.2	Structure	105
4.26.3	Usage Scope	105
4.26.4	Performance.....	105
4.26.5	Example	105
4.26.6	Related messages	105
4.26.7	See also.....	105
4.27	Reply EHX Controls Card Status.....	106
4.27.1	Description.....	106
4.27.2	Structure	106
4.27.3	Usage.....	106
4.27.4	Scope.....	106

4.27.5	Related Messages	107
4.27.6	See also.....	107
4.28	Request GPIO/SFO Status.....	108
4.28.1	Description.....	108
4.28.2	Structure	108
4.28.3	Usage.....	108
4.28.4	Scope.....	108
4.28.5	Performance.....	109
4.28.6	Example	109
4.28.7	Special notes.....	109
4.28.8	Related messages	109
4.28.9	See also.....	109
4.29	Reply GPIO/SFO Status	110
4.29.1	Description.....	110
4.29.2	Structure	110
4.29.3	Usage.....	111
4.29.4	Scope.....	111
4.29.5	Performance.....	111
4.29.6	Example	111
4.29.7	Special notes.....	111
4.29.8	Related messages	111
4.29.9	See also.....	111
4.30	Request Input Level Actions	112
4.30.1	Description.....	112
4.30.2	Structure	112
4.30.3	Usage.....	112
4.30.4	Scope.....	112
4.30.5	Performance.....	113
4.30.6	Example	113
4.30.7	Special notes.....	113
4.30.8	Related messages	113
4.30.9	See also.....	113
4.31	Request Input Level Status	114
4.31.1	Description.....	114
4.31.2	Structure	114

4.31.3	Usage	114
4.31.4	Scope	114
4.31.5	Performance.....	114
4.31.6	Example	114
4.31.7	Special notes.....	115
4.31.8	Related messages	115
4.31.9	See also.....	115
4.32	Reply Input Level Status	116
4.32.1	Description.....	116
4.32.2	Structure	116
4.32.3	Usage.....	116
4.32.4	Scope	117
4.32.5	Performance.....	117
4.32.6	Example	117
4.32.7	Special notes.....	117
4.32.8	Related messages	117
4.32.9	See also.....	117
4.33	Request Output Level Actions	118
4.33.1	Description.....	118
4.33.2	Structure	118
4.33.3	Usage.....	118
4.33.4	Scope	118
4.33.5	Performance.....	119
4.33.6	Example	119
4.33.7	Special notes.....	119
4.33.8	Related messages	119
4.33.9	See also.....	119
4.34	Request Output Level Status	120
4.34.1	Description.....	120
4.34.2	Structure	120
4.34.3	Usage Scope	120
4.34.4	Performance.....	120
4.34.5	Example	120
4.34.6	Special notes.....	120
4.34.7	Related messages	121

4.34.8	See also.....	121
4.35	Reply Output Level Status	122
4.35.1	Description.....	122
4.35.2	Structure	122
4.35.3	Usage.....	122
4.35.4	Scope.....	122
4.35.5	Performance.....	123
4.35.6	Example	123
4.35.7	Special notes.....	123
4.35.8	Related messages	123
4.35.9	See also.....	123
4.36	Request Panel Status	124
4.36.1	Description.....	124
4.36.2	Structure	124
4.36.3	Usage.....	124
4.36.4	Scope.....	124
4.36.5	Performance.....	124
4.36.6	Example	124
4.36.7	Special notes.....	125
4.36.8	Related messages	125
4.36.9	See also.....	125
4.37	Reply Panel Status	126
4.37.1	Description.....	126
4.37.2	Structure	126
4.37.3	Usage.....	127
4.37.4	Scope.....	127
4.37.5	Performance.....	127
4.37.6	Example	127
4.37.7	Special notes.....	127
4.37.8	Related messages	127
4.37.9	See also.....	127
4.38	Request System Messages.....	128
4.38.1	Description.....	128
4.38.2	Structure	128
4.38.3	Usage.....	128

4.38.4	Scope	128
4.38.5	Performance.....	129
4.38.6	Example	129
4.38.7	Special notes.....	129
4.38.8	Related messages	129
4.39	Request System Status	130
4.39.1	Description.....	130
4.39.2	Structure	130
4.39.3	Usage.....	130
4.39.4	Performance.....	130
4.39.5	Scope	130
4.39.6	Example	130
4.39.7	Special notes.....	131
4.39.8	Related messages	131
4.39.9	See also.....	131
4.40	Reply System Card Status	132
4.40.1	Description.....	132
4.40.2	Structure	132
4.40.3	Usage.....	133
4.40.4	Performance.....	133
4.40.5	Scope	133
4.40.6	Special notes.....	133
4.40.7	Related messages	133
4.40.8	See also.....	134
4.41	Request Panel Keys Status.....	135
4.41.1	Description.....	135
4.41.2	Structure	135
4.41.3	Usage.....	135
4.41.4	Scope	135
4.41.5	Performance.....	135
4.41.6	Example	136
4.41.7	Related messages	136
4.41.8	See also.....	136
4.42	Reply Panel Keys Status	137
4.42.1	Description.....	137

4.42.2	Structure	137
4.42.3	Usage	138
4.42.4	Scope	138
4.42.5	Performance.....	138
4.42.6	Special notes.....	138
4.42.7	Related messages	138
4.42.8	See also.....	138
4.43	Request Panel Keys Action.....	139
4.43.1	Description.....	139
4.43.2	Structure	139
4.43.3	Usage	140
4.43.4	Scope	140
4.43.5	Special notes.....	140
4.43.6	Performance.....	140
4.43.7	Related messages	140
4.43.8	See also.....	140
4.44	Reply Panel Keys Status	141
4.44.1	Description.....	141
4.44.2	Structure	141
4.44.3	Usage	141
4.44.4	Scope	141
4.44.5	Special notes.....	142
4.44.6	Performance.....	142
4.44.7	Example	142
4.44.8	Related messages	142
4.44.9	See also.....	142
4.45	Request Port Info	143
4.45.1	Description.....	143
4.45.2	Structure	143
4.45.3	Usage	143
4.45.4	Scope	143
4.45.5	Performance.....	144
4.45.6	Example	144
4.45.7	Related messages	144
4.45.8	See also.....	144

4.46	Reply Port Info.....	145
4.46.1	Description.....	145
4.46.2	Structure	145
4.46.3	Usage.....	147
4.46.4	Scope.....	148
4.46.5	Special notes.....	148
4.46.6	Performance.....	148
4.46.7	Related messages	148
4.46.8	See also.....	148
4.47	Request Locally Assigned Keys	149
4.47.1	Description.....	149
4.47.2	Structure	149
4.47.3	Usage.....	149
4.47.4	Scope.....	150
4.47.5	Performance.....	150
4.47.6	Example	150
4.47.7	Related messages	150
4.47.8	See also.....	150
4.48	Reply Locally Assigned Keys.....	151
4.48.1	Description.....	151
4.48.2	Structure	151
4.48.3	Usage.....	154
4.48.4	Scope.....	154
4.48.5	Special notes.....	154
4.48.6	Performance.....	154
4.48.7	Example	154
4.48.8	Related messages	154
4.48.9	See also.....	154
4.49	Request Assigned Keys	155
4.49.1	Description.....	155
4.49.2	Structure	155
4.49.3	Usage.....	155
4.49.4	Scope.....	155
4.49.5	Performance.....	156
4.49.6	Example	156

4.49.7	Related messages	156
4.49.8	See also.....	156
4.50	Reply Assigned Keys.....	157
4.50.1	Description.....	157
4.50.2	Structure	157
4.50.3	Usage.....	160
4.50.4	Scope.....	160
4.50.5	Special notes.....	160
4.50.6	Performance.....	160
4.50.7	Example	160
4.50.8	Related messages	160
4.50.9	See also.....	160
4.51	Request Card Info	161
4.51.1	Description.....	161
4.51.2	Structure	161
4.51.3	Usage.....	161
4.51.4	Scope.....	161
4.51.5	Related messages	161
4.51.6	Performance.....	161
4.51.7	Example	161
4.51.8	Related messages	162
4.51.9	See also.....	162
4.52	Reply Card Info.....	163
4.52.1	Description.....	163
4.52.2	Structure	163
4.52.3	Usage.....	165
4.52.4	Scope.....	165
4.52.5	Related messages	165
4.52.6	See also.....	166
4.53	Request Conference / Fixed Group Members Edits	167
4.53.1	Description.....	167
4.53.2	Structure	167
4.53.3	Usage.....	167
4.53.4	Scope.....	167
4.53.5	Performance.....	167

4.53.6	Example	167
4.53.7	Related messages	168
4.53.8	See also.....	168
4.54	Reply Conference Assignments	169
4.54.1	Description.....	169
4.54.2	Structure	169
4.54.3	Usage.....	170
4.54.4	Scope.....	170
4.54.5	Performance.....	170
4.54.6	Example	170
4.54.7	Related messages	170
4.54.8	See also.....	170
4.55	Set Config for Multiple Keys	171
4.55.1	Description.....	171
4.55.2	Structure	171
4.55.3	Usage.....	175
4.55.4	Scope.....	175
4.55.5	Special notes.....	175
4.55.6	Performance.....	175
4.55.7	Example	175
4.55.8	Related messages	175
4.55.9	See also.....	175
4.56	Reply Set Config for Multiple Keys.....	176
4.56.1	Description.....	176
4.56.2	Structure	176
4.56.3	Usage.....	177
4.56.4	Scope.....	177
4.56.5	Performance.....	177
4.56.6	Example	177
4.56.7	Related messages	177
4.56.8	See also.....	177
4.57	Request Forced Listen Edits	178
4.57.1	Description.....	178
4.57.2	Structure	178
4.57.3	Usage.....	178

4.57.4	Scope	178
4.57.5	Performance.....	178
4.57.6	Example	178
4.57.7	Related messages	178
4.57.8	See also.....	178
4.58	Reply Forced Listen Edits.....	179
4.58.1	Description.....	179
4.58.2	Structure	179
4.58.3	Usage.....	180
4.58.4	Scope.....	180
4.58.5	Performance.....	180
4.58.6	Example	180
4.58.7	Related messages	180
4.58.8	See also.....	180
4.59	Request Forced Listen Actions	181
4.59.1	Description.....	181
4.59.2	Structure	181
4.59.3	Usage.....	181
4.59.4	Scope.....	182
4.59.5	Performance.....	182
4.59.6	Example	182
4.59.7	Usage.....	182
4.59.8	Related messages	182
4.59.9	See also.....	182
4.60	Request Remote Key Actions.....	183
4.60.1	Description.....	183
4.60.2	Structure	183
4.60.3	Usage.....	185
4.60.4	Scope.....	185
4.60.5	Performance.....	185
4.60.6	Example	185
4.60.7	Special notes.....	186
4.60.8	Related messages	186
4.60.9	See also.....	186
4.61	Reply Remote Key Actions	187

4.61.1	Description.....	187
4.61.2	Structure	187
4.61.3	Usage.....	188
4.61.4	Scope.....	188
4.61.5	Performance.....	189
4.61.6	Example	189
4.61.7	Special notes.....	189
4.61.8	Related messages	189
4.61.9	See also.....	189
4.62	Request Remote Key Action Status.....	190
4.62.1	Description.....	190
4.62.2	Structure	190
4.62.3	Usage.....	190
4.62.4	Scope.....	190
4.62.5	Performance.....	191
4.62.6	Example	191
4.62.7	Special notes.....	191
4.62.8	Related messages	191
4.62.9	See also.....	191
4.63	Reply Remote Key Action Status	192
4.63.1	Description.....	192
4.63.2	Structure	192
4.63.3	Usage.....	193
4.63.4	Scope.....	193
4.63.5	Performance.....	194
4.63.6	Example	194
4.63.7	Related messages	194
4.63.8	See also.....	194
4.64	Request VoIP Status	195
4.64.1	Description.....	195
4.64.2	Scope.....	195
4.64.3	Structure	195
4.64.4	Usage.....	195
4.64.5	Scope.....	195
4.64.6	Performance.....	195

4.64.7	Example	195
4.64.8	Related messages	195
4.64.9	See also.....	195
4.65	Reply VOIP Status	196
4.65.1	Description.....	196
4.65.2	Structure	196
4.65.3	Additional Information	197
4.65.4	Status Enums	197
4.65.5	Usage.....	198
4.65.6	Scope.....	198
4.65.7	Performance.....	198
4.65.8	Example	198
4.65.9	Related messages	198
4.65.10	See also.....	198
4.66	Request to Change Telephone Hook State	199
4.66.1	Description.....	199
4.66.2	Structure	199
4.66.3	Usage.....	199
4.66.4	Scope.....	199
4.66.5	Performance.....	199
4.66.6	Example	199
4.66.7	Related messages	200
4.66.8	See also.....	200
4.66.9	Additional Information	200
4.67	Request Set System Time.....	201
4.67.1	Description.....	201
4.67.2	Structure	201
4.67.3	Usage.....	201
4.67.4	Scope.....	201
4.67.5	Performance.....	201
4.67.6	Example	201
4.67.7	Related messages	202
4.67.8	See also.....	202
4.67.9	Additional Information	202
4.68	Request Peripheral Info	203

4.68.1	Description.....	203
4.68.2	Structure	203
4.68.3	Usage.....	203
4.68.4	Scope.....	203
4.68.5	Performance.....	203
4.68.6	Example	203
4.68.7	Related messages	203
4.68.8	See also.....	204
4.69	Reply Peripheral Info	205
4.69.1	Description.....	205
4.69.2	Structure	205
4.69.3	Usage.....	206
4.69.4	Scope.....	206
4.69.5	Performance.....	206
4.69.6	Related messages	206
4.69.7	Additional Information	206
4.69.8	See also.....	206
4.70	Request IFB Status.....	207
4.70.1	Description.....	207
4.70.2	Structure	207
4.70.3	Additional Information	208
4.70.4	Usage.....	208
4.70.5	Scope.....	208
4.70.6	Performance.....	208
4.70.7	Example	208
4.70.8	Related messages	208
4.70.9	See also.....	208
4.71	Reply IFB Status	209
4.71.1	Description.....	209
4.71.2	Structure	209
4.71.3	Additional Information	211
4.71.4	Usage.....	212
4.71.5	Scope.....	212
4.71.6	Performance.....	212
4.71.7	Example	212

4.71.8	Related messages	212
4.71.9	See also.....	212
4.72	Request IFB Set.....	213
4.72.1	Description.....	213
4.72.2	Structure	213
4.72.3	Additional Information	215
4.72.4	Usage.....	215
4.72.5	Scope.....	215
4.72.6	Performance.....	216
4.72.7	Example	216
4.72.8	Related messages	216
4.72.9	See also.....	216
4.73	Request Entity Info.....	217
4.73.1	Description.....	217
4.73.2	Structure	217
4.73.3	Usage.....	217
4.73.4	Scope.....	217
4.73.5	Performance.....	217
4.73.6	Example	217
4.73.7	Related messages	218
4.73.8	See also.....	218
4.74	Reply Entity Info	219
4.74.1	Description.....	219
4.74.2	Structure	219
4.74.3	Additional Information	220
4.74.4	Usage.....	220
4.74.5	Scope.....	220
4.74.6	Performance.....	220
4.74.7	Example	220
4.74.8	Related messages	220
4.74.9	See also.....	220
4.75	Request VOX Threshold Status	221
4.75.1	Description.....	221
4.75.2	Structure	221
4.75.3	Usage.....	221

4.75.4	Scope	221
4.75.5	Performance.....	221
4.75.6	Example	221
4.75.7	Related messages	222
4.75.8	See also.....	222
4.76	Reply Vox Threshold Status	223
4.76.1	Description.....	223
4.76.2	Structure	223
4.76.3	Additional Information	223
4.76.4	Usage.....	224
4.76.5	Scope.....	224
4.76.6	Performance.....	224
4.76.7	Example	224
4.76.8	Related messages	224
4.76.9	See also.....	224
4.77	Request Vox Threshold set.....	225
4.77.1	Description.....	225
4.77.2	Structure	225
4.77.3	Additional Information	226
4.77.4	Usage.....	226
4.77.5	Scope.....	226
4.77.6	Performance.....	226
4.77.7	Example	226
4.77.8	Related messages	226
4.77.9	See also.....	226
4.78	Beltpack Status Reply	227
4.78.1	Description.....	227
4.78.2	Structure	227
4.78.3	Usage.....	228
4.78.4	Scope.....	228
4.78.5	Performance.....	228
4.78.6	Example	228
4.78.7	Related messages	228
4.78.8	See also.....	228
4.79	Reply Panel Status	229

4.79.1	Description.....	229
4.79.2	Structure	229
4.79.3	Additional Information	229
4.79.4	Usage.....	230
4.79.5	Scope	230
4.79.6	Performance.....	230
4.79.7	Example	230
4.79.8	Related messages	230
4.79.9	See also.....	230
4.80	Request SRecord Transmission Initiation.....	231
4.80.1	Description.....	231
4.80.2	Structure	231
4.80.3	Additional information	232
4.80.4	Usage.....	232
4.80.5	Scope	232
4.80.6	Performance.....	232
4.80.7	Example	232
4.80.8	Related messages	232
4.80.9	See also.....	232
4.81	SRecord Transmission	233
4.81.1	Description.....	233
4.81.2	Structure	233
4.81.3	Additional information	233
4.81.4	Usage.....	233
4.81.5	Scope	233
4.81.6	Performance.....	234
4.81.7	Example	234
4.81.8	Related messages	234
4.81.9	See also.....	234
4.82	Request Rack Properties: Active Configuration Bank.....	235
4.82.1	Description.....	235
4.82.2	Structure	235
4.82.3	Additional information	235
4.82.4	Usage.....	235
4.82.5	Scope	235

4.82.6	Performance.....	235
4.82.7	Example	235
4.82.8	Related messages	235
4.82.9	See also.....	235
4.83	Reply Rack Properties: Active Configuration Bank.....	236
4.83.1	Description.....	236
4.83.2	Structure	236
4.83.3	Additional information	236
4.83.4	Usage.....	236
4.83.5	Scope.....	236
4.83.6	Performance.....	236
4.83.7	Example	236
4.83.8	Related messages	236
4.83.9	See also.....	236
4.84	Request Rack Properties: Rack State Get	237
4.84.1	Description.....	237
4.84.2	Structure	237
4.84.3	Additional information	237
4.84.4	Usage.....	237
4.84.5	Scope.....	237
4.84.6	Performance.....	237
4.84.7	Example	237
4.84.8	Related messages	237
4.84.9	See also.....	237
4.85	Reply Rack Properties: Rack State Get.....	238
4.85.1	Description.....	238
4.85.2	Structure	238
4.85.3	Additional information	238
4.85.4	Usage.....	239
4.85.5	Scope.....	239
4.85.6	Performance.....	239
4.85.7	Example	239
4.85.8	Related messages	239
4.85.9	See also.....	239
4.86	Request Audio Monitor Actions	240

4.86.1	Description.....	240
4.86.2	Structure	240
4.86.3	Usage.....	241
4.86.4	Scope.....	241
4.86.5	Performance.....	241
4.87	Reply Audio Monitor Actions.....	242
4.87.1	Description.....	242
4.87.2	Structure	242
4.87.3	Additional information	243
4.87.4	Scope.....	243
4.87.5	Performance.....	243
4.87.6	Example	243
4.87.7	Related messages	243
4.88	Request Telephony Client Get State	244
4.88.1	Description.....	244
4.88.2	Structure	244
4.88.3	Additional information	244
4.88.4	Scope.....	245
4.88.5	Performance.....	245
4.88.6	Example	245
4.88.7	Related messages	245
4.89	Reply Telephony Client Get State.....	246
4.89.1	Description.....	246
4.89.2	Structure	246
4.89.3	Additional information	247
4.89.4	Scope.....	247
4.89.5	Performance.....	247
4.89.6	Example	247
4.89.7	Related messages	247
4.90	Request Telephony Client Disconnect Incoming	248
4.90.1	Description.....	248
4.90.2	Structure	248
4.90.3	Additional information	248
4.90.4	Scope.....	248
4.90.5	Performance.....	249

4.90.6	Example	249
4.91	Reply Telephony Client Disconnect Incoming	250
4.91.1	Description	250
4.91.2	Structure	250
4.91.3	Additional information	250
4.91.4	Scope	250
4.91.5	Performance	250
4.91.6	Example	250
4.92	Request Telephony Client Disconnect Outgoing	251
4.92.1	Description	251
4.92.2	Structure	251
4.92.3	Additional information	251
4.92.4	Scope	251
4.92.5	Performance	251
4.93	Reply Telephony Client Disconnect Outgoing	252
4.93.1	Description	252
4.93.2	Structure	252
4.93.3	Additional information	252
4.93.4	Scope	252
4.93.5	Performance	252
4.93.6	Example	252
4.94	Request Telephony Client Dial Info Outgoing	253
4.94.1	Description	253
4.94.2	Structure	253
4.94.3	Additional information	253
4.94.4	Scope	253
4.94.5	Performance	253
4.95	Request Telephony Client Dial Info Incoming	254
4.95.1	Description	254
4.95.2	Structure	254
4.95.3	Additional information	254
4.95.4	Scope	254
4.95.5	Performance	254
4.96	Request Get Proxy Indication State	255
4.96.1	Description	255

4.96.2	Structure	255
4.96.3	Additional information	255
4.96.4	Scope	255
4.96.5	Performance.....	255
4.97	Reply Get Proxy Indication State.....	256
4.97.1	Description.....	256
4.97.2	Structure	256
4.97.3	Additional information	257
4.97.4	Scope	257
4.97.5	Performance.....	257
4.98	Request Set Proxy Indication State	258
4.98.1	Description.....	258
4.98.2	Structure	258
4.98.3	Additional information	259
4.98.4	Scope	259
4.98.5	Performance.....	259
4.99	Reply Set Proxy Indication State.....	260
4.99.1	Description.....	260
4.99.2	Structure	260
4.99.3	Additional information	261
4.99.4	Scope	261
4.99.5	Performance.....	261
4.100	Request Get Proxy Display Data	262
4.100.1	Description.....	262
4.100.2	Structure	262
4.100.3	Additional information	262
4.100.4	Scope	262
4.100.5	Performance.....	262
4.101	Reply Get Proxy Display Data	263
4.101.1	Description.....	263
4.101.2	Structure	263
4.101.3	Additional information	264
4.101.4	Scope	264
4.101.5	Performance.....	264
4.102	Request Set Proxy Display Data	265

4.102.1	Description.....	265
4.102.2	Structure	265
4.102.3	Additional information	266
4.102.4	Scope.....	266
4.102.5	Performance.....	266
4.103	Reply Set Proxy Display Data	267
4.103.1	Description.....	267
4.103.2	Structure	267
4.103.3	Additional information	268
4.103.4	Scope.....	268
4.103.5	Performance.....	268
4.104	Request Panel Keys Public Get State.....	269
4.104.1	Description.....	269
4.104.2	Structure	269
4.104.3	Additional information	269
4.104.4	Scope.....	269
4.104.5	Performance.....	269
4.105	Reply Panel Keys Public Get State	270
4.105.1	Description.....	270
4.105.2	Structure	270
4.105.3	Additional information	270
4.105.4	Scope.....	271
4.105.5	Performance.....	271
4.106	Request Panel Keys Status Auto Updates	272
4.106.1	Description.....	272
4.106.2	Structure	272
4.106.3	Additional information	272
4.106.4	Scope.....	272
4.106.5	Performance.....	272
4.107	Reply Panel Keys Status Auto Updates.....	273
4.107.1	Description.....	273
4.107.2	Structure	273
4.107.3	Additional information	273
4.107.4	Scope.....	273
4.107.5	Performance.....	273

4.108	Request Panel Keys Public Set State	274
4.108.1	Description	274
4.108.2	Structure	274
4.108.3	Additional information	274
4.108.4	Scope	275
4.108.5	Performance	275
4.109	Reply Panel Keys Public Set State	276
4.109.1	Description	276
4.109.2	Structure	276
4.109.3	Additional information	276
4.109.4	Scope	277
4.109.5	Performance	277
4.110	Reply Telephony Key Status	278
4.110.1	Description	278
4.110.2	Structure	278
4.110.3	Additional information	279
4.110.4	Scope	279
4.110.5	Performance	279
4.111	Set Panel Audio Front End State Request	280
4.111.1	Description	280
4.111.2	Scope	280
4.111.3	Structure	280
4.111.4	Additional Information	280
4.112	Set Panel Audio Front End State Reply	281
4.112.1	Description	281
4.112.2	Scope	281
4.112.3	Structure	281
4.112.4	Additional Information	281
4.113	Get Panel Audio Front End State Request	282
4.113.1	Description	282
4.113.2	Scope	282
4.113.3	Structure	282
4.113.4	Additional Information	282
4.114	Get Panel Audio Front End State Reply	283
4.114.1	Description	283

4.114.2	Scope	283
4.114.3	Structure	283
4.114.4	Additional Information	283
4.115	Request Telephony Key Status Enable	284
4.115.1	Description.....	284
4.115.2	Structure	284
4.115.3	Additional information	284
4.115.4	Scope.....	284
4.115.5	Performance.....	284
4.116	Reply Telephony Key Status Enable.....	285
4.116.1	Description.....	285
4.116.2	Structure	285
4.116.3	Additional information	285
4.116.4	Scope.....	285
4.116.5	Performance.....	285
4.117	Request System Crosspoint.....	286
4.117.1	Description.....	286
4.117.2	Structure	286
4.117.3	Additional information	286
4.117.4	Scope.....	286
4.117.5	Performance.....	286
4.118	Reply System Crosspoint	287
4.118.1	Description.....	287
4.118.2	Structure	287
4.118.3	Additional information	288
4.118.4	Scope.....	288
4.118.5	Performance.....	288
4.119	Request Panel Keys Unlatch All.....	289
4.119.1	Description.....	289
4.119.2	Structure	289
4.119.3	Additional information	289
4.119.4	Scope.....	289
4.119.5	Performance.....	289
4.120	Reply Panel Keys Unlatch All	290
4.120.1	Description.....	290

4.120.2	Structure	290
4.120.3	Additional information	290
4.120.4	Scope	290
4.120.5	Performance.....	290
4.121	Request Panel Discovery	291
4.121.1	Description.....	291
4.121.2	Structure	291
4.121.3	Additional information	291
4.121.4	Scope	291
4.121.5	Performance.....	291
4.122	Reply Panel Discovery	292
4.122.1	Description.....	292
4.122.2	Structure	292
4.122.3	Additional information	292
4.122.4	Scope	292
4.122.5	Performance.....	292
4.123	Request IP Panel List	293
4.123.1	Description.....	293
4.123.2	Structure	293
4.123.3	Additional information	293
4.123.4	Scope	293
4.123.5	Performance.....	293
4.124	Reply IP Panel List.....	294
4.124.1	Description.....	294
4.124.2	Structure	294
4.124.3	Additional information	295
4.124.4	Scope	295
4.124.5	Performance.....	295
4.125	Request IP Panel Settings Assign	296
4.125.1	Description.....	296
4.125.2	Structure	296
4.125.3	Additional information	297
4.125.4	Scope	297
4.125.5	Performance.....	297
4.126	Reply IP Panel Settings Assign.....	298

4.126.1	Description.....	298
4.126.2	Structure	298
4.126.3	Additional information	298
4.126.4	Scope.....	298
4.126.5	Performance.....	298
4.127	Request Panel Shift Page Action.....	299
4.127.1	Description.....	299
4.127.2	Structure	299
4.127.3	Additional information	299
4.127.4	Scope.....	300
4.127.5	Performance.....	300
4.128	Reply Panel Shift Page Action	301
4.128.1	Description.....	301
4.128.2	Structure	301
4.128.3	Additional information	301
4.128.4	Scope.....	301
4.128.5	Performance.....	302
4.129	Request Key Group Action	303
4.129.1	Description.....	303
4.129.2	Structure	303
4.129.3	Usage.....	303
4.129.4	Scope.....	304
4.129.5	Performance.....	304
4.129.6	Example	304
4.129.7	Related messages	304
4.129.8	See also.....	304
4.130	Request Key Group Status	305
4.130.1	Description.....	305
4.130.2	Structure	305
4.130.3	Usage.....	305
4.130.4	Scope.....	305
4.130.5	Performance.....	305
4.130.6	Example	305
4.130.7	Related messages	306
4.130.8	See also.....	306

4.131	Reply Key Group Status	307
4.131.1	Description.....	307
4.131.2	Structure	307
4.131.3	Usage.....	307
4.131.4	Scope.....	307
4.131.5	Performance.....	308
4.131.6	Example	308
4.131.7	Related messages	308
4.131.8	See also.....	308
4.132	Request All Key Group Statuses.....	309
4.132.1	Description.....	309
4.132.2	Structure	309
4.132.3	Usage.....	309
4.132.4	Scope.....	309
4.132.5	Performance.....	309
4.132.6	Example	309
4.132.7	Related messages	309
4.132.8	See also.....	309
4.133	Request Frame Status.....	310
4.133.1	Description.....	310
4.133.2	Structure	310
4.133.3	Usage.....	310
4.133.4	Scope.....	310
4.133.5	Performance.....	310
4.133.6	Example	310
4.133.7	Related messages	310
4.133.8	See also.....	310
4.134	Reply Frame Status	311
4.134.1	Description.....	311
4.134.2	Structure	311
4.134.3	Usage.....	311
4.134.4	Scope.....	312
4.134.5	Performance.....	312
4.134.6	Example	312
4.134.7	Related messages	312

4.134.8	See also.....	312
4.135	Request Xpt and Level Status.....	313
4.135.1	Description.....	313
4.135.2	Structure	313
4.135.3	Usage.....	313
4.135.4	Scope.....	313
4.135.5	Performance.....	314
4.135.6	Example	314
4.135.7	Related messages	314
4.135.8	See also.....	314
4.136	Reply Xpt and Level Status	315
4.136.1	Description.....	315
4.136.2	Structure	315
4.136.3	Usage.....	316
4.136.4	Scope.....	316
4.136.5	Performance.....	316
4.136.6	Example	316
4.136.7	Related messages	316
4.136.8	See also.....	316
4.137	Request Trunk Usage Statistics.....	317
4.137.1	Description.....	317
4.137.2	Structure	317
4.137.3	Usage.....	317
4.137.4	Scope.....	317
4.137.5	Performance.....	317
4.137.6	Example	317
4.137.7	Related messages	317
4.137.8	See also.....	318
4.138	Reply Trunk Usage Statistics	319
4.138.1	Description.....	319
4.138.2	Structure	319
4.138.3	Usage.....	320
4.138.4	Scope.....	321
4.138.5	Performance.....	321
4.138.6	Example	321

4.138.7	Related messages	321
4.138.8	See also.....	321
4.139	Request Rack Configuration Status.....	322
4.139.1	Description.....	322
4.139.2	Structure	322
4.139.3	Usage.....	322
4.139.4	Scope.....	322
4.139.5	Performance.....	322
4.139.6	Example	322
4.139.7	Related messages	322
4.139.8	See also.....	323
4.140	Reply Rack Configuration Status	324
4.140.1	Description.....	324
4.140.2	Structure	324
4.140.3	Usage.....	325
4.140.4	Scope.....	325
4.140.5	Performance.....	325
4.140.6	Example	325
4.140.7	Related messages	325
4.140.8	See also.....	325
4.141	Request To Enable/Disable IPA Card Redundancy Switch .	326
4.141.1	Description.....	326
4.141.2	Structure	326
4.141.3	Usage.....	326
4.141.4	Scope.....	326
4.141.5	Performance.....	326
4.141.6	Example: disable all card switching	326
4.141.7	Related messages	327
4.141.8	See also.....	327
4.142	Reply To Enable or Disable IPA Card Redundancy Switch .	328
4.142.1	Description.....	328
4.142.2	Structure	328
4.142.3	Usage.....	328
4.142.4	Scope.....	328
4.142.5	Performance.....	328

4.142.6	Example	328
4.142.7	Related messages	329
4.142.8	See also.....	329
4.143	Request Macro Panel Keys Public State	330
4.143.1	Description.....	330
4.143.2	Structure	330
4.143.3	Additional information	330
4.143.4	Scope.....	330
4.143.5	Performance.....	330
4.144	Reply Macro Panel Keys Public State.....	331
4.144.1	Description.....	331
4.144.2	Structure	331
4.144.3	Additional information	332
4.144.4	Scope.....	332
4.144.5	Performance.....	332
4.145	Request Alt Text State	333
4.145.1	Description.....	333
4.145.2	Structure	333
4.145.3	Additional information	333
4.145.4	Scope.....	333
4.145.5	Performance.....	333
4.146	Reply Alt Text State.....	334
4.146.1	Description.....	334
4.146.2	Structure	334
4.146.3	Additional information	334
4.146.4	Scope.....	335
4.146.5	Performance.....	335
4.147	Request Alt Text Set.....	336
4.147.1	Description.....	336
4.147.2	Structure	336
4.147.3	Additional information	336
4.147.4	Scope.....	336
4.147.5	Performance.....	336
4.148	Reply Alt Text Set	337
4.148.1	Description.....	337

4.148.2	Structure	337
4.148.3	Additional information	337
4.148.4	Scope	337
4.148.5	Performance.....	337
4.149	Request Assigned Keys (With Labels)	338
4.149.1	Description.....	338
4.149.2	Structure	338
4.149.3	Usage.....	338
4.149.4	Scope	339
4.149.5	Performance.....	339
4.149.6	Example	339
4.149.7	Related messages	339
4.149.8	See also.....	339
4.150	Reply Assigned Keys (With Labels)	340
4.150.1	Description.....	340
4.150.2	Structure	340
4.150.3	Usage.....	343
4.150.4	Scope	343
4.150.5	Special notes.....	343
4.150.6	Performance.....	344
4.150.7	Example	344
4.150.8	Related messages	344
4.150.9	See also.....	344
4.151	Request Role State	345
4.151.1	Description.....	345
4.151.2	Structure	345
4.151.3	Additional information	345
4.151.4	Scope	345
4.151.5	Performance.....	345
4.152	Reply Role State	346
4.152.1	Description.....	346
4.152.2	Structure	346
4.153	Request Role State Set	348
4.153.1	Description.....	348
4.153.2	Structure	348

4.153.3	Additional information	348
4.153.4	Scope	348
4.153.5	Performance.....	348
4.154	Reply Role State Set.....	349
4.154.1	Description.....	349
4.154.2	Structure	349
4.154.3	Additional information	349
4.154.4	Scope	349
4.154.5	Performance.....	349
4.155	Request Network Redundancy Endpoint Status.....	350
4.155.1	Description.....	350
4.155.2	Structure	350
4.155.3	Additional information	350
4.155.4	Scope	350
4.155.5	Performance.....	350
4.156	Reply Network Redundancy Endpoint Status	351
4.156.1	Description.....	351
4.156.2	Structure	351
4.156.3	Additional information	351
4.156.4	Scope	352
4.156.5	Performance.....	352
4.157	Request Network Redundancy Card Status	353
4.157.1	Description.....	353
4.157.2	Structure	353
4.157.3	Additional information	353
4.157.4	Scope	353
4.157.5	Performance.....	353
4.158	Reply Network Redundancy Card Status.....	354
4.158.1	Description.....	354
4.158.2	Structure	354
4.158.3	Additional information	354
4.158.4	Scope	354
4.158.5	Performance.....	355
4.1	Request Panel Connection Management Action	355
4.1.1	Description.....	355

4.1.2	Structure	355
4.1.3	Additional information	355
4.1.4	Scope	355
4.1.5	Performance.....	355
4.1.6	Example	356
4.2	Reply Panel Connection Management Action	356
4.2.1	Description.....	356
4.2.2	Structure	356
4.2.3	Additional information	356
4.2.4	Scope	356
4.2.5	Performance.....	357
4.2.6	Example	357
5	APPENDIX A - GAIN LEVEL TRANSLATION TABLE....	364
6	APPENDIX B - GLOSSARY OF TERMS	365
7	GLOSSARY	366

1 INTRODUCTION TO HCI REFERENCE

1.1 WHAT IS HCI?

The Host Computer Interface (HCI) is an interface which can be used to connect a Clear-Com matrix communication system to another system (the "host"). This can enable an integration into a larger system where certain, or all, operations of the are remotely controlled, preset, or automated using third party software or controllers.

Most types of Clear-Com matrix can be configured and controlled remotely.

EHX it is a person-machine interface for configuring the operation of the Clear-Com matrix. EHX uses the HCI API for real-time control of the matrix. This HCI API is available for third-party client development.

When using HCI, it is necessary to set up a baseline "map" configuration in EHX and download it to the matrix in the same way as if the Clear-Com system were operating as standalone.

1.2 WHO USES HCI?

HCI is a standard to which you can work. It is platform independent; you write your own routines in your normal operating system to enable the host to talk to the Clear-Com matrix.

HCI requires a competent programmer to implement.

This document is intended to be used by customers who wish to control the Clear-Com Central Switching Unit (CSU) using the Host Computer Interface (HCI).

1.3 WHAT HCI COMPRISES

HCI is ordered as an option for Clear-Com matrix systems. It comprises of:

- HCI license, supplied as a passcode for the nominated matrix.

Two versions are available of the HCI License

- **HCI-E-HX**

Eclipse HX License:

Host Control Interface, (1) matrix for full automation API

Note: This provides access to all functionality in this document.

Site License option is available for multi-matrix installations.

- **HCI-LIVE-HX**

Eclipse HX License:

Host Control Interface Live, (1) matrix for Controls & Crosspoint API

Note: This provides access to EHX configured Controls (Chapters

4.25 ... 4.29) and Crosspoints (chapters 4.13 ... 4.18).

- Two manuals: The Reference Manual (this document) and Host Interface Library Programmers' Manual. The Programmers' Manual is HCI 1.0 compliant, and serves as a useful source code reference for HCI 2.0 programmers.
- The Host Interface Library. This is a library of C functions which will be useful if you are programming in a Windows environment. This has mostly been ported to the HCI 2.0 interface but is typically only used as a programmer's reference i.e. example code.
- PC Interface layer. Run on the host and allows quick access to the link.
- NETSIM. This is a simulator which simulates the host. Its purpose is to enable you to check your understanding of the message structure and try out different messages on the matrix using a human interface.
- HILTEST. This is a test program which runs every function in the Host Interface Library. This is supplied as a compiled .EXE file (a DOS program which is written in C) and the source code.

1.4 TYPE OF MESSAGE

Messages to the matrix from the host are of two types:

- Action Request. This is a request to the matrix to do something.
- Status Request. This is a request to the matrix to provide some information.
- Messages from the matrix to the host are also of two types (but the distinction between them is different):
- Solicited Message. This is sent in response to a status request.
- Unsolicited Message. This is a message from the CSU reporting a change in the system operation or status which the host did not request.

1.5 SCOPE AND LIMITATIONS

With some limitations on whether HCI and ADM can operate simultaneously, HCI 2.0 is available to work with any Clear-Com Eclipse HX matrix.

To use the HCI API, the license passcode, documentation must be purchased, and the software licensing terms and conditions must be adhered to. Clear-Com engineering support for 3rd party HCI implementation should be quoted for new HCI implementations.

Please contact Clear-Com Customer Support for details.

1.6 SCOPE OF THIS DOCUMENT

Sections 2 describes the overall system operation from the point of view of the host computer.

Section 3 describes the transport level protocols that are supported to pass messages between the host computer and the CSU.

Section 4 describes the messages that may be sent between the host and the CSU.

In the following sections, the CSU start up and reset procedure is described, followed by general descriptions of the operations available once the system has entered the running state. The following conventions have been used:

- when 16 bit words are sent via the Ethernet, the most significant byte is sent first, least significant byte second.
- the bit ordering of bytes sent via the data connection is compatible with that used by IBM PCs.
- where diagrammatic representations of bytes or 16 bit words are used, the rightmost bit is bit 0 which is the least significant bit of the byte or 16 bit word respectively.

Hexadecimal numbers are prefixed with 0x, e.g. decimal 10 = 0x0A, decimal 240 = 0xF0.

Note: HCI is a licensed option and is not supplied as standard. Customers requiring this functionality should contact Clear-Com for further information.

2 OVERVIEW

2.1 START UP

When the CSU is powered up or reset, various aspects of the CSU are re-initialised including the communications hardware and software.

During reset process, spurious characters may be sent to the host by the CSU on a data connection. A correctly implemented protocol will ignore these characters. Should the host send a message whilst the CSU is resetting, no acknowledgement (ACK) of the message will be given by the CSU.

When the reset sequence has been completed by the CSU, the protocol will attempt to establish the communications link. If valid packets are transferred without error, the CSU will then send an automatically generated Reply CPU Status message. This message tells the host that the CSU is now in the running state and is ready to receive and process further commands.

2.2 THE RUNNING STATE

Once the CSU has sent the first Reply CPU Status message, the host can then send and receive messages to and from the CSU as required. During normal operation, the host can request the CSU to carry out an action and the CSU generates replies to keep the host informed of what changes take place. Not all of the messages that may be generated by the CSU are documented here, and any unspecified messages that are received by the host should be ignored.

2.3 REPLIES AND REQUESTS

Messages between the host and the CSU are of two types, request or reply. The host "requests" that the CSU takes some action and the CSU "replies" with information about one particular feature of the CSU.

There are two sorts of request available to the host. The CSU can be requested to carry out an action or to report some status information. For instance, the Request Conference Actions message is used to connect or disconnect a port to or from a conference and a Request Conference Status message causes the CSU to generate a reply telling the host about the ports currently connected to the specified conference.

Reply messages may be generated by the CSU in response to a request by the host or when the status of some part of the system changes. The messages generated in response to a request contain a snapshot of the feature that has been requested. An automatically generated reply message typically contains only the information of the feature that has changed.

2.4 BROADCAST SYSTEM MESSAGES

When the host sends the first "System Request Messages" to the CSU, the CSU will send all Broadcast System Messages that may have been queued since CSU reset. There is a limit to the size of the queue of 600 messages. These messages convey textual information about the status of the system, many of which may be ignored during normal operation. Two fields are provided to allow the easy filtering of these messages, so that only desired messages need to be fully processed.

2.5 SYSTEM FEATURES

There are two types of status message available to report on the current operating status of the system, CPU Status and System Status messages.

The Request and Reply CPU Status messages should be used to find out what state the processor is currently in, what type of reset was last executed by the processor and how long the processor has been the master processor. The host computer should track any CSU resets using the Reply CPU Status message as certain types of reset may clear out assignments that have been made by the host.

The Request and Reply System Status messages should be used to find out the current status of each board in the system.

The only action type message for the CPU is the Request CPU Reset message. This message should be used to execute one of two types of system reset that are available. Once a Request CSU Reset message has been sent by the host, the CSU will ACK the request and then execute the requested reset.

2.6 CROSSPOINTS

The host computer may request an action on any crosspoint within the CSU. This message should be used to create or remove a uni-directional audio route between two specified ports. There are four possible actions that can be achieved using this message:

- To actively enable it (add enable)
- To cancel the enable (delete enable)

or

- To actively inhibit (block crosspoint) it (add inhibit)
- To cancel the inhibit (delete inhibit)

For a remote system only the first two are available; the inhibit facility cannot be used.

An add enable request will attempt to create the specified audio path through the crosspoint and a delete enable will attempt to remove the specified audio path. An add inhibit request will attempt to actively ~~cause a disconnection of~~

block the audio path through the crosspoint, and a delete inhibit request will cancel this attempt. The outcome of an add enable request and add inhibit request clash on a crosspoint depends on the priority associated with each request, but if they are of the same priority, the inhibit will succeed.

Each add enable action for a crosspoint is stored on an action stack. If two add enable actions are received by the CSU (from whatever source) for a crosspoint, then two corresponding delete enable actions are required to cancel the enable actions; one for each enable action stored on the stack. The delete enable actions remove exactly equivalent add actions from the action stack and are not stored. In general, the host should avoid using more than one simultaneous request to turn on or inhibit a crosspoint.

In order to monitor the changes to a specified range of crosspoints, the Request and Reply Crosspoint Status messages should be used. The CSU also generates Reply Crosspoint Status messages whenever a crosspoint changes state, and a full regular update once a minute.

2.7 CONFERENCES

Conferences (partylines) in the CSU are implemented by mixing the input signals of all talkers on the conference and feeding the result to all listeners on the conference. Any talkers that are also listening to the conference will not hear the result to prevent feedback. This mode of operation is also known as "mix-minus". It is also possible to connect other audio sources to a given conference using request conference actions from the host or Clear-Com panels.

The Request Conference Actions message allows the host computer to connect any given port to any given conference as either a talker or a listener. To connect a port as a talker and a listener, two actions must be sent, one to connect the port as talker and another to connect the port as a listener. One port may be connected to multiple conferences by sending multiple conference actions. Disconnections of ports from conferences are achieved by sending delete type actions. The message format also supports the sending of multiple conference actions in one Request Conference Actions message.

Conference actions are very similar to crosspoint actions in that they have the same direction bit mechanism to connect and disconnect a port. However, there is no inhibit mechanism. Conference actions also operate on the individual crosspoint stacks in the same way as crosspoint actions.

The Request and Reply Conference Status messages should be used to determine the current state of any of the conferences in the CSU. The CSU will automatically generate Reply Conference Status messages whenever any change to the conference status is detected.

2.8 GENERAL PURPOSE INPUTS AND OUTPUTS

General Purpose Outputs (GPOs) are used to control equipment that is external to the CSU with each output having two possible states, on or off. When on, a circuit is made via relay contacts and when off, the circuit is broken. The normal state is off (open circuit).

General Purpose Inputs (GPIs) are used to stimulate actions within the CSU. When a GPI is pulled down to 0V, the input is on and any control programmed via EHX is executed. The normal state of the input is high.

A special function output (SFO) simulates a GPI being connected directly to a GPO but the SFOs have no physical inputs or outputs. SFOs are equivalent to Controls in EHX.

The Request GPO/SFO Actions message should be used to control the state of specified GPOs or SFOs. GPO/SFO actions have an equivalent to the crosspoint stacks and have an identical inhibit/enable mechanism for adding and deleting GPO/SFO actions.

The state of the matrix GPIOs and SFOs can be monitored on a per card basis using the Request and Reply GPIO/SFO Status messages. Each card can be either be a GPIO card or a SFO card. The type of card, GPIO or SFO, is determined by the configuration of the CSU.

2.9 AUDIO LEVEL CONTROL

There are different methods of controlling the audio levels in the Clear-Com matrix. These control the input, output and crosspoint levels, and can alter these levels as described below.

The level of the signal entering the matrix may be controlled using the Request Input Level Actions message. The current settings of the input levels can be determined using the Request and Reply Input Level Status messages. Input level controls act independently of any crosspoint and output level controls.

Output level control is provided in a similar manner to input levels and for all outputs is altered by digital processing. The messages used are Request Output Level Actions, Request Output Level Status & Reply Output Level Status.

Crosspoint level control is provided by digital processing. The messages used are Request Crosspoint Level Actions, Request Crosspoint Level Status & Reply Crosspoint Level Status.

Input and output level control will conflict with an Assignment Panel controlling input and output levels. None of the level controls outlined above will conflict with panels controlling crosspoints routed to themselves.

The resultant level of a crosspoint is the combination of input level, crosspoint level and output level - for example if the input level is -3dB, the crosspoint level is +4dB and the output level is +5dB, then the actual gain from input to output is +6dB.

2.10 KEY ASSIGNMENT

The assignment of keys on Clear-Com panels can be achieved by using the Request Locally key Actions message. A list of the key assignments that have been made by the host computer can be accessed using the Request and Reply Local Key Action Status messages.

2.11 CONFIGURATION FILES

It is possible to interrogate the CSU to find out information about the configuration of the CSU. The configuration map information must have previously been sent to the CSU using configuration files or be present in the Flash memory fitted to the CPU card. By using the Request Configuration File message the map signature table can be requested. The CSU returns this with the Reply Configuration File message. The .SIG file is sent automatically by the CSU, so that the host can be informed immediately if the map changes at any time. This may occur when the CSU is reset or a new map has been downloaded using EHX.

2.12 ACTION STATUS

The Request and Reply Action Status messages are used to keep the host informed of the current actions that have taken place in the CSU. Whenever a Request Conference, Crosspoint or GPIO/SFO Action message is processed by the CSU, the CSU will respond with a Reply Action Status message that echoes the action(s) contained in the Request. Should the host lose track of the actions that it has requested, the host should send a Request Action Status message.

The CSU will then generate a list of all the actions made by the host and send them in as many Reply Action Status messages as needed.

2.13 SENDING MULTIPLE COMMANDS

The user must check that a reply message has been received when sending commands to the CSU. For example, after a "Request Crosspoint Actions" message, the CSU will confirm receipt of the message by issuing a "Reply Actions" message. If this message is not received, the request was lost.

3 **TRANSPORT PROTOCOLS**

3.1 **ETHERNET LINK**

The Ethernet Link uses TCP/IP to provide a secure data stream. In order to maintain a packet orientated interface, each data packet is formatted as shown below, and must be transmitted with a TCP/IP "Push" command to ensure the complete data packet is transmitted at the appropriate time.

3.1.1 **Packets Structure**

All transport packets have the same structure which is:

Start 5A0F	Length	Message	End 2E8D
------------	--------	---------	----------

Ethernet Link Packets Structure

The start and end fields, 16 bit words, allow synchronisation of data in the TCP stream.

The length field, 16 bit word, holds the number of bytes of the complete transport packet.

The message field may contain between 2 and 836 bytes of data. The contents of this field are described in the message description section.

3.1.2 **Connection Ports**

After CSU reset the active connections are reset and a range of ports will be passively opened. It is up to the user to regulate which hosts attach to which ports. All ports are treated equally by the CSU.

Available ports: 52001 - 52020

Any Host IP address and port number is accepted as a connection. Only one Host can connect to each of the available ports at a time.

3.1.3 **Ping Facility**

An HCI ping facility has been provided to allow the Host to periodically check that the CSU is alive. If the CSU receives a packet with a size of 10 message, thus:

5A 0F 00 10 00 00 2E 8D

It will be echoed back to the relevant Host.

Excluding network delays, the CSU should respond in less than one second. Since TCP/IP is a secure protocol no retries are required - if the echo is not

received within a few seconds (allowing for other data on the link and the network delay) then the CSU is not alive.

.

3.1.4 Client Session Closure

When ending the HCI session with the matrix central equipment the HCI client should forcefully close the socket connection, as opposed to gracefully flushing the socket before closure.

This approach results the RST bit being used to close the socket as opposed to the FIN bit that signals the need to flush.

The nature of the traffic on the HCI TCP socket makes the reliability of the FIN based closure uncertain and less reliable.

4 MESSAGES

4.1 INTRODUCTION

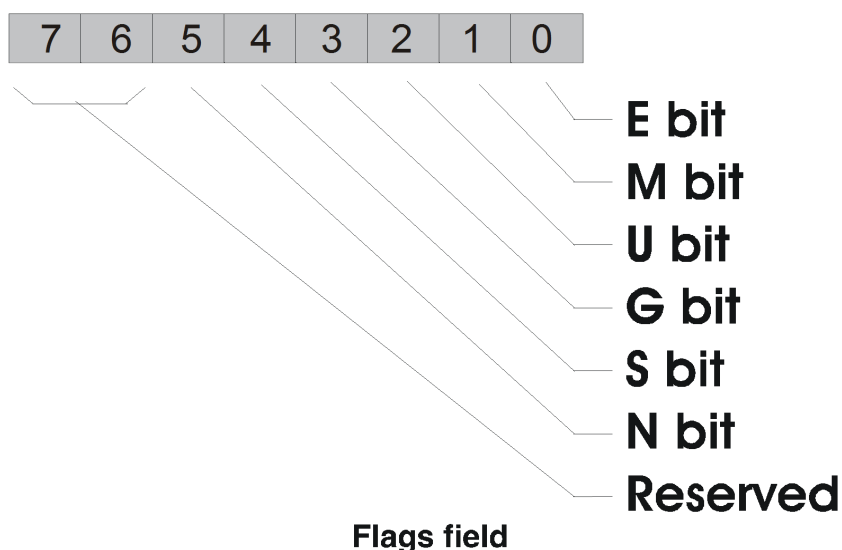
This section describes the message field of the transport packet. It is made up of a number of fields, the message ID, the flags field, the protocol identification fields and the message data field. The message ID field identifies the type of message and is the first 16 bit short of the message field. The third byte is the flags field and is described in detail below.

The protocol identification field is used to differentiate HCI 1.0 and 2.0 messaging and to provide future backwards compatibility.

The remaining bytes of the message field form the data section of the message which has a maximum size of 834 bytes. The contents of this field are described in detail in the message description section.

4.2 FLAGS

The flags field is used by the host and the CSU to allow the recipient of the message to deal with it correctly. The exact usage of the bits in this field are explained below.



4.2.1 Host

The host will set the G flag to 1 and all other bits of the flag field to 0 whenever it transmits a message. If it is necessary to send any messages that are longer than the maximum message length, the message should be split in to two parts and sent as two separate messages of the same type.

4.2.2 CSU

The flag bits are set by the CSU as follows:

E	Set to 1 in all packets where another packet is expected to follow. The last packet in the sequence has this bit set to 0 (see example below).
M	Normally set to 0, but may be ignored if set
U	Set to 1 in a status change message. Set to 0 in a status reply message. (Not set for Reply Conference Status messages.
G	Global bit. This can be ignored by the host.
S	Start bit. Set to 1 in the first packet sent (see example below).
N	Set to 1.

All other bits are set to 0.

The following examples show the possible responses for the Reply Crosspoint Status message generated by the CSU when a change has been detected. The data section of the messages is indicated by an ellipsis.

1 packet	
Packet 1:	00 0E 34 ...
2 packets	
Packet 1:	00 0E 35 ...
Packet 2:	00 0E 24 ...
3 or more packets	
Packet 1:	00 0E 35 ...
Packet 2 to n-1:	00 0E 25 ...
Packet n:	00 0E 24 ...

4.3 Protocol identification

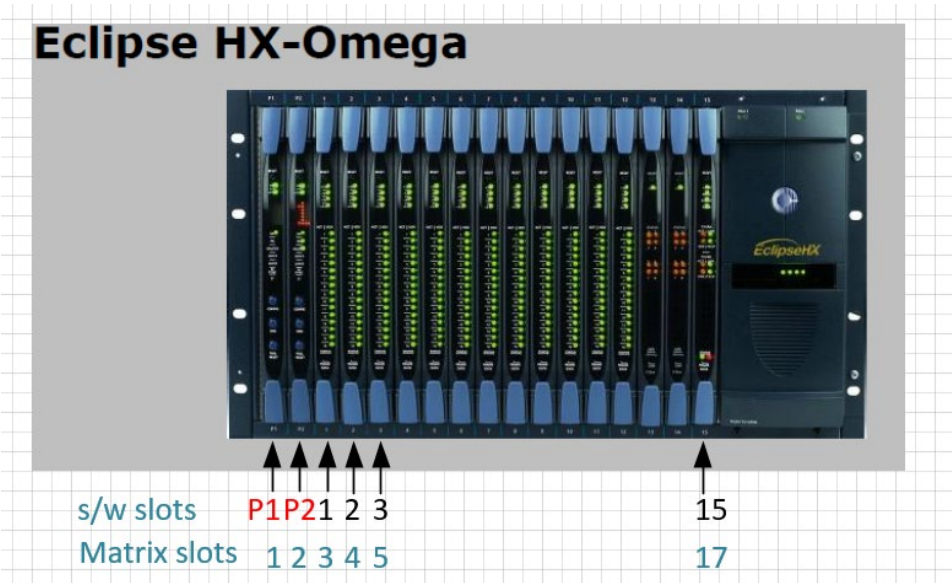
Every HCI 2.0 message contains a protocol identifier and protocol version number (schema). The four bytes following the flag byte will always be 0xAB, 0xBA, 0xCE, 0xDE. The presence of these bytes indicates that this is an HCI 2.0 message.

This is followed by the protocol version byte. The initial version of every HCI 2.0 message is 0x01. If the payload of a message changes in subsequent versions, the version identifier will be incremented to allow reference to the appropriate message structure.

4.4 Slot numbering scheme

For any card slot number field, the slot numbering scheme is as follows:

Matrix physical slot	s/w address	
0	P1 -CPU	
1	P2 CPU – (Master default)	
2	# 1 Audio card	(MVX, IVC, EQU, E-Dante, IPA, etc)
3	# 2 Audio card	(MVX, IVC, EQU, E-Dante, IPA, etc)
4	# 3 Audio card	(MVX, IVC, EQU, E-Dante, IPA, etc)
5	# 4 Audio card	(MVX, IVC, EQU, E-Dante, IPA, etc)



The total range supported is from 1 to 20. (Current Eclipse HX product range only requires the addressing of 17 slots as above.)

4.5 FIELD SIZE LIMITATIONS

The message structure has been designed to allow for expansion up to a maximum of 1024 ports but the actual limits depend on the size of the system. The ranges given in the messages section are the maximums allowable by the protocol and not for the system that has been supplied.

Note: The Host Computer Interface has been designed to process correctly formatted messages efficiently. Messages that are sent by the host that are out of range may not be correctly processed by the CSU. It is the responsibility of the host computer to ensure that values sent to the CSU are within the range of the equipment currently installed in the CSU. No messages are provided to inform the host that the message was not understood by the CSU.

4.6 HOW TO INTERPRET THE DATA MESSAGE DESCRIPTIONS

Port Number	Port Function:	Label:	Description
1.1 (1)	V 2RU Rotary	First	V-Series 2RU Rotary on 1
1.2 (2)	Direct	Second	Direct
1.3 (3)	FOR-22	Third	FOR-22 on 3

Message name	Usage and limitations
Description	A general description of the message.
Structure	Describes the structure of the message. Each field of the message is specified in turn, first the size of the field and then the values that it can take. Elements addressed in the HCI protocol are zero based. The EHX user interface and dial code addressing of elements is instead one based. For example, Port 1 in the EHX cards and port screen is actually port 0 when being controlled or monitored using HCI.

Message name	Usage and limitations
	This applies to Ports, Roles, Conferences, Fixed Groups and IFBs.
Usage	A detailed description of how to use the message. This includes how to interpret the fields where necessary.
Scope	System limitations to the message, e.g. valid card types that the messages will work to, message direction.
Performance	This section describes the performance limitations of the message type. The overall performance of the system is determined by the number of transactions that have to be processed per unit time. Unless otherwise indicated, processor time is typically, but not guaranteed <5mS. Response time is typically, but not guaranteed, <50mS. These figures exclude comms link delays, and vary according to system loading and message size.
Example	This section contains a text description of an example use of the message. It is followed by a hexadecimal display of the described example message.
Related messages	This section contains references to other messages that are used with this message.

Message name	Usage and limitations
See also	This section includes references to other sections of this document that describe how the messages fit into the overall usage of the system.

4.6.1 Dial code format

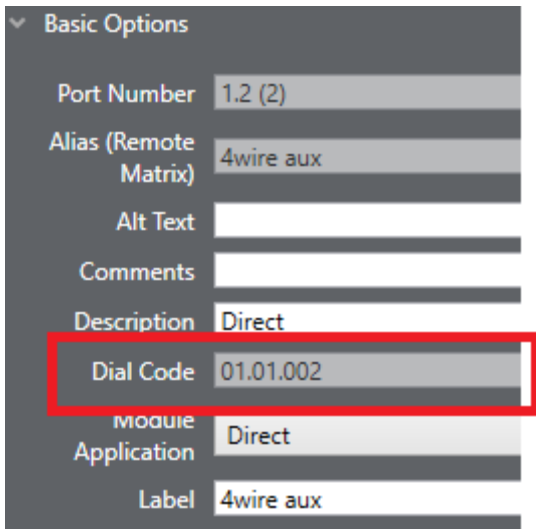
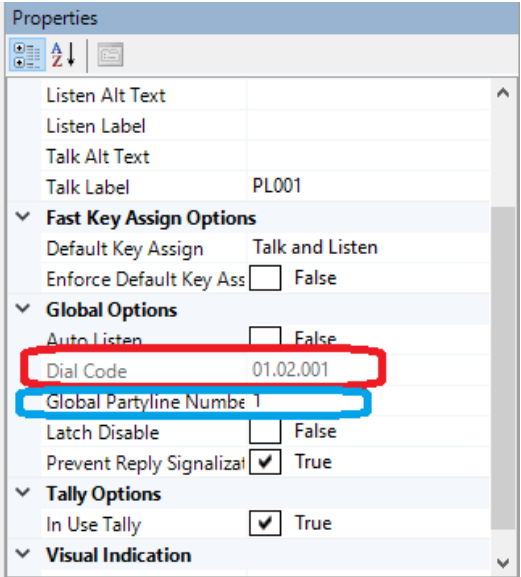
The dial code format is as follows:

Byte 1 – Matrix ID

Byte 2 – Entity type (PORT= 1, CONF= 2, GROUP= 3, IFB= 4, GPIO= 5)

Byte 3 and 4 – Instance of type.

Dial codes are used to address intercom entities in the system, for example fast assign functionality on panels and in some HCI messages.

Panel / 4wire / interface dialcode	Partyline dialcode
	

Fixed Group dialcode

Properties

Call Management Group Options

Call Management Group ☐ False

Call Management Group ☐ 20 seconds

Fast Key Assign Options

Default Key Assign Talk and Listen

Enforce Default Key Ass ☐ False

Global Options

Dial Code 01.03.002

Global Group Number 2

Latch Disable ☐ False

Page Override ☐ False

Tally Options

In Use Tally ☐ False

Visual Indication

Visual Indication Overri No Override

IFB dialcode

Basic Options

Alias (Remote Matrix) IFB001

Alt Text

Comments

Description Virtual IFB

Dial Code 01.04.001

Label IFB001

GPIO / SFO / EHX Control dialcode using card / pin

Properties

Engineer Comments

Listen Alt Text

Listen Label

Talk Alt Text

Talk Label CT001

Fast Key Assign Options

Default Key Assign Talk and Listen

Enforce Default Key Ass ☐ False

Global Options

Control Identifier Seed 0

Latch Disable ☐ False

Listen Control Card:Pin 28:2

Talk & Listen Card:Pin 28:0

Talk Control Card:Pin 28:1

Visual Indication

Visual Indication Overri No Override

4.7 Broadcast System Message

4.7.1 Description

This message is sent by the CSU whenever broadcast information is generated by the CSU.

4.7.2 Structure

The message structure below defines the complete payload between the HCI start and end bytes.

Length	16 bit word – Length of entire payload including HCI start and end marker bytes
Message ID:	16 bit word 1 (0x0001).
Flags:	1 byte, See FLAGS on page 59.
Class:	16 bit word, 0 = Fatal Error, 1 = Non - Fatal Error, 2 = Warning, 3 = Information, 4 = Debug, 5 = Log to disk, 6 - 65535 = Reserved.
Code:	16 bit word – Unique or component allocated event ID
Reserved:	4 bytes
Text:	180 bytes maximum – Null terminated event text

4.7.3 Usage

The Class field should be used to filter out unwanted classes of message.

The Code field is set to a message number and can also be used to filter out unwanted messages. Some messages have a unique code, but many are given a common code identifying a category of event.

The Text field contains the text of the message as a C style, null terminated string of characters.

4.7.4 Scope

CSU to Host only.

4.7.5 Performance

4.7.6 Example

The following example shows the following string being sent with class 6 and code 0x33C

13/05/30 18:04:55.152,20|Current Xpt Memory usage 1(0%) / 0(0%)

00 51 00 01 30 00 06 03 3c 00 00 00 00 31 33 2f 30 35 2f 32 30 20 31 38 3a 30
34 3a 35 35 2e 31 35 32 2c 32 30 7c 43 75 72 72 65 6e 74 20 58 70 74 20 4d
65 6d 6f 72 79 20 75 73 61 67 65 20 31 28 30 25 29 20 2f 20 30 28 30 25 29 00

4.7.7 Special notes

None.

4.7.8 Related messages

None

4.7.9 See also

4.8 Request Actions Status

4.8.1 Description

This message is used to request the CSU to send a list of the actions of the specified type made by the specified host.

4.8.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema	Action type	Host ID
------------	-------	--------------	-----------------	-------------	---------

Request Actions Status

Message ID:	16 bit word, 15 (0x000F)
Flags:	1 byte, See FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Action type:	16 bit word
bit 0	Crosspoint
bit 1	Reserved
bit 2	GPIO/SFO
bits 3, 4	Reserved
bit 5	Conference
bits 6 – 15	Reserved.

4.8.3 Usage

This message should be used when the host has become disconnected from the CSU (e.g. power failure of host).

The bit fields in the Action Type field are not mutually exclusive, so a list of all actions made by the host could be obtained by setting all bits to 1 (0xFFFF).

If the global flag is set in the flags field, then actions that have been created from any host will be included in the reply. If this global flag is not set, then only the actions that match the HCI connection IP address will be sent back to the host in the reply.

4.8.4 Scope

Host to CSU only.

4.8.5 Performance

4.8.6 Example

This message was sent by the host to request a list of all the crosspoint actions that had been sent by the host.

00 0F 08 AB BA CE DE 01 00 01

4.8.7 Special notes

None.

4.8.8 Related messages

Reply Actions Status.

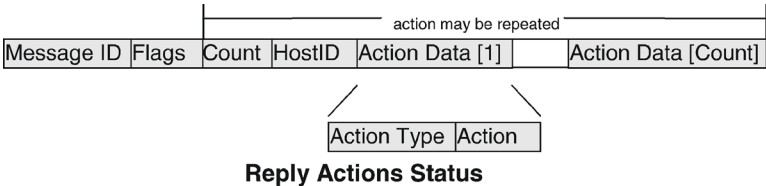
4.8.9 See also

4.9 Reply Actions Status

4.9.1 Description

This message contains a list of all of the actions sent by a host of the types that are specified by that host. It is generated whenever a Request Crosspoint, GPIO/SFO or Conference Actions message has been received and stored by the CSU and in response to a Request Actions Status message.

4.9.2 Structure



4.9.3 Usage

The Action field is set according to the message sent to the CSU. Refer to the appropriate Actions message for details of the information in the fields within the Action field.

4.9.4 Scope

CSU to Host only.

4.9.5 Performance

4.9.6 Example

This example shows the automatically generated reply to the request to connect port 300 to conference 5 as a talker.

00 10 34 AB BA CE DE 01 00 01 00 00 00 00 00 20 04 0B 05 2C 00 00 00 00

4.9.7 Special notes

None.

4.9.8 Related messages

Request Action Status.

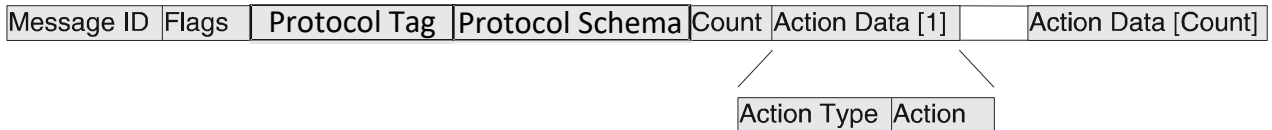
4.9.9 See also

4.10 Request Conference Actions

4.10.1 Description

This message is used to connect or disconnect specified ports to specified conferences.

4.10.2 Structure



Request Conference Actions

Message ID:	16 bit word, 17 (0x0011)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16 bit word
Action Data:	
Action Type:	16 bit word, set to 0x0020
Action:	4 x 16 bit words
word 0:	
bit 0	direction bit, 0 = delete, 1 = add
bit 1,2	port number, bits 8 & 9
bit 3	0 = listen, 1 = talk
bits 4 - 9	0
bit 10	1
Bits 11 - 12	Conference number, bits 8 & 9
bits 11 - 15	0

word 1	
bits 0 - 7	port number, bits 0 - 7
bits 8 - 13	conference number, bits 0 - 7
bits 14,15	0
words 2 & 3	set to 0

4.10.3 Usage

A single conference action can request that a connection between one port and one conference is either added or deleted. The connection may be either talk or listen. The Count field of this message should be set to the number of Action Data fields used.

The CSU will reply to this request with a Reply Action Status message.

4.10.4 Scope

There is a maximum of 400 conferences available, each of which may have a maximum of 64 simultaneous talkers and a maximum of 64 simultaneous listeners.

Host to CSU port numbers are in the range 0-1023 only.

4.10.5 Performance

4.10.6 Example

This example shows the port 300 being connected to conference 5 as a talker.

00 11 08 AB BA CE DE 01 00 01 00 20 04 0B 05 2C 00 00 00 00

4.10.7 Special notes

None.

4.10.8 Related messages

Reply Action Status, Request Conference Status, Reply Conference Status.

4.10.9 See also

4.11 Request Conference Status

4.11.1 Description

Asks the CSU for a list of the users connected to the specified conference.

4.11.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema	Conference number
------------	-------	--------------	-----------------	-------------------

Request Conference Status

Message ID:	16 bit word, 19 (0x0013)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Conference number:	16 bit word

4.11.3 Usage

This should be used to obtain a list containing all talkers and listeners that are connected to the specified conference.

4.11.4 Scope

Host to CSU only.

4.11.5 Performance

4.11.6 Example

This example asks the CSU to send a list of the current connections to conference 17.

00 13 08 AB BA CE DE 01 00 11

4.11.7 Special notes

None.

4.11.8 Related messages

Request Conference Actions, Reply Conference Status.

4.11.9 See also

4.12 Reply Conference Status

4.12.1 Description

This message contains a list of all the currently connected ports to a conference. It is generated in response to Request Conference Status message or whenever a port is connected to or removed from any conference.

4.12.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema	Port Count	Conference Data	Port Data[1]		Port Data[Count]
------------	-------	--------------	-----------------	------------	-----------------	--------------	--	------------------

Reply Conference Status

Message ID:	16 bit word, 20 (0x0014)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Port Count:	16 bit word
Conference Data:	16 bit word:
bits 0 - 12	conference number
bits 13,14	set to 0
bit 15	set to 1
Port Data:	16 bit word:
bits 0 - 12	port number, range 0 - 1023
bit 13	0 = not listener, 1 = listener
bit 14	0 = not talker, 1 = talker
bit 15	set to 0

4.12.3 Usage

Port Count contains the number of Port Data items plus the Conference Data item.

Port Data holds the port number and the type of connection, talker and/or listener.

4.12.4 Scope

CSU to host only.

4.12.5 Performance

4.12.6 Example

In the following example, the CSU has generated a reply to a Request Conference Status message of conference 12. The conference has 5 ports connected to it; 10, 39 as talkers; 34, 42 as listeners and port 82 as a talker and a listener.

```
00 14 30 AB BA CE DE 01 00 06 80 0C 40 0A 40 27 20 22 20 2A 60 52
```

4.12.7 Special notes

This message does not set the update flag of the flags field when a change is reported.

4.12.8 Related messages

Request Conference Status, Request Conference Actions.

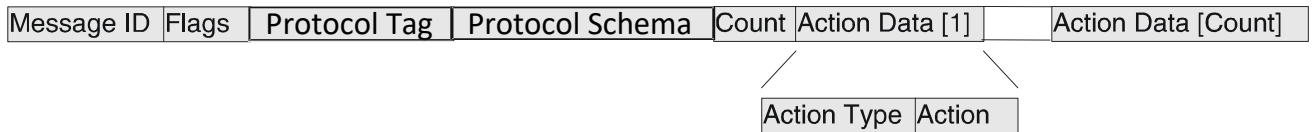
4.12.9 See also

4.13 Request Crosspoint Actions

4.13.1 Description

This message is used by the host to change the state of one or more crosspoints in the CSU.

4.13.2 Structure



Request Crosspoint Actions

Message ID:	16 bit word, 17 (0x0011)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16 bit word
Action Data:	
Action Type:	16 bit word, set to 0x0001
Action:	4 x 16 bit words:
Word 0:	
bit 0	direction bit, 0 = delete, 1 = add
bits 1,2	destination port number, bits 8,9
bits 3 – 9	set to 0
bit 10	set to 1
bits 11, 12	source port number, bits 8, 9
bit 13	set to 1

bit 14	Set to 0
bit 15	set to 0
Word 1:	
bits 0 - 7	destination port number, bits 0 - 7
bits 8 - 15	source port number, bits 0 - 7
Word 2:	
bits 0 - 15	set to 0
Word 3:	
bit 0	set to 0
bit 1	set to 1
bit 2	set to 0
bits 3 - 9	set to 1
bit 10	set to 0
bit 11	enable/disable, 0 = enable, 1 = inhibit.
bit 12	set to 0
bits 13 - 15	crosspoint priority, set to 2 or 3 for local CSU only, 1 otherwise

4.13.3 Usage

Count should be set to the number of different Action Data elements in the message. The destination and source port numbers are formed from two fields in each case.

Each crosspoint action makes a request to change the status of a specified crosspoint. Each crosspoint action works in one audio direction only, so to set a talk and listen route between two ports, two crosspoint actions must be sent.

4.13.4 Scope

Host to CSU only.

4.13.5 Performance

4.13.6 Example

4.13.7 Special notes

None.

4.13.8 Related messages

Request Crosspoint Status, Reply Crosspoint Status.

4.13.9 See also

Request Conference Actions, Request GPO Actions, Request SFO Actions.

4.14 Request Crosspoint Status

4.14.1 Description

This message asks the CSU for the connection status for each of the specified ports.

4.14.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema	Count	Port [1]		Port [Count]
------------	-------	--------------	-----------------	-------	----------	--	--------------

Request Crosspoint Status

Message ID:	16 bit word, 13 (0x000D)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16 bit word, range 1 - 495
Port:	16 bit word, range 0 - 1023

4.14.3 Usage

Count should be set to the same number of ports for which status information is requested. Each port number should be set to a valid port number.

4.14.4 Scope

Host to CSU only.

4.14.5 Performance

4.14.6 Example

The host requests the current status of ports 127, 255 and 300.

00 0D 08 AB BA CE DE 01 00 03 00 7F 00 FF 01 2C

4.14.7 Special notes

None.

4.14.8 Related messages

Request Crosspoint Actions, Reply Crosspoint Status

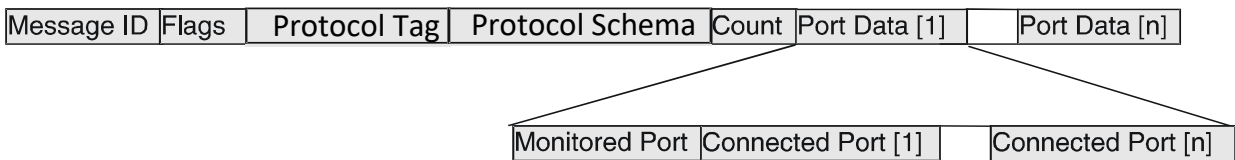
4.14.9 See also

4.15 Reply Crosspoint Status

4.15.1 Description

This message contains information on the status of the crosspoints that are made to ports. There are two versions of this reply, one in response to a Request Crosspoint Status message and the other as a result of an automatically generated response to a change in crosspoint status.

4.15.2 Structure



Reply Crosspoint Status

Message ID:	16 bit word, 14 (0x000E)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16 bit word
Port data:	
Monitored Port:	
bits 0 - 12	port number 0 - 1023
bits 13,14	set to 0
bit 15	monitored port bit, set to 1
Connected Port:	
bits 0 - 12	port number, 0 - 1023
bit 13	0 = not listener, 1 = listener

bit 14	0 = not talker, 1 = talker
bit 15	connected port bit, set to 0

4.15.3 Usage

The CSU may generate very large messages using this message which will be split up into one or more packets using the continuation flags.

Messages that are replies to the Request Crosspoint Status message have, for each monitored port, a list of ports talking and/or listening to it.

Messages that are generated by the CSU when changes occur only contain a list of all the ports that have changed their talking and/or listening state to another port.

Count holds the number of 16 bit words that follow the count field in the rest of the message.

4.15.4 Scope

CSU to host only.

4.15.5 Performance

4.15.6 Example

This message was generated by the CSU when the crosspoint connecting port 10 to port 12 was made as a talk only. 0E 34 00 00 02 80 AB BA CE DE 01 0A 40 0C

4.15.7 Special notes

This message is generated automatically once per minute to give a full status list for all monitored ports requested via the "Request Crosspoint Status" message since the last CSU reset.

4.15.8 Related messages

Request Crosspoint Actions, Request Crosspoint Status.

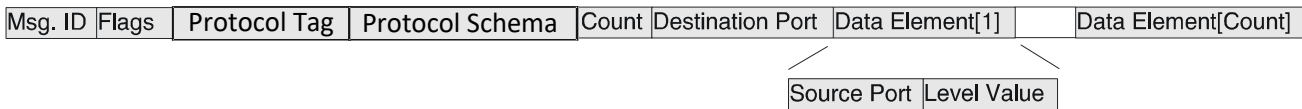
4.15.9 See also

4.16 Request Crosspoint Level Actions

4.16.1 Description

This message asks the CSU to change the gain level(s) of the specified crosspoint(s) using level codes as defined in APPENDIX B - GAIN LEVEL TRANSLATION TABLE

4.16.2 Structure



Request Crosspoint Level Actions

Message ID:	16 bit word, 38 (0x0026)
Message Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16 bit word
Destination Port:	16 bit word, 0 - 1023
Source Port:	16 bit word, 0 - 1023
Level Value:	16 bit word, 0 - 255

4.16.3 Usage

Count should be set to the number of data elements in the message.

The crosspoint is specified by the source and destination port numbers and the audio route that is affected is in the source to destination direction only.

The Level Value is used to set the crosspoint to the gain specified.

4.16.4 Scope

Host to CSU only.

4.16.5 Performance

4.16.6 Example

This example shows the message sent by the host computer to set a crosspoint that connects port 5 to port 12 to the gain 0xb9.

00 26 08 AB BA CE DE 01 00 01 00 0C 00 05 00 B9

4.16.7 Special notes

None.

4.16.8 Related messages

Reply Crosspoint Level Status.

4.17 Request Crosspoint Level Status

4.17.1 Description

This message will request all non-zero crosspoint level information for the specified destination port This message will cause the CSU to generate a Reply Crosspoint Level Status message.

4.17.2 Structure

Msg. ID	Flags	Protocol Tag	Protocol Schema	Count	Destination Port[1]		Destination Port[Count]
---------	-------	--------------	-----------------	-------	---------------------	--	-------------------------

Request Crosspoint Level Status

Message ID:	16 bit word, 39 (0x0027)
Message Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16 bit word
Destination Port:	16 bit word, 0 - 1023

4.17.3 Usage

Count should be set to the number of Destination Ports in the message.

4.17.4 Scope

Host to CSU only.

4.17.5 Performance

4.17.6 Example

This message was sent when the host computer needed to re-establish the current crosspoint levels of any connections made to ports 16 and 32.

00 27 08 AB BA CE DE 01 00 02 00 10 00 20

4.17.7 Special notes

None

4.17.8 Related messages

Request Crosspoint Level Actions, Reply Crosspoint Level Status.

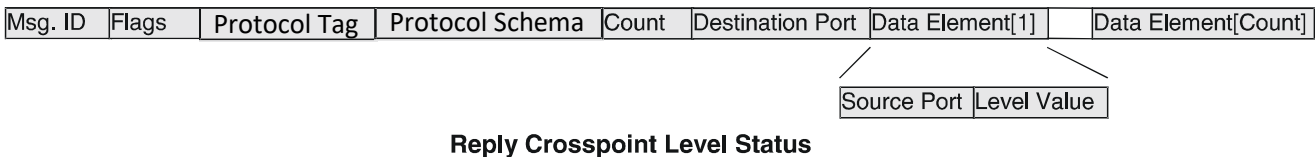
4.17.9 See also

4.18 Reply Crosspoint Level Status

4.18.1 Description

This message contains a list of all the HCI applied crosspoint levels adjustments for a specified destination. It is generated by the CSU in response to a Request Crosspoint Level Action or Crosspoint Level Status message.

4.18.2 Structure



Message ID:	16 bit word, 40 (0x0028)
Message Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16 bit word
Destination Port:	16 bit word, 0 - 1023
Source Port:	16 bit word, 0 - 1023
Level Value:	16 bit word, 0 - 255

4.18.3 Usage

Count holds the number of data elements in the message.

The Level Value reports the current gain value of the crosspoint and not the value that is used in the Request Crosspoint Level Actions message. The gain of the crosspoint can be calculated from the formula: gain (dB) = (Level Value - 204) x 0.355 (see also Appendix C)

4.18.4 Scope

CSU to host only.

4.18.5 Performance

4.18.6 Example

This message was sent by the CSU to report the current crosspoint levels for destination port 3. Two non-default crosspoints were currently made, a route from source port 5 at level 221 (+6dB) and another route from source 8 at level 187 (-6dB).

00 28 34 AB BA CE DE 01 00 02 00 03 00 05 00 DD 00 08 00 BB

4.18.7 Special notes

None.

4.18.8 Related messages

Request Crosspoint Level Action, Request Crosspoint level status

Request Input / Output Level Actions, Request Input / Output Level Status

4.18.9 See also

4.19 Request CPU Reset

4.19.1 Description

This message instructs the CPU to execute one of the four available reset instructions.

4.19.2 Structure

Message ID	Protocol Tag	Protocol Schema	Flags	Reset type
------------	--------------	-----------------	-------	------------

Request CPU Reset

Message ID:	16 bit word, 41 (0x0029)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Reset Type:	1 byte, bit 0 =-red, bit 1 = black, bits 2 - 7 = 0

4.19.3 Usage

The effect on the system depends on the type of reset requested.

A red reset clears out all existing host route assignments and remakes them. Existing audio routes are maintained during this process and are then updated when complete. Communications with the host will be lost during this process.

A red and black reset resets the CSU hardware which causes any existing audio routes to be cleared. All existing host assignments are cleared and are not remade. Communications with the host will be lost during this process. All panels will download the current configuration information from the CSU / EHX configuration.

4.19.4 Scope

Host to CSU only.

4.19.5 Performance

Processor time required: N/A. Response time:

< 100ms until reset process begins. Reset process takes between 15 and 60 seconds before communications restored, depending on system size.

Maximum message frequency: N/A.

4.19.6 Example

The host would send the following message to execute a black reset.

00 29 08 AB BA CE DE 01 02

4.19.7 Special notes

This message should be used with care as it may cause inconvenience to users of the system.

4.19.8 Related messages

Request CPU Status, Reply CPU Status.

4.19.9 See also

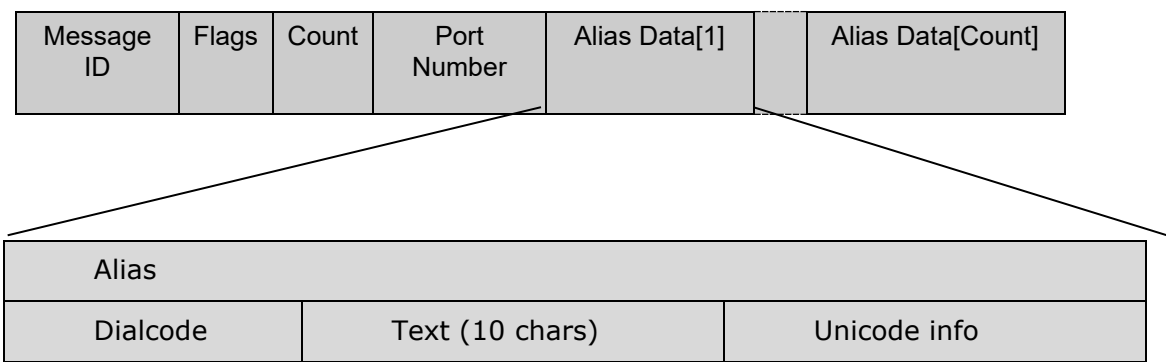
4.20 Request Unicode Alias Add

4.20.1 Description

This message is used by the host to change the text of a Directory Entry specified in the system configuration map. By adding an alias for a specific Directory Entry, this text will be used in place of the Directory Entry text until the alias is deleted. The command allows the alias to be used in the local system only, in remote systems only, or both.

This text is up to 10 unicode (UTF-16 BE) characters.

4.20.2 Structure



Message ID:	16 bit word, (0x00F4) (244d)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16-bit Word, number of key assignments in message
bits 10 - 13	unused
bit 14	Set to 1 to inhibit the alias from being used on remote systems

bit 15	Set to 1 to inhibit the alias from being used on local systems
Dialcode:	4 bytes. See Dial code format section.
Text:	10 unicode chars (wchar_t)
Unicode info:	
Bits 0-14	unused
Bit 15	Set to 1 to show it is Unicode text

4.20.3 Usage

The Count field contains the number of Alias Data fields in the message.

4.20.4 Scope

Host to CSU only.

4.20.5 Performance

4.20.6 Example

Add unicode alias to system 1 port 10 DESCRIPTION

```
00 F4 08 AB BA CE DE 01 00 01 00 00 01 01 00 0A 00 44 00 45 00 53 00 43 00 52
00 49 00 50 00 54 00 49 00 4F
```

4.20.7 Special notes

4.20.8 Related messages

Request Unicode alias list

4.20.9 See also

4.21 Reply Unicode Alias Status

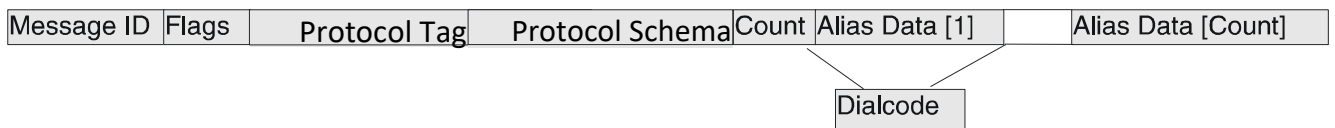
4.21.1 Description

This message is sent in two circumstances:

In response to Request Alias Add, in which case the U flag is set (flag word is set to 0x0C)

In response to Request Alias List, in which case the U flag is clear (flag word is set to 0x08)

4.21.2 Structure



Reply Alias Status

Message ID:	16 bit word, (0x00F5) (245d)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16-bit Word, number of key assignments in message
bits 10 - 13	unused
bit 14	Set to 1 to inhibit the alias from being used on remote systems
bit 15	Set to 1 to inhibit the alias from being used on local systems
Dialcode:	4 bytes. See Dial code format section.
Text:	20 bytes. 10 unicode chars (wchar_t)

Unicode info:	
Bits 0-13	unused
Bit 14	Set to 1 if alias is only to be applied to listen keys (currently only used for E-Dante ports)
Bit 15	Set to 1 to show it is Unicode text

4.21.3 Usage

The data in this message, when sent in response to Request Alias Add should be identical to the Request command and can be used for verification purposes.

In response to Request Alias List, the data will list all aliases that are currently in force on the local system, and have been requested by HCI connections to the local system. Aliases requested by other systems are NOT listed in this message.

4.21.4 Scope

CSU to Host only.

4.21.5 Performance

4.21.6 Example

The following example lists the alias for local system (system 1) port 2 as being "12345 ", sent in response to Request Alias Add.

```
00 83 0C AB BA CE DE 01 00 01 00 00 01 01 00 02 00 31 00 32 00 33 00 34 00
35 00 20 00 20 00 20 00 20 00 20 80 00
```

4.21.7 Special notes

None.

4.21.8 Related messages

Request Alias Add, Request Alias Delete, Reply Alias Delete, Request Alias List

4.21.9 See also

4.22 Request Unicode Alias List

4.22.1 Description

This message is used by the host to request a list of aliases made by the host to the local system.

4.22.2 Structure

Message ID:	16 bit word (0x00F6) (246d)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.

4.22.3 Usage

In response to this command, the CSU generates "Reply Unicode Alias Status".

4.22.4 Scope

Host to CSU only.

4.22.5 Performance

4.22.6 Example

The following example requests a list of aliases.

00 86 08 AB BA CE DE 01

4.22.7 Special notes

None.

4.22.8 Related messages

Request Unicode Alias Add, Reply Unicode Alias Status, Request Alias Delete, Reply Alias Delete.

4.22.9 See also

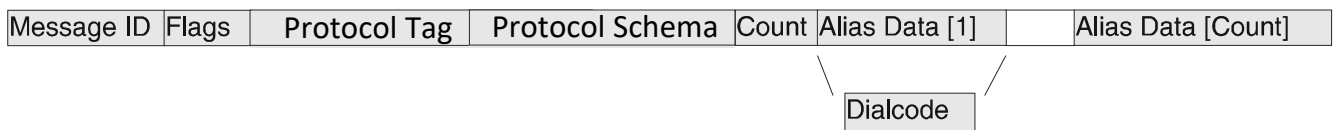
Reply Unicode Alias Status,
Request Alias Delete,
Reply Alias Delete.

4.23 Request Alias Delete

4.23.1 Description

This message is used by the host to delete an alias previously requested. Only aliases made by an HCI link to the local system can be deleted.

4.23.2 Structure



Request Alias Delete

Message ID:	16 bit word, 132 (0x0084)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16 bit word
Alias Data:	
Dialcode:	4 Bytes
Bits 0 - 15	Entity instance e.g. 1 st port in frame would have instance 1
Bits 16 - 23	Entity type – see entity definitions – Note in this document it is above.
Bits 24 - 31	Target System Number – for future expansion, must currently local i.e. set to the matrix ID of the receiving frame or 0.

4.23.3 Usage

Count should be set to the number of Alias Data elements in the message.

The Dialcode field should contain the dialcode of the Directory Entry to be aliased.

If there is no Alias for the supplied Dialcode, no error is generated, and the dialcode will be listed in the response.

In response to this command, the CSU generates "Reply Alias Delete".

4.23.4 Scope

Host to CSU only.

4.23.5 Performance

4.23.6 Example

The following example deletes the alias for the first port on system 1.

```
00 84 08 AB BA CE DE 01 00 01 01 01 00 01
```

4.23.7 Special notes

None.

4.23.8 Related messages

Request Alias Add, Reply Alias Status, Reply Alias Delete, Request Alias List

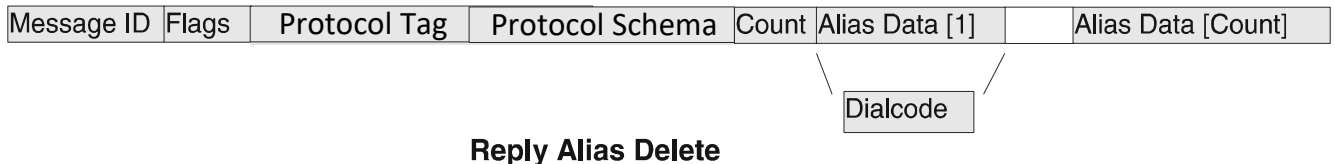
4.23.9 See also

4.24 Reply Alias Delete

4.24.1 Description

This message is sent in response to Request Alias Delete.

4.24.2 Structure



Message ID:	16 bit word, 133 (0x0085)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16 bit word
Alias Data:	
Dialcode:	4 Bytes
Bits 0 - 15	Entity instance e.g. 1 st port in frame would have instance 1
Bits 16 – 23	Entity type – see entity definitions – Note in this document it is above.
Bits 24 - 31	Target System Number – for future expansion, must currently local i.e. set to the matrix ID of the receiving frame or 0.

4.24.3 Usage

The data in this message, when sent in response to Request Alias Delete should be identical to the Request command and can be used for verification purposes.

4.24.4 Scope

CSU to Host only.

4.24.5 Performance

4.24.6 Example

The following example shows the response to a request to delete the alias for the first port on system 1.

```
00 85 08 AB BA CE DE 01 00 01 01 01 00 01
```

4.24.7 Special notes

None.

4.24.8 Related messages

Request Alias Add, Reply Alias Status, Request Alias Delete, Request Alias List

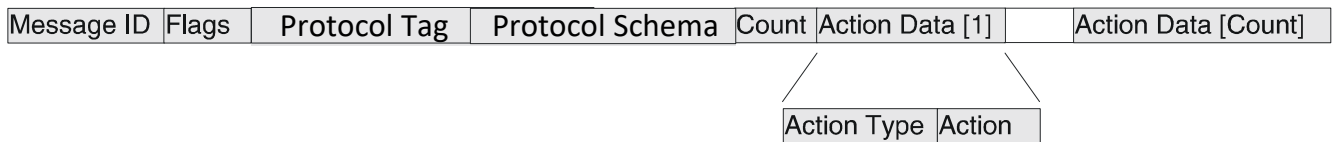
4.24.9 See also

4.25 Request EHX Control Actions

4.25.1 Description

This message is used to change the state of general-purpose outputs (GPOs) and EHX controls. EHX controls can be used within the CSU configuration map to trigger matrix GPO-Relays, routes, logic. They can also be attached to matrix GPI-Inputs and triggered by an external source

4.25.2 Structure



Request GPO/SFO Actions

Message ID:	16 bit word, 17 (0x0011)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16 bit word
Action Data:	
Action Type:	16 bit word, set to 0x0004
Action:	4 x 16 bit word
Word 0:	
bit 0	direction bit, 0 = delete, 1 = add
bits 1 - 3	set to 0
bit 4	set to 1
bits 5 - 9	set to 0
bit 10	set to 1
bits 11, 12	set to 0

bit 13	set to 1
bits 14,15	set to 0
Word 1:	
bits 0 - 4	pin number, 0 - 31
bits 5 - 7	set to 0
bits 8 - 12	card number, 0 - 31
bits 13 - 15	set to 0
Word 2:	
bits 0 - 15	set to 0
Word 3:	
bit 0	set to 0
bit 1	set to 1
bit 2	set to 0
bits 3 - 9	set to 1
bit 10	set to 0
bit 11	map type, 0 = enable, 1 = inhibit
bit 12	set to 0
bits 13, 14	set to 1
bit 15	set to 0

4.25.3 Usage

Count should be set to the number of different Action Data elements in the message.

The card number determines the type of output that will be affected. ~~GPIO-cards are in the range 0—15 and SFO EHX control "cards" are in the range 28 - 127,~~ although different configurations can be set up by Clear-Com.

4.25.4 Scope

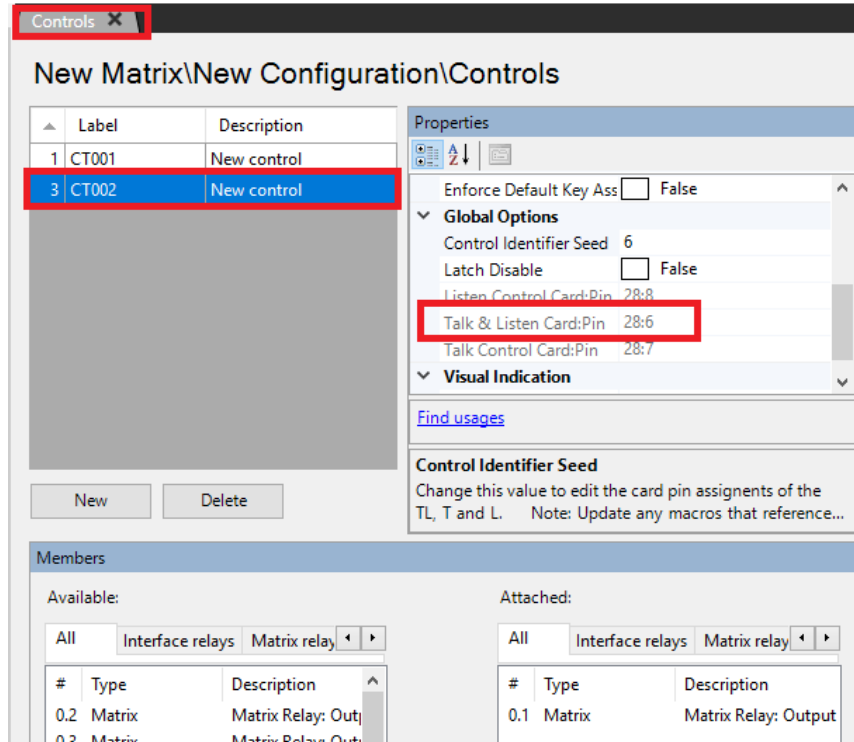
Host to CSU only.

Matrix GPIOs and-EHX controls supported only.

4.25.5 Performance

4.25.6 Example

This example was sent by the host to request that card 28 pin 6 be turned on.



(the second GPIO card in the CSU) be turned on.

00 11 08 AB BA CE DE 0100 01 00 04 24 11 01 02 00 00 63 FA

4.25.7 Special notes

The card numbers are auto generated by the EHX application as controls are created in the configuration map. Each control consists of 3 pins. Only use the Talk & listen control pin

4.25.8 Related messages

Request EHX Control Status, Reply EHX Control Status.

4.25.9 See also

4.26 Request EHX Control Card Status

4.26.1 Description

This message causes the CSU to generate a Reply EHX Control Card Status message.

4.26.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema
------------	-------	--------------	-----------------

Request GPIO/SFO Card Status

Message ID:	16 bit word, 21 (0x0015)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.

This message has no data component.

4.26.3 Usage Scope

Host to CSU only. Matrix GPIOs and EHX Controls supported only.

4.26.4 Performance

4.26.5 Example

This example asks the host for the current status of all the EHX Control cards.

00 15 08 AB BA CE DE 01

4.26.6 Related messages

Reply EHX Controls Card Status.

4.26.7 See also

4.27 Reply EHX Controls Card Status

4.27.1 Description

This message contains the current state of the Matrix GPIOs and EHX Control cards associated with the specified Control card. It is generated whenever pins on a control change state or in response to a Request EHX Control Card Status message.

4.27.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema	Card Count	Card Data [1]	Card Data [Count]
------------	-------	--------------	-----------------	------------	---------------	-------------------

Reply GPIO/SFO Card Status

Message ID:	1 byte, 22 (0x16)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Card Count:	16 bit word
Card Data: 16 bit word	
bits 0 - 12	card number
bit 13	set to 0
bit 14	card status, 0 = absent, 1 = present
bit 15	card type, 0 = GPIO card, 1 = SFO card

4.27.3 Usage

Card Count has the number of cards reported by the message. Each Card Data field reports the status of each card.

4.27.4 Scope

CSU to host only. Matrix GPIOs and EHX Controls supported only.

4.27.5 Related Messages

Request GPIO/SFO EHX Control Card Status.

4.27.6 See also

4.28 Request GPIO/SFO Status

4.28.1 Description

This message instructs the Central Switching Unit (CSU) to send the status of all of the general-purpose inputs (GPI) and general-purpose outputs (GPO) pins on the specified general purpose input/output (GPIO) card. This message is also used for special function outputs (SFO) in the same way.

4.28.2 Structure

Message ID	Protocol Tag	Protocol Schema	Flags	GPI card number
------------	--------------	-----------------	-------	-----------------

Request GPIO/SFO Status

Message ID:	1 byte, 23 (0x17)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
GPIO/SFO card number:	16 bit word, range 0 - 128
bit 15	card type, 0 = GPIO card, 1 = SFO card

4.28.3 Usage

The card number determines the type of input/output that will be reported. GPIO cards are normally in the range 0 - 12 and SFO "cards" are in the range 32 -128.

4.28.4 Scope

Host to CSU only. Matrix GPIOs and SFO supported only.

4.28.5 Performance

4.28.6 Example

This example asks the host for the current status of all of the pins on

GPIO/SFO card

00 10 17 AB BA CE DE 01 08 0A

4.28.7 Special notes

GPIO card numbers do not correspond to the slot position of the cards in the rack(s) of the CSU.

4.28.8 Related messages

Request GPO/SFO Actions, Reply GPIO/SFO Status.

4.28.9 See also

4.29 Reply GPIO/SFO Status

4.29.1 Description

This message contains the current state of the input and output pins on specified general-purpose input/output (GPIO) card or SFO "card". It is generated whenever a pin changes state on any GPIO or SFO card and in response to a Request GPIO/SFO Status message.

4.29.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema	Pin Count	Card Data	Pin Data [1]	Pin Data [Count]
------------	-------	--------------	-----------------	-----------	-----------	--------------	------------------

Reply GPIO/SFO Status

Message ID:	1 byte, 24 (0x18)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Pin Count:	16 bit word
Card Data: 16 bit word	
bits 0 - 12	card number, 0 -128
bits 13 - 14	set to 0
bit 15	set to 1
Pin data: 16 bit word	
bits 0 - 12	pin number, 0 - 31
bit 13	state of pin, 0 = off, 1 = on
bit 14	type of pin, 0 = output, 1 = input
bit 15	set to 0

4.29.3 Usage

When the message is a reply to a Request GPIO/SFO Status message, only the pins that are active are present in the message. When the message has been generated by the CSU in response to changes on the card, only the pins that have changed state are present in the message.

Pin Count holds the number of Pin Data items plus 1, so if three pins have changed, Pin Count will hold 4.

Card Data contains the logical number of the card being reported. GPIO cards are normally in the range 0 - 12 and SFO "cards" are normally in the range 28 - 128.

4.29.4 Scope

CSU to host only.

4.29.5 Performance

4.29.6 Example

This example shows the reply generated by the CSU when a Request

GPIO Status was sent by the host to find out what the current state of all the inputs on GPIO card 03. The outputs of pins 5, 12 and 25 are low (active), and the input pins 7, 14, 21 and 25 are low (active)

```
00 18 32 AB BA CE DE 01 00 08 80 03 20 05 60 07 20 0C 60 0E 60 15 20 19 60 19
```

4.29.7 Special notes

GPIO card numbers do not correspond to the slot position of the cards in the rack(s) of the CSU.

4.29.8 Related messages

Request GPO/SFO Actions, Request GPIO/SFO Status.

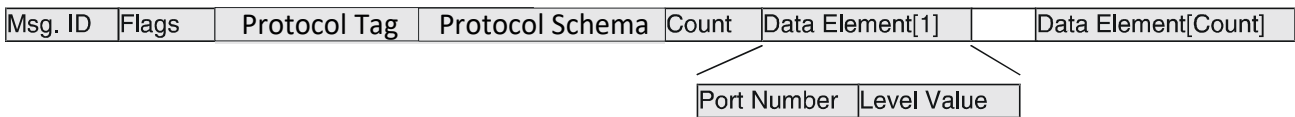
4.29.9 See also

4.30 Request Input Level Actions

4.30.1 Description

This message instructs the CSU to change the input gain (coming into the Matrix) for the specified input port(s) to the specified gain value(s).

4.30.2 Structure



Request Input Level Actions

Message ID:	16 bit word, 32 (0x0020)
Message Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16 bit word
Port number:	16 bit word, 0 - 1023
Level value:	16 bit word, 0 - 255

4.30.3 Usage

Count should be set to the number of data elements in the message. Gain (dB)= (Level Value - 204) x 0.355 except for a value of 0 which represents a cut (see APPENDIX B - GAIN LEVEL TRANSLATION TABLE).

Note: Input levels can be adjusted between the range of -72dB to +18dB. A gain level of 0 can also be set, this is the full cut gain level.

4.30.4 Scope

Host to CSU only.

4.30.5 Performance

4.30.6 Example

This example shows the data sent by the host computer to set the input levels of port 5 to +10dB, port 258 to +5dB and port 305 to -5dB.

00 20 08 AB BA CE DE 01 00 03 00 05 00 E8 01 02 00 DA 01 31 00 BD

4.30.7 Special notes

4.30.8 Related messages

Request Input Level Status, Reply Input Level Status.

4.30.9 See also

4.31 Request Input Level Status

4.31.1 Description

This message asks the CSU to send the host information about the current settings of all the audio input levels.

4.31.2 Structure

Msg. ID	Flags	Protocol Tag	Protocol Schema
---------	-------	--------------	-----------------

Request Input Level Status

Message ID:	16 bit word, 33 (0x0021)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.

This message has no data component.

4.31.3 Usage

This message will cause the CSU to generate a Reply Input Level Status message containing the current settings of all the audio input levels for all input ports.

4.31.4 Scope

Host to CSU only.

4.31.5 Performance

4.31.6 Example

This message was sent when the host computer needed to re-establish the current input level status after being disconnected for a period of time.

00 21 08 AB BA CE DE 01

4.31.7 Special notes

None

4.31.8 Related messages

Request input Level Actions, Reply Input Level Status.

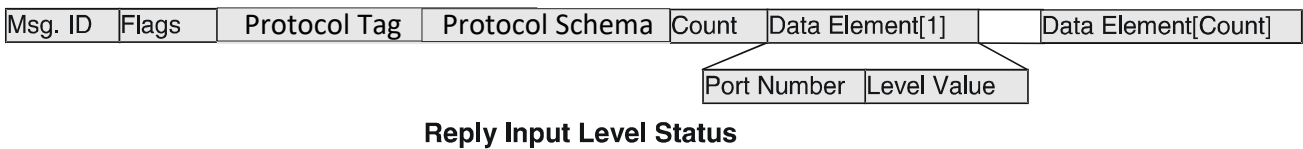
4.31.9 See also

4.32 Reply Input Level Status

4.32.1 Description

This message is generated by the CSU when a Request Input Level Status or Actions message has been received. Only non-zero values will be included in the reply to this request.

4.32.2 Structure



Message ID:	16 bit word, 34 (0x0022)
Message Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16 bit word
Port number:	16 bit word, 0 - 1023
Level value:	16 bit word, 0 - 255

4.32.3 Usage

Count holds the number of data elements in the message.

The level value corresponds to a gain value described in appendix C.

When issued in response to a status request, the update flag will not be set and the data will list all ports, excluding those set to 0dB. When issued in response to an action request, the update flag will be set, and the data will list all ports altered as a result of the action request including those set to 0dB, but excluding those that failed to change due to an incorrect request.

4.32.4 Scope

CSU to host only.

4.32.5 Performance

4.32.6 Example

This message was sent by the CSU to report the current input levels of ports 48 to 63. Port 48 was set to +9dB, port 49 was set to -6dB and the remaining 14 ports were set to 0dB.

00 22 34 AB BA CE DE 01 00 02 00 30 00 E5 00 31 00 BB

4.32.7 Special notes

None.

4.32.8 Related messages

Request Input Level Actions, Request Input Level Status

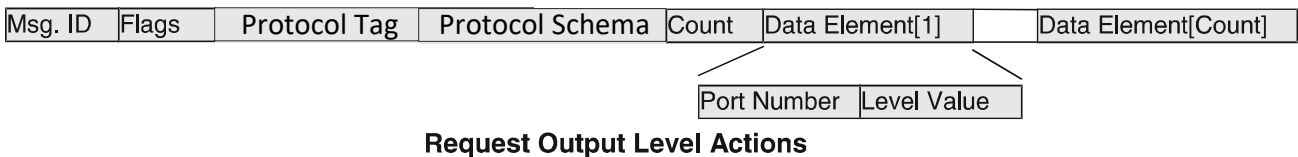
4.32.9 See also

4.33 Request Output Level Actions

4.33.1 Description

This message asks the CSU to change the Output signal level(s) leaving the CSU (matrix) of the specified output port(s) to the new gain value(s) in the range -72dB to +18dB. A gain level of 0 can also be used indicating full cut.

4.33.2 Structure



Message ID:	16 bit word, 35 (0x0023)
Message Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16 bit word
Port number:	16 bit word, 0 - 1023
Level value:	16 bit word, 0 - 255

4.33.3 Usage

Count should be set to the number of data elements in the message.

The output Level Values are calculated from the formula $\text{gain (dB)} = (\text{Level Value} - 204) \times 0.355$

except for a value of 0 which represents a cut (see appendix C).

4.33.4 Scope

Host to CSU only.

4.33.5 Performance

4.33.6 Example

This example shows the data sent by the host computer to set the output levels of port 5 to +6dB, port 258 to -6dB and port 305 to -12dB.

00 23 08 AB BA CE DE 01 00 03 00 05 00 DD 01 02 00 BB 01 31 00 AA

4.33.7 Special notes

None.

4.33.8 Related messages

Request Output Level Status, Reply Output Level Status.

4.33.9 See also

4.34 Request Output Level Status

4.34.1 Description

This message asks the CSU to send the host information about the current settings of the audio output levels of all audio output ports. Only non-zero values will be included in the reply to this request.

4.34.2 Structure

Msg. ID	Flags	Protocol Tag	Protocol Schema
---------	-------	--------------	-----------------

Request Output Level Status

Message ID:	16 bit word, 36 (0x0024)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.

This message has no data component.

4.34.3 Usage Scope

Host to CSU only.

4.34.4 Performance

4.34.5 Example

This message was sent when the host computer needed to re-establish the current output level status after being disconnected for a period of time.

00 24 08 AB BA CE DE 01

4.34.6 Special notes

None

4.34.7 Related messages

Request Output Level Actions, Reply Output Level Status.

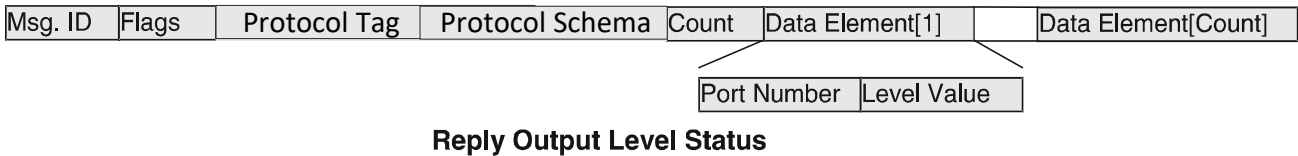
4.34.8 See also

4.35 Reply Output Level Status

4.35.1 Description

This message is generated by the CSU in response to a Request Output Level Status message.

4.35.2 Structure



Message ID:	16 bit word, 37 (0x0025)
Message Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16 bit word
Port number:	16 bit word, 0 - 1023
Level value:	16 bit word, 0 - 255

4.35.3 Usage

Count holds the number of data elements in the message.

When issued in response to a status request, the update flag will not be set, and the data will list all ports, excluding those set to 0dB. When issued in response to an action request, the update flag will be set, and the data will list all ports altered as a result of the action request including those set to 0dB, but excluding those that failed to change due to an incorrect request.

4.35.4 Scope

CSU to host only.

4.35.5 Performance

4.35.6 Example

This message was sent by the CSU to report the current output levels of ports 48 to 63. Port 48 was set to +9dB, port 49 was set to -6dB and the remaining 14 ports were set to 0dB.

00 25 34 AB BA CE DE 01 00 02 00 30 00 E5 00 31 00 BB

4.35.7 Special notes

None.

4.35.8 Related messages

Request Output Level Actions, Request Output Level Status

4.35.9 See also

4.36 Request Panel Status

4.36.1 Description

This message asks the CSU to send the host information about the current state (online / offline) that of the panels/endpoints that are available to the system.

4.36.2 Structure

Msg. ID	Flags	Protocol Tag	Protocol Schema
---------	-------	--------------	-----------------

Request Panel Status

Message ID:	16 bit word, 5 (0x0005)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.

4.36.3 Usage

This message may be used at any time to generate a Reply Panel Status message and has no data component.

4.36.4 Scope

Host to CSU only.

4.36.5 Performance

4.36.6 Example

This example shows the message sent by the Host.

00 05 08 AB BA CE DE 01

4.36.7 Special notes

None

4.36.8 Related messages

Reply Panel Status.

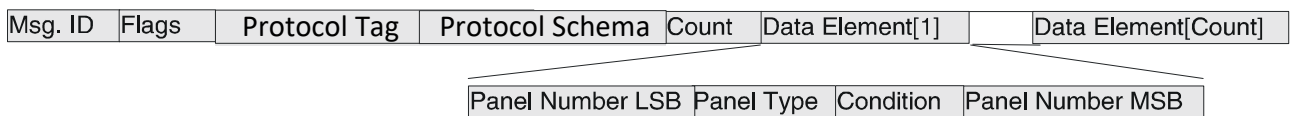
4.36.9 See also

4.37 Reply Panel Status

4.37.1 Description

This message is sent by the CSU in response to a Request Panel Status message or whenever the state of a panel/endpoint state changes. A complete list of panels that are expected will be generated in response to a Request Panel Status message. Only the panels that have changed state are reported in an automatically generated reply message.

4.37.2 Structure



Reply Panel Status

Message ID:	16 bit word, 30 (0x001E)
Message Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16 bit word
Panel number LSB:	1 byte
Panel type:	PANEL_GENERIC == 1 PANEL_FS1_ANTENNA_1_9_GHZ == 2 PANEL_FS2_ANTENNA_1_9_GHZ == 3 PANEL_FS2_ANTENNA_2_4_GHZ == 4 PANEL_ANTENNA_TYPE_UNKNOWN == 5 PANEL_DIRECT == 6 // Includes hosted direct PANEL_FS2_ANTENNA_5_0_GHZ == 7 PANEL_AES67_DIRECT == 8

Condition: 1 byte	
State	7 bits, 0 = unknown, 1 = good, 2 = faulty, 3 = reserved
AoIP Device	1 bit, 1 = AoIP device
Panel num MSB:	1 byte

4.37.3 Usage

Count holds the number of data elements in the message.

The panel number is made from the two bytes, Panel Number LSB (least significant byte) and Panel Number MSB (most significant byte).

4.37.4 Scope

CSU to host only.

4.37.5 Performance

4.37.6 Example

This message was sent by the CSU after panel 11 was disconnected from the CSU. The panel previously had key monitoring enabled.

00 1E 34 AB BA CE DE 01 00 01 0B 01 82 00

4.37.7 Special notes

None.

4.37.8 Related messages

Request Panel Status.

4.37.9 See also

4.38 Request System Messages

4.38.1 Description

This message is used to request the broadcast of System event log messages

4.38.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema	Unused
------------	-------	--------------	-----------------	--------

Request System Messages

Message ID:	16 bit word, 2 (0x0002)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Unused:	16 bit word

4.38.3 Usage

System event log messages are stored in the CSU until requested to be broadcast by this message.

System messages are transmitted at the time of generation. A maximum of 600 messages can be stored in the CSU awaiting transmission. Once 600 have been stored, event log messages below the class of WARNING are discarded. When the store is full the oldest messages will be discarded in favour of newer messages.

4.38.4 Scope

Host to CSU only.

4.38.5 Performance

4.38.6 Example

00 02 08 AB BA CE DE 01 00 00

4.38.7 Special notes

None.

4.38.8 Related messages

Broadcast System Message Request System Status, Request System

4.39 Request System Status

4.39.1 Description

This message causes the CSU to generate a reply containing the current status of all the cards and rear connector units in the CSU.

4.39.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema
------------	-------	--------------	-----------------

Request System Status

Message ID:	16 bit word, 3 (0x0003)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.

This message has no data component.

4.39.3 Usage

4.39.4 Performance

Processor time required: < 10ms.

Response time: < 150 ms until message generation. Maximum message frequency: N/A

4.39.5 Scope

Host to CSU only.

4.39.6 Example

This example was sent by the host to find out what the current status of the cards in the CSU was.

00 03 08 AB BA CE DE 01

4.39.7 Special notes

None.

4.39.8 Related messages

Reply System Status

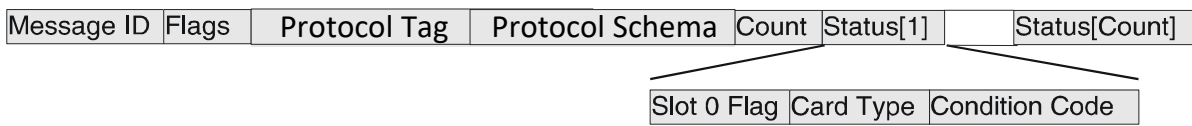
4.39.9 See also

4.40 Reply System Card Status

4.40.1 Description

This message contains the current status of all the cards in the CSU. It is sent whenever a card changes state and in response to a request system status message.

4.40.2 Structure



Reply System Card Status

Message ID:	16 bit word, 4 (0x0004)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16 bit word
Status: 16 bit word	
bits 0 - 7	Condition code
	0 - Unknown
	1 = Good
	2 = Faulty/absent
bits 8 - 14	Card type
	0 = unknown
	1 = CPU Master

	2 = CPU Slave
	3 = MVX
	4 = E-MADI
	5 = E-DANTE
	6 = E-IPA
	23 = PCM30
	26 = E-FIB
	27 = EQU/IVC32/LMC64
	11 = reserved
	14 = reserved
	16 - 18 = reserved
	24 - 127 = reserved
bit 15	Slot 0 flag
	0 = this is not slot 0 of the rack
	1 = this is slot 0 of the rack

4.40.3 Usage

The slots in a rack always number slot 0 upwards, such that CPU cards are always in slot 0 and/or slot 1.

4.40.4 Performance

Processor time required: < 20ms.

Response time: < 150ms.

Maximum message frequency: 5 per second.

4.40.5 Scope

CSU to host only.

4.40.6 Special notes

None.

4.40.7 Related messages

Request System Status, Reply System RCU status.

4.40.8 See also

4.41 Request Panel Keys Status

4.41.1 Description

Requests the latch status of keys on a specific panel or role

4.41.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema	Slot	Port
------------	-------	--------------	-----------------	------	------

Field	Description
Message ID:	16 bit word 177 (0x00B1).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Slot	1 byte, Card Slot Number 4.4 Slot numbering scheme
Port offset	1 byte, Port offset from first port of the card

4.41.3 Usage

Requests the latched status of all the keys on the specified panel.

4.41.4 Scope

Host to CSU only.

4.41.5 Performance

This request may produce a lot of messages from the CSU to the host.

4.41.6 Example

Request the all keys status on Panel 20.

00 B1 08 AB BE CE DE 01 00 14

4.41.7 Related messages

Reply Panel Key Status

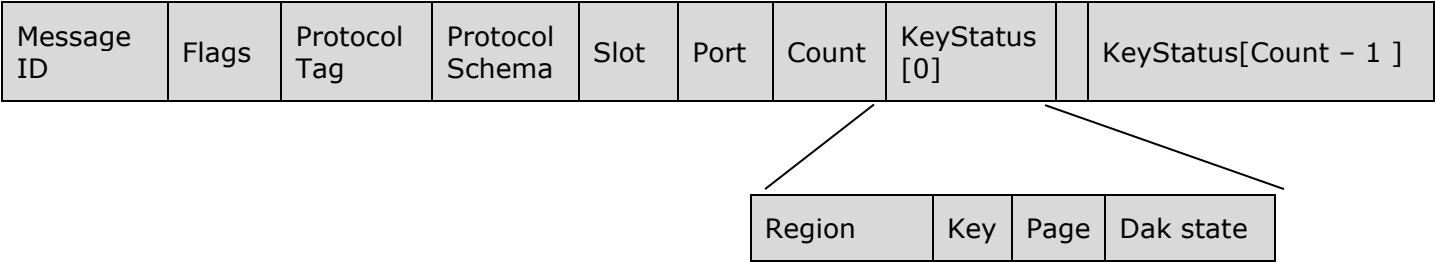
4.41.8 See also

4.42 Reply Panel Keys Status

4.42.1 Description

Returns the status of all the keys available on the requested panel or role.

4.42.2 Structure



Field	Description
Message ID:	16 bit word 178 (0x00B2)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Slot:	1 byte, Card Slot Number 4.4 Slot numbering scheme
Port offset:	1 byte, Port offset from first port of the card
Count:	16 bit, Number of keys statuses for this panel. This all key status held even paged(shift page) statuses.
Region:	1 byte, Region on this panel.
Key:	1 byte, Key on this region.
Page:	1 byte, Page on this region

Dak state:	1 byte
Key_state	Bits 0 & 1, 0 = OFF, 1 = ON
Listen_mode	Bit 2, Comb Key on istation set to listen mode
Reserved	Bit 3
Pot_state	Bits 4 to 7, Pot Position
Color	1 byte – Colour index as set in EHX

4.42.3 Usage

Sends the latch status of all keys on a specified panel, including keys on different pages and regions.

4.42.4 Scope

Host to CSU only.

4.42.5 Performance

4.42.6 Special notes

Note that many messages may be required to be transmitted by the CSU to the host as it is expected the status data for all keys on a panel may be larger than the maximum data size for a message, in that flags will be used as in section 4.2 of the HCI document.

4.42.7 Related messages

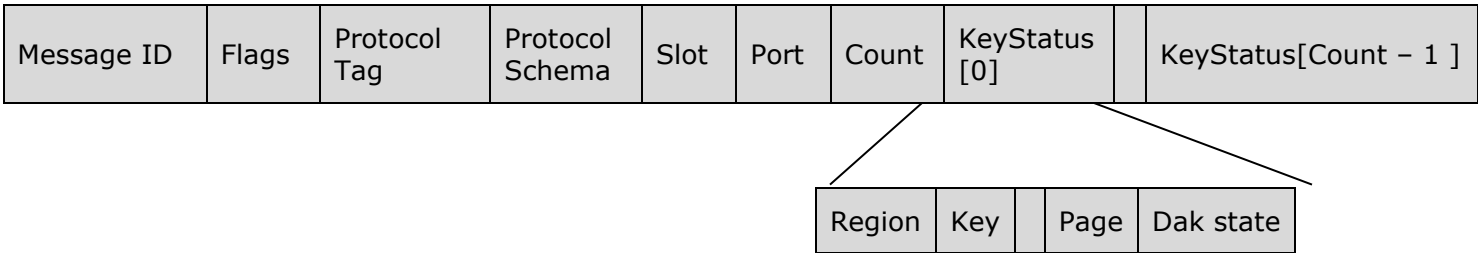
4.42.8 See also

4.43 Request Panel Keys Action

4.43.1 Description

Sets the state of one or more keys on a specified panel.

4.43.2 Structure



Field	Description
Message ID:	16 bit word 179 (0x00B3)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Slot:	Card Slot Number 1 byte 4.4 Slot numbering scheme
Port:	Port offset from first port of the card 1 byte
Count:	Number of key update elements in this message
Region:	Region on this panel. 1 byte
Key:	Key on this region. 1 byte
Page:	Page on this region 1 byte
Dak state:	1 byte

Key_state	0 = OFF, 1 = ON, 2 = ON2(internal use only) Bits 0 - 1
Listen_mode	Comb Key on istation set to listen mode Bit 2
Reserved	Bit 3
Pot_state	Pot Position Bits 4 - 7

4.43.3 Usage

Allows the host to toggle the on / off state of a specific key(s) on a panel.

4.43.4 Scope

Host to CSU only.

4.43.5 Special notes

Note: Many messages may be required to be transmitted by the host to the CSU as it is expected the status data for all keys on a panel may be larger than the maximum data size for a message, in that case the flags will be used as in section 4.2 of the HCI document.

4.43.6 Performance

4.43.7 Related messages

4.43.8 See also

4.44 Reply Panel Keys Status

4.44.1 Description

Returns the status of all the keys toggled on/ off by the request panel key action request

4.44.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema	Slot	Port	Count
------------	-------	--------------	-----------------	------	------	-------

<i>Field</i>	<i>Description</i>
Message ID:	16 bit word 180(0x00B4)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Slot:	1 byte Card Slot Number 4.4 Slot numbering scheme
Port:	1 byte Port offset from first port of the card
Count:	Number of keys statuses received for this panel. 16 bit word

4.44.3 Usage

- Used to acknowledge reception of the panel key action request from the Host.

4.44.4 Scope

CSU to Host only.

4.44.5 Special notes

4.44.6 Performance

4.44.7 Example

One key status received and processed for the panel on the first port on the card in slot 3.

00 C8 30 AB BA CE DE 01 03 00 00 01

4.44.8 Related messages

4.44.9 See also

4.45 Request Port Info

4.45.1 Description

Request the connected port type (panel, interface, trunk, FreeSpeak BP) and some additional port information for example firmware and port settings per card

4.45.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema	Slot Number
------------	-------	--------------	-----------------	-------------

Field	Description
Message ID:	16 bit word 183 (0x00B7).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Slot Number:	Card slot number. 16 Bit Word 4.4 Slot numbering scheme

4.45.3 Usage

Can be used to retrieve port information from a configured/semi-configured card.

4.45.4 Scope

Host to CSU only.

4.45.5 Performance

4.45.6 Example

Request Information for first MVX Slot 1. (Slot numbers start at 1, so the first MVX is slot 3)

00 B7 30 AB BA CE DE 01 00 03

4.45.7 Related messages

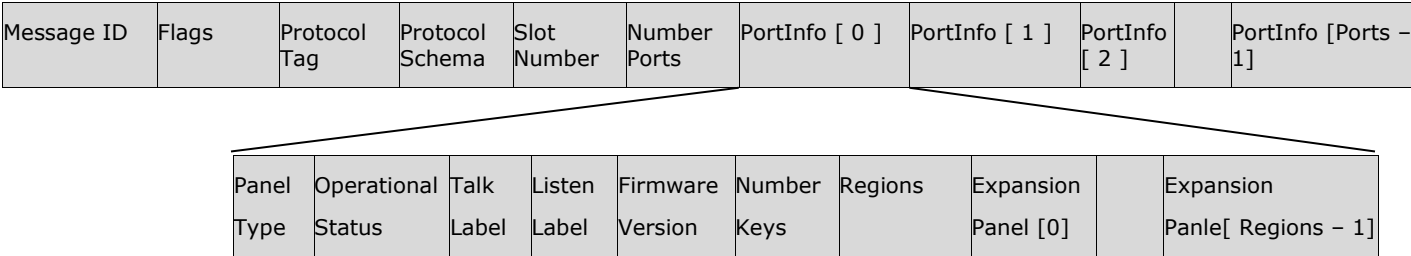
4.45.8 See also

4.46 Reply Port Info

4.46.1 Description

Returns the connected port type (panel, interface, trunk, FreeSpeak BP) and some additional port information for example firmware and port settings per card

4.46.2 Structure



Field	Description
Message ID:	16 bit word 184 (0x00B8)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Slot Number:	Card slot number, 2 bytes 4.4 Slot numbering scheme
Number Ports:	Number of ports in this response. 1 byte
The fields below are repeated for each port in the message	
Port number:	The reported port 16 bit word
Panel Type:	Type ID of this panel, last online ID 16 bit word iStation 0x8000 ICS 1016 0x8008 ICS 1008 0x800E V-Series 1U Lever 0x8010 V-Series 1U Push 0x8011 V-Series 2U Lever 0x8012 V-Series 2U Push 0x8013 V-Series Desk Lever 0x8014 V-Series Desk Push0x8015

	V-Series 1U Rotary 0x8016 V-Series 2U Rotary 0x8019 V-Series Desk Rotary 0x801A V-Series 2U (32 Key) 0x8020 CCI-22 0x8100 FOR-22 0x8102 Tel-14/SIP 0x8106 LQ 0x8110 Edge Beltpacks 0x8204
Panel Operational Status: 1 byte	
Online	0 = offline, 1 = online Bit 0
SubType	Bit 1 Usage – If 1 this means the following depending on panel type LQ Panel Type – LQ V-Series Panel Type – V-Series Iris Panel Generation
Port Type Bits 2 – 7	NULL 0 Beltpack 1 Wireless Port (E-IPA) 2 Transceiver (EQue) 3 EQue Direct 4 Speed Dial 5 FOR-22 6 PABX 8 Direct 10 EQue Trunk 11 Panel 14 Trunk (non EQue) 18 Tel-14/SIP 19 Fibre Trunk 25 EQue T1 Trunk 27 LMC 28 CCI-22 29 HelixNet 30 EQue Direct (T1 E&M) 31 EQue Direct (E1 E&M) 31 EQue Direct (T1) 31

Panel Firmware:	8 byte string 8 bytes
Boot Version:	(major(1).minor(1).revision(2)) 4 bytes
Kernel Version:	(major(1).minor(1).revision(2)) 4 bytes
FileSystem Version:	(major(1).minor(1).revision(2)) 4 bytes
Reserved:	Unused, reserved for future expansion. 4 bytes
Talk & Listen Label:	Talk & Listen Label for this panel 10 words
Talk & Listen Alias Label:	Talk Label for this panel 10 words
Number Of Keys:	Number of keys on panel 8 bits
AnswerBackTimeout:	Answer back timeout for panel(1-60 Secs), Note:- 0 means timeout disabled. 8 bits
Number of Expansion panels:	Number of expansion panels supported 8 bits
Expansion Panel start region	The region the first expansion panel starts at 8 bits
Expansion Panel [0]	Region 1 byte
	Type 16 bit word
	Current Page 1 byte
	App Version 4 bytes Where: 1byte = major 1byte = minor 2bytes = revision
	Boot Version 4 bytes Where: 1byte = major 1byte = minor 2bytes = revision
Expansion Panel [Regions - 1]	Region 1 byte
	Type 16 bit word – 1 == Lever, 2 == Push, 3 == Rotary, 4 == 16 key lever (V32), 0 == Not set
	Current Page 1 byte
	App Version 4 bytes
	Boot Version 4 bytes

4.46.3 Usage

Used to return the port type and panel information and expansion panels.

4.46.4 Scope

CSU to Host only.

4.46.5 Special notes

Note: Many messages may be required to be transmitted by the CSU to the host as it is expected the panel information data may be larger than the maximum data size for a message, in that case the flags will be used as in section 4.2 of the HCI document.

4.46.6 Performance

4.46.7 Related messages

4.46.8 See also

4.47 Request Locally Assigned Keys

4.47.1 Description

Requests all locally assigned key configuration for selected panel. A locally assigned key is a key that has been assigned by means other than map configuration download e.g. EHX online key assign, HCI API assignment or panel fast key assign, scroll group key assignment etc.

4.47.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema	Slot Number	Port
------------	-------	--------------	-----------------	-------------	------

Field	Description
Message ID:	16 bit word 185 (0x00B9).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Slot:	Card Slot Number 1 byte 4.4 Slot numbering scheme
Port:	Port offset from first port of the card 1 byte

4.47.3 Usage

Can be used to retrieve locally assigned keys for a particular panel.

4.47.4 Scope

Host to CSU only.

4.47.5 Performance

4.47.6 Example

Request Information for Slot 2 Port Offset 1

00 B9 08 AB BA CE DE 01 02 01

4.47.7 Related messages

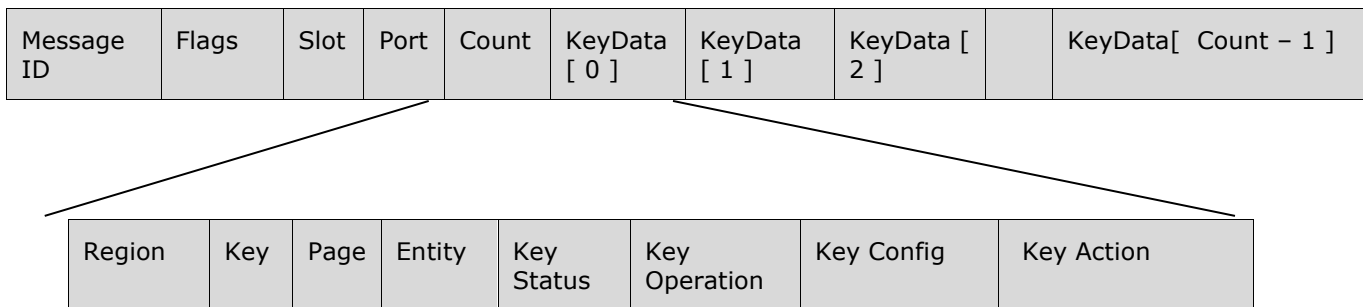
4.47.8 See also

4.48 Reply Locally Assigned Keys

4.48.1 Description

Returns the configuration of all locally assigned keys for selected panel. A locally assigned key is a key that has been assigned by means other than map configuration download e.g. EHX online key assign, HCI API assignment or panel fast key assign, scroll group key assignment etc.

4.48.2 Structure



Field	Description	Size
Message ID:	16 bit word 186, (0x00BA)	
Flags:	1 byte, see FLAGS on page 59.	
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.	
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.	
Slot:	Card Slot Number 1 byte 4.4 Slot numbering scheme	
Port:	Port offset from first port of the card 1 byte	

Count:	Number of locally assigned keys on this panel. 1 byte	
Region:	Region this key is on 1 byte	
Key:	This keys Id 1 byte	
Page:	Page this key is on 1 byte	
Entity:	e.g EN_PORT, EN_GROUP 16 bit word	
Key Status:		
KeyState	Bits 6 to 7	
ListenMode	Bit 5	
PotAssigned	Bit 4	
PotState	Bits 0 to 3	
PotNumber	16 bit word	
Key Operation: 4 bytes		
Unpaged	0 paged, 1 unpaged Bit 8	
TextMode	0 not displayed, displayed Bit 7	
Dual	0 normal, 1 double width Bit 6	
Dial	0 dial mode off, 1 dial mode on Bit 5	
LatchMode Bits 0 – 3	0 – LATCH_NON_LATCH 1 – LATCH 2 – NON_LATCH 3 – RETURN_AFTER_HOLD 4 – TRI_ROT 5 – TRI_TOG 6 – RETURN_AFTER_LATCH 7 – DEL_LATCH 8 – HKS_LATCH	

Group Bits 4 – 7	If non-zero, other keys on the same panel with the same group number will be deselected if this key is selected	
Deactivating	1 all interlock group keys may be off Bit 3	
Make/Break	1 interlock group make before break Bit 2	
Cross Page	1 keys interlocked across pages in same region Bit 1	
CMAPSi Sp1	CMAPSi flag special 1 Bit 0	
CMAPSi Sp2	CMAPSi flag special 2 Bit 7	
Region Value Bit 4 – 6	Key region number	
Stacked group	If set indicates that the key is a stacked group Bit 3	
unused	Bit 0 – 2	
Page Value	If the key is a page key, this value is the page it makes a switch to when pressed, 1 byte	
Key Config: 26 bytes		
System Number	System Number 1 byte	
Unused	Spare 1 byte	
Specific	Use depends on entity 16 Bit Word	
Secondary	Dcc/DialCode(speed_dial) 16 Bit word	
KeyAction: 16 Bit word		
Force Listen	0 disable, 1 enable Bit 15	
Talk	0 disable, 1 enable Bit 14	
Listen	0 disable, 1 enable Bit 13	
Hold to Talk	0 disable, 1 enable Bit 12	

Initial State	0 not selected, 1 selected Bit 11	
Assign Locally	0 disable, 1 enable Bit 10	
Assign Remotely	0 disable, 1 enable Bit 9	
Locally Assigned	0 un-assigned, 1 assigned Bit 8	
Reserved	Unused bits 0 – 7	
GUID	Unique ECS GUID Number 16 Bytes	

4.48.3 Usage

Used to return locally assigned key information for selected panel.

Missing byte info

4.48.4 Scope

CSU to Host only.

4.48.5 Special notes

Note: Many messages may be required to be transmitted by the CSU to the host as it is expected the local key information data may be larger than the maximum data size for a message, in that case the flags will be used as in section 4.2 of the HCI document.

4.48.6 Performance

4.48.7 Example

4.48.8 Related messages

Request Locally Assigned Keys.

4.48.9 See also

4.49 Request Assigned Keys

4.49.1 Description

Requests all assigned key configuration for selected panel. These assignments are the net result of the map configuration downloaded baseline plus any HCI, online, panel based assignments on top of this.

4.49.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema	Slot Number	Port
------------	-------	--------------	-----------------	-------------	------

Field	Description
Message ID:	16 bit word 231 (0x00E7).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte. Schema 1 and 2 currently supported. The request format is the same for both schemas, but more information is returned in the reply when schema 2 is requested.
Slot:	Card Slot Number 1 byte 4.4 Slot numbering scheme
Port:	Port offset from first port of the card 1 byte

4.49.3 Usage

Can be used to retrieve locally assigned keys for a particular panel.

4.49.4 Scope

Host to CSU only.

4.49.5 Performance

4.49.6 Example

Request Information for Slot 2 Port Offset 1

00 B9 08 AB BA CE DE 01 02 01

4.49.7 Related messages

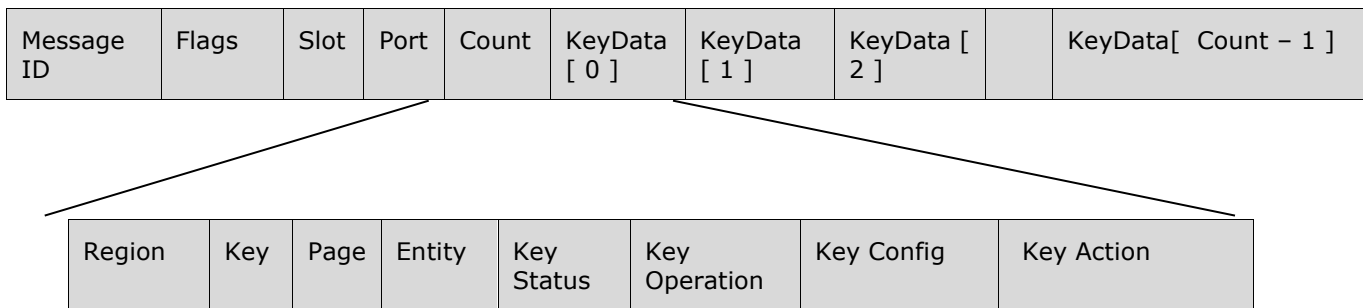
4.49.8 See also

4.50 Reply Assigned Keys

4.50.1 Description

Returns the configuration of all assigned keys for selected panel. These assignments are the net result of the map configuration downloaded baseline plus any HCI, online, panel-based assignments on top of this.

4.50.2 Structure



Field	Description	Size
Message ID:	16 bit word 232, (0x00E8)	
Flags:	1 byte, see FLAGS on page 59.	
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.	
Protocol Schema:	1 byte. Schema 1 and 2 supported.	
Slot:	Card Slot Number 1 byte 4.4 Slot numbering scheme	
Port:	Port offset from first port of the card 1 byte	
Endpoint Type:	2 bytes. Only include this field if schema 2 was requested.	

	The field is only used when the slot and offset fields above target a role. If this is not a role target then this field is set to 0. This field states the endpoint type currently used by the role e.g. 0x8200 would signify that the FS II beltpack is currently in use and therefore that these are FS II BP keys.	
Count:	Number of locally assigned keys on this panel. 2 bytes	
Region:	Region this key is on 1 byte	
Key:	This keys Id 1 byte	
Page:	Page this key is on 1 byte	
Entity:	e.g EN_PORT, EN_GROUP 16 bit word	
Key Status:		
KeyState	Bits 6 to 7	
ListenMode	Bit 5	
PotAssigned	Bit 4	
PotState	Bits 0 to 3	
PotNumber	16 bit word	
Key Operation: 4 bytes		
Unpaged	0 paged, 1 unpaged Bit 7	
TextMode	0 not displayed, displayed Bit 6	
Dual	0 normal, 1 double width Bit 5	
Dial	0 dial mode off, 1 dial mode on Bit 4	
LatchMode Bits 0 – 3	0 – LATCH_NON_LATCH 1 – LATCH 2 – NON_LATCH 3 – RETURN_AFTER_HOLD 4 – TRI_ROT	

	5 – TRI_TOG 6 – RETURN_AFTER_LATCH 7 – DEL_LATCH 8 – HKS_LATCH	
Group Bits 4 – 7	If non-zero, other keys on the same panel with the same group number will be deselected if this key is selected	
Deactivating	1 all interlock group keys may be off Bit 3	
Make/Break	1 interlock group make before break Bit 2	
Cross Page	1 keys interlocked across pages in same region Bit 1	
CMAPSi Sp1	CMAPSi flag special 1 Bit 0	
CMAPSi Sp2	CMAPSi flag special 2 Bit 7	
Region Value Bit 4 – 6	Key region number	
Stacked group	If set indicates that the key is a stacked group Bit 3	
unused	Bit 0 – 2	
Page Value	If the key is a page key, this value is the page it makes a switch to when pressed, 1 byte	
Key Config: 26 bytes		
System Number	System Number 1 byte	
Unused	Spare 1 byte	
Specific	Use depends on entity 16 Bit Word	
Secondary	Dcc/DialCode(speed_dial) 16 Bit word	
KeyAction: 16 Bit word		
Force Listen	0 disable, 1 enable Bit 15	

Talk	0 disable, 1 enable Bit 14	
Listen	0 disable, 1 enable Bit 13	
Hold to Talk	0 disable, 1 enable Bit 12	
Initial State	0 not selected, 1 selected Bit 11	
Assign Locally	0 disable, 1 enable Bit 10	
Assign Remotely	0 disable, 1 enable Bit 9	
Locally Assigned	0 un-assigned, 1 assigned Bit 8	
Reserved	Unused bits 0 – 7	
GUID	Unique ECS GUID Number 16 Bytes	

4.50.3 Usage

Used to return locally assigned key information for selected panel.

Missing byte info

4.50.4 Scope

CSU to Host only.

4.50.5 Special notes

Note: Many messages may be required to be transmitted by the CSU to the host as it is expected the local key information data may be larger than the maximum data size for a message, in that case the flags will be used as in section 4.2 of the HCI document.

4.50.6 Performance

4.50.7 Example

4.50.8 Related messages

Request Locally Assigned Keys.

4.50.9 See also

4.51 Request Card Info

4.51.1 Description

Retrieves the card information at a specified slot together with its health status.

4.51.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema
------------	-------	--------------	-----------------

Field	Description
Message ID:	16 bit word 195 (0x00C3)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Slot:	Slot number 1 byte 4.4 Slot numbering scheme

4.51.3 Usage

Used to request the status of all matrix cards.

4.51.4 Scope

4.51.5 Related messages

Reply Card Info

4.51.6 Performance

4.51.7 Example

Requesting the card info for the card in slot 3 i.e. the first audio slot.

00 C3 30 AB BA CE DE 01 03

4.51.8 Related messages

Reply Card Info.

4.51.9 See also

4.52 Reply Card Info

4.52.1 Description

Returns the card information at a specified slot together with its health status.

4.52.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema	Count	CardInfo [0]		CardInfo[Count - 1]
------------	-------	--------------	-----------------	-------	--------------	--	----------------------

Field	Description	
Message ID:	16 bit word 196 (0x00C4)	
Flags:	1 byte, see FLAGS on page 59.	
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.	
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.	
Count:	Number of card infos to return. 1 byte	
Slot type:	Slot type,-1 byte See below for info	
Expected Card type:	Card type, 1 byte See below for info	
Current Card type:	card_type,-1 byte	
Health:	Health of the card see below 1 byte	
Rear connector unit:	If set indicates a rear connector card 1 bit :0	
unused:	Unused bits :1 to 7	
Rack Number:	Rack number starting from 1 1 byte	
Slot Number:	Slot number, 1 byte 4.4 Slot numbering scheme	

First port:	First port on card 2 bytes	
Last port:	Last port on card 2 bytes	
Channels:	Number of channels (typically number of ports) 1 byte	
Dtmf board present:	MVX card only, bit map of dtmf cards. 0xFFFF indicates all 16 ports have a dtmf encoder/decoder 2 bytes	
App Version Str:	Version string. 64 bytes	
Boot Version Str:	Boot Version String 64 bytes	
Fpga Version Str:	Fpga Version String 64 bytes	

Health of card

0	PCB_GOOD	<i>/* Correct type fitted 164nitializat and believed working */</i>
1	PCB_ABSENT	<i>/* Cannot detect any card */</i>
2	PCB_FAULTY	<i>/* Correct card believed busted permanently */</i>
3	PCB_DETECTED	<i>/* Card detected but slot ID not yet stable */</i>
4	PCB_WAITING	<i>/* Correct card first/later found in need of 164nitialization */</i>
5	PCB_MISFIT	<i>/* Detected wrong type of card */</i>
6	PCB_POWERINGUP	<i>/* Correct card only just inserted considered not yet stable */</i>

7	PCB_INITIALISING	<i>/* Correct card stable for ready state to be checked */</i>
8	PCB_SUSPECT	<i>/* Correct card failed some tests but maybe still usable */</i>
9	PCB_UNTESTED	<i>/* Not yet tested – this is only used for FRM channels */</i>

Slot Type

0	NO_SLOT
1	CPU_SLOT
2	DCC_SLOT
3	AUDIO_SLOT

Card Type

0	UNKNOWN
1	CPUMASTER
2	CPUSLAVE
3	MVX
4	MADI
26	FIBRE
27	EQUE/IVC32/LMC64

4.52.3 Usage

Sent back to HCI client in response to the Request Card Info message.

4.52.4 Scope

4.52.5 Related messages

Request Card Info.

4.52.6 See also

4.53 Request Conference / Fixed Group Members Edits

4.53.1 Description

Request all locally edit to members of all Fixed group or Conference (partyline).

4.53.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema	Edit Type	GUID
------------	-------	--------------	-----------------	-----------	------

Field	Description
Message ID:	16 bit word 197 (0x00C5)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Edit Type:	2 bytes (GROUP = 3, EN_CONF = 2)
Unused:	16 bytes – set to all 0's

4.53.3 Usage

4.53.4 Scope

Allows the host to request all members of all Conferences or Fixed Groups

Note Conferences / Fixed Groups are Global, and members may have been added from remote CSUs (matrices)

4.53.5 Performance

4.53.6 Example

Request to get all members for all conference edits.

00 C5 30 AB BA CE DE 01 00 02 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

4.53.7 Related messages

Reply Conference Assignments.

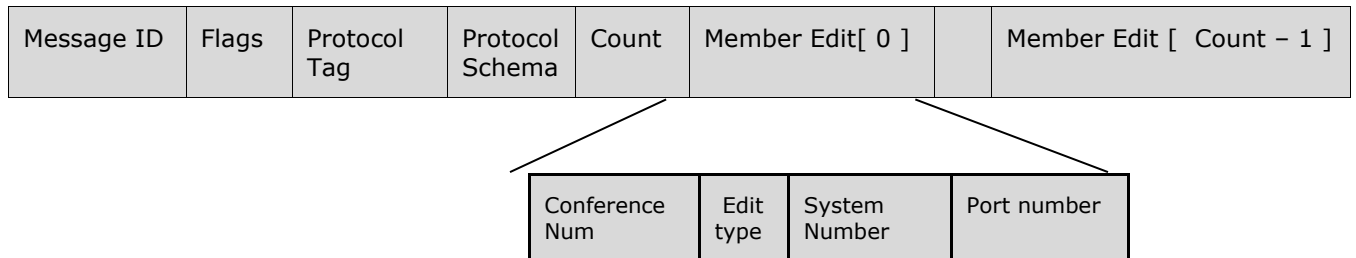
4.53.8 See also

4.54 Reply Conference Assignments

4.54.1 Description

Returns members of all conferences specifying whether it is a talk, listen and whether it has been edited locally.

4.54.2 Structure



Field	Description
Message ID:	16 bit word 198 (0x00C6)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	Count of items returned 16 bit word
Member edit:	
Conference / Number	Conference / number starting at 1 16 bit word
Edit Type	Bit masks 1 byte Talk = 0x01 Listen = 0x02

	Local deleted = 0x04 Local Assigned = 0x08 Map Assigned = 0x10 Local Override = 0x20
Full Port Identifier:	
System number	1 to 15 1 byte
Port number:	1 to 1023 16 bit word

4.54.3 Usage

4.54.4 Scope

4.54.5 Performance

4.54.6 Example

4.54.7 Related messages

Request Conference / Fixed Group Members

4.54.8 See also

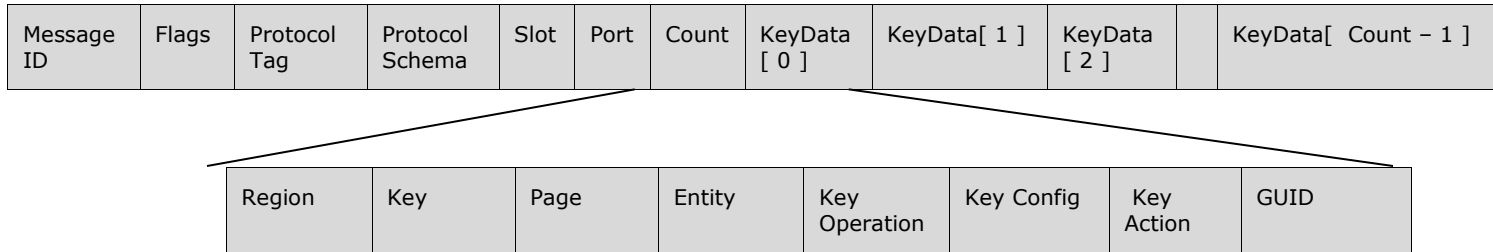
Reply Fixed Conference Assignments

4.55 Set Config for Multiple Keys

4.55.1 Description

Sets the config for multiple keys on a panel. This message provides an advanced level of key property control e.g. latch mode and activation type.

4.55.2 Structure



Field	Description
Message ID:	16 bit word 205, (0x00CD)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte. Schemas 1 and 2 currently supported. If schema 1 is used do not include the Endpoint Type in the header of the message (see below).
Slot:	Card Slot Number 1 byte
Port:	Port offset from first port of the card 1 byte
Endpoint Type:	2 bytes. Only include this field if using schema 2. The field is only used when the slot and offset fields above target a role. If this is not an assignment to a role set this field to 0. When using schema 2 and assigning keys to a role this field allows a particular endpoint type support by the role to be targeted e.g. setting this field to 0x8200 would assign the key to the FS II beltpack key layout associated with the role.

	If an assignment is made to a role and schema 1 is used the matrix will attempt to assign this key to the first endpoint associated with the role. When using schema 2, and a V-Series panel type is used in this field it will match the first V-Series panel type found against role if an exact match cannot be found.
Count:	Number of keys to assign to this panel. 1 byte
The remaining entries are sent for each key	
Region:	Region this key is on 1 byte
Key:	This keys Id 2 bytes
Page:	Page this key is on 1 byte
Entity:	e.g EN_PORT, EN_GROUP. The system number of the entity on the key must be set to a non-zero value. 16 bit word
Key Operation: 4 bytes	
Unpaged	0 paged, 1 unpaged Bit 0
TextMode	0 not displayed, displayed Bit 1
Dual	0 normal, 1 double width Bit 2
Dial	0 dial mode off, 1 dial mode on Bit 3
LatchMode	0 – LATCH_NON_LATCH 1 – LATCH 2 – NON_LATCH

	3 – RETURN_AFTER_HOLD 4 – TRI_ROT 5 – TRI_TOG 6 – RETURN_AFTER_LATCH 7 – DEL_LATCH 8 – HKS_LATCH Bits 4 – 7
Group	If non-zero, other keys on the same panel with the same group number will be deselected if this key is selected Bits 8 – 11
Deactivating	1 all interlock group keys may be off Bit 12
Make/Break	1 interlock group make before break Bit 13
Cross Page	1 keys interlocked across pages in same region Bit 14
Special 1	Bit 15 - Only used for IFB listen keys. 0 == Return listen, 1 == Destination listen. Otherwise set to 0.
Special 2	Bit 16 - Set to 0
Region Value	If the key is a page key, this value is the region it switches page on when pressed. Bit 17 – 19
Page Value	If the key is a page key, this value is the page it makes a switch to when pressed. Bit 20 – 23
Stacked group	If set indicates that the key is a stacked group Bit 24
Unused	Bit 25 – 31
Key Config: 26 bytes	
System Number	System Number 1 byte
Unused	Spare

	1 byte
Specific	Use depends on entity 16 Bit Word
Secondary	Dcc/DialCode(speed_dial) 16 Bit word
KeyAction: 16 Bit word	
Force Listen	0 disable, 1 enable Bit 0
Talk	0 disable, 1 enable Bit 1
Listen	0 disable, 1 enable Bit 2
Hold to Talk	0 disable, 1 enable Bit 3
Initial State	0 not selected, 1 selected Bit 4
Assign Locally	0 disable, 1 enable Bit 5
Assign Remotely	0 disable, 1 enable Bit 6
Locally Assigned	0 un-assigned, 1 assigned Bit 7
Radio	0 = FREQ 1 = TXSEL 2 = RXSEL 3 = TXM/S 4 = RXM/S Bits 8 – 10
Reserved	Unused bits

	Bits 11 – 15
GUID	Unique ECS GUID Number 16 Bytes

4.55.3 Usage

Used to quickly set 1 to n keys config for selected panel. Note however that the config is then stored in NVRAM and not with the map in FLASH.

4.55.4 Scope

CSU to Host only.

4.55.5 Special notes

Keys can be deleted using this message by setting the entity type field to EN_NULL, that is 0.

4.55.6 Performance

4.55.7 Example

4.55.8 Related messages

Reply Set Config For Multiple Keys

4.55.9 See also

4.56 Reply Set Config for Multiple Keys

4.56.1 Description

Replies to the Set Config for Multiple Keys message for selected panel.

4.56.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema	Slot	Port
------------	-------	--------------	-----------------	------	------

Field	Description
Message ID:	16 bit word 206 (0x00CE).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte. Schemas 1 and 2 currently supported.
Slot:	1 byte, Card Slot Number
Port:	1 byte, Port offset from first port of the card
Endpoint Type:	2 bytes. Only include this field if using schema 2. The field is only used when the slot and offset fields above target a role. If this is not an assignment to a role this field is set to 0. When using schema 2 and assigning keys to a role this field allows a particular endpoint type support by the role to be targeted e.g. setting this field to 0x8200 would assign the key to the FS II beltpack key layout associated with the role. The value in the reply is the value received in the request.
Keys:	2 bytes, Number of keys processed

4.56.3 Usage

Responds to a Set config for Multiple Keys.

4.56.4 Scope

Host to CSU only.

4.56.5 Performance

4.56.6 Example

Request update for keys on panel Slot 3 (1st audio slot) Port Offset 1
00 CE 08 AB BA CE DE 01 01 01

4.56.7 Related messages

4.56.8 See also

4.57 Request Forced Listen Edits

4.57.1 Description

Request all forced listen cross points currently made

Message ID	Flags	Protocol Tag	Protocol Schema
------------	-------	--------------	-----------------

4.57.2 Structure

Field	Description
Message ID:	16 bit word 201 (0x00C9)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.

4.57.3 Usage

4.57.4 Scope

4.57.5 Performance

4.57.6 Example

Request all forced listens in matrix.

00 C9 30 AB BA CE DE 01

4.57.7 Related messages

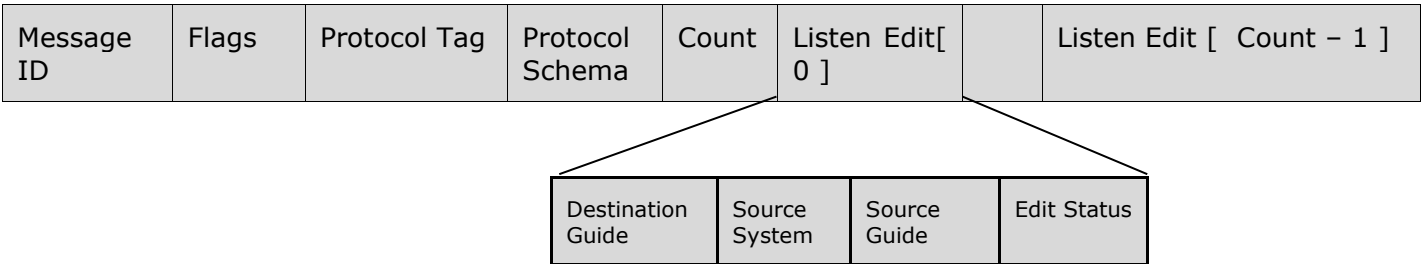
4.57.8 See also

4.58 Reply Forced Listen Edits

4.58.1 Description

Returns information on all locally made forced listens in the matrix

4.58.2 Structure



Field	Description
Message ID:	16 bit word 202 (0x00CA)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	Count of number of edits returned 16 bit word
Entry:	
Source port	Port number of source 2 bytes
Source system	System Number of source. 1 byte
Destination port	Port number of destination 2 bytes

Destination system	System number of destination. 1 byte
Edit Status	ADDED = 1, DELETED = 0 1 byte

4.58.3 Usage

4.58.4 Scope

4.58.5 Performance

4.58.6 Example

4.58.7 Related messages

Request Forced Listen Edits.

4.58.8 See also

4.59 Request Forced Listen Actions

4.59.1 Description

This message allows the host to set a forced listen crosspoint in the CSU (matrix)

4.59.2 Structure

Message ID:	16 bit word 225 (0x00E1)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16-bit Word, number of forced listen assignments in message
Forced Listen Action:	
Unused:	1 byte, Set to 0
Source system Number:	1 byte, The system number of the source port
Source port:	16-bit Word, source port
Destination port:	16-bit Word, destination port
Edit Action:	1 Byte, 1= Add Forced listen 0= Remove Forced listen

4.59.3 Usage

Sets multiple forced listens routes in the matrix. These are different to standard crosspoints as they are set at high priority level and they appear in the local advanced forced listen screen in EHX.

4.59.4 Scope

Host to CSU only.

4.59.5 Performance

4.59.6 Example

Request Force listen from system 2 port 3 to port 1 (0 indexed numbering)

00 E1 08 AB BA CE DE 01 00 01 00 02 00 03 00 01 01

4.59.7 Usage

4.59.8 Related messages

4.59.9 See also

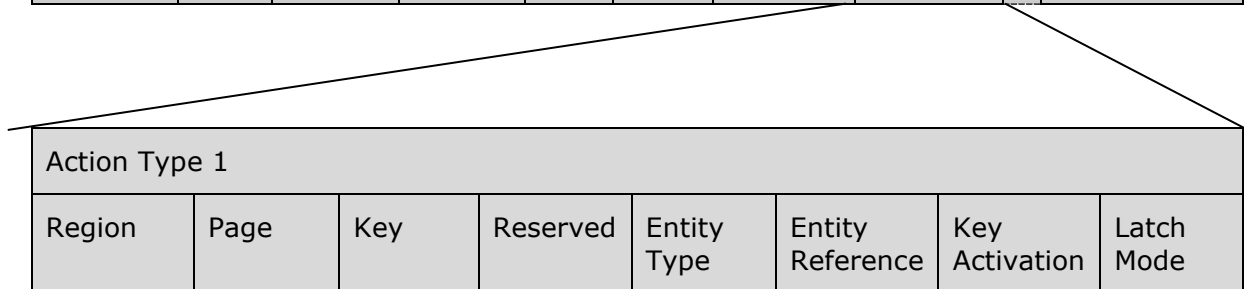
4.60 Request Remote Key Actions

4.60.1 Description

This message is used to assign keys on the target panel to a new route with the selected activation.

4.60.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema	Type	Count	Port Number	Action Data[0]	Action Data[Count]
------------	-------	--------------	-----------------	------	-------	-------------	----------------	--------------------



Message ID:	16 bit word 235 (0x00EB)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Type:	1 byte: AssignmentType ActionType1 = 1 All other types reserved for future use.
Count:	16-bit Word, number of key assignments in message
Port Number:	16-bit Word, Port Number of target panel for key assignment
Action Type 1	
Region:	1 byte, region of panel where key to be assigned is located

Page:	1 byte, page of panel where key to be assigned is located
Key:	1 byte, key position number of key to be assigned
Reserved:	1 byte, set to 0
Entity Type:	2 bytes, EN_NULL = 0x0000 remove assignment from key EN_PORT = 0x0001 for add port assignment to key EN_CONF = 0x0002 to add a Conference to a key EN_GROUP = 0x0003 to add a Fixed Group to a key EN_IFB = 0x0004 to add an IFB to a key
Entity Reference:	4 bytes, data defining entity of given type for assignment on the key For ports (EntityType = EN_PORT) Byte 1 = System number of entity for assignment on the. Byte 2 = reserved, set to 0 16 Bit Word = PortNumber (0 to 1023) For Conferences (EntityType = EN_CONF) Byte 1 = System number of entity for assignment on the. Byte 2 = reserved, set to 0 1 Byte = Conf Number (1to199) 1 Byte = 0; For HX Conf Number = 16 bit Word For Fixed Groups (EntityType = EN_GROUP) Byte 1 = System number of entity for assignment on the. Byte 2 = reserved, set to 0 16 Bit Word = Group Number (0 to 100) For IFBs (EntityType = EN_IFB) Byte 1 = System number of entity for assignment on the.

This message is used to delete the same key.

00 EB 08 AB BA CE DE 01 01 00 01 00 05 01 00 01 00 00 00 01 00 00 02 03

4.60.7 Special notes

Assignments which were successful will returned in a Reply Remote Key Actions message.

Assignments can also be retrieved using Request Remote Key Action Status

4.60.8 Related messages

Request Remote Key Action Status, Reply Remote Key Action Status.

4.60.9 See also

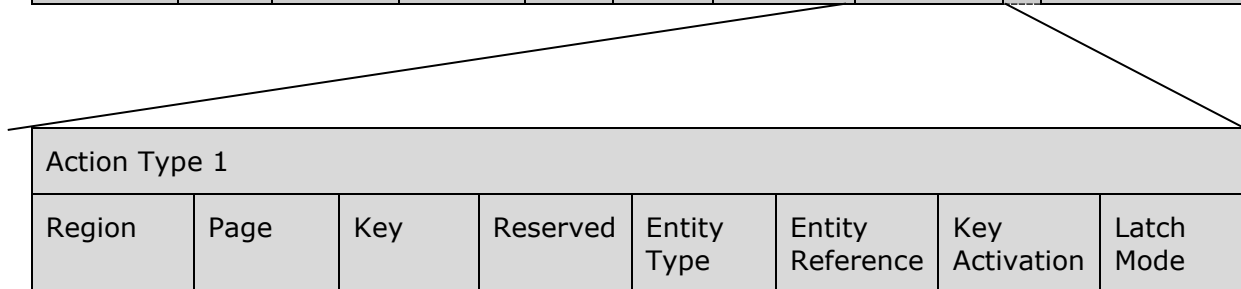
4.61 Reply Remote Key Actions

4.61.1 Description

This message shows all keys that have been assigned remotely on the target panel

4.61.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema	Type	Count	Port Number	Action Data[0]	Action Data[Count]
------------	-------	--------------	-----------------	------	-------	-------------	----------------	--------------------



Message ID:	16 bit word 236 (0x00EC)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Type:	1 byte: AssignmentType ActionType1 = 1 All other types reserved for future use.
Count:	16-bit Word, number of key assignments in message
Port Number:	16-bit Word, Port Number of target panel for key assignment
Action Type 1	
Region:	1 byte, region of panel where key to be assigned is located

Page:	1 byte, page of panel where key to be assigned is located
Key:	1 byte, key position number of key to be assigned
Reserved:	1 byte, set to 0
Entity Type:	2 bytes, NULL = 0x0000 remove assignment from key PORT = 0x0001 for add port assignment to key CONF = 0x0002 for add of conf etc... GROUP = 0x0003 IFB = 0x0004 GPIO..... = 0x0005 CONTROL.. = 0x0006
Entity Reference:	4 bytes, data defining entity of given type for assignment on the key Byte 1 = System number of entity for assignment on the. Byte 2 = reserved, set to 0 16 Bit Word = Specific reference of type e.g. port number
Key Activation:	1 Byte, 1 = TalkOnly, 2 = ListenOnly 3 = TalkAndListen 4 = Dual Talk And Listen. 5 = Talk And Forced Listen.
Latch Mode:	1 Byte, Not currently supported, set to 0

4.61.3 Usage

Used in reply to a Remote key action request

4.61.4 Scope

CSU to host only

4.61.5 Performance

4.61.6 Example

This message was sent by the CSU in response to a key being setup on a panel connected on EHX Port 0, region 1, page 0, key number 1 with TalkListen action to EHX Port 2.

```
00 EC 10 AB BA CE DE 01 01 00 01 00 00 01 00 01 00 00 01 01 00 00 02 03 00
```

4.61.7 Special notes

To determine if key assignments were actions a Request Remote Key Action Status can be sent to the CSU for a specific panel that the assignment was targeted to.

4.61.8 Related messages

Request Remote Key Action Status, Reply Remote Key Action Status.

4.61.9 See also

4.62 Request Remote Key Action Status

4.62.1 Description

This message causes the CSU to generate a message containing all of the key assignment actions, the message allows all remote assignments to be retrieved for all panels and alternatively allows the assignments for a specific panel to be retrieved.

4.62.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema	Type	Port Number
------------	-------	--------------	-----------------	------	-------------

Message ID:	16 bit word 237 (0x00ED)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte. Schemas 1 and 2 supported. If schema 2 is supported the endpoint type associated with the target role port is included in the reply.
Type:	1 byte, ActionType1 = 1 Other types reserved for future use
Port Number:	16 bit Word: Targets response to the specified panel. Only supports a request for a single panel at a time.

4.62.3 Usage

4.62.4 Scope

Host to CSU only.

4.62.5 Performance

4.62.6 Example

This message was sent by the host to get a list of all remote key assignments for the panel on port 4.

00 ED 08 AB BA CE DE 01 01 00 04

4.62.7 Special notes

None.

4.62.8 Related messages

Request Remote Key Actions, Reply Remote Key Action Status.

4.62.9 See also

4.63 Reply Remote Key Action Status

4.63.1 Description

This message will be sent to the host whenever a new key assignment is made, or in response to a Request Remote Key Action Status message.

4.63.2 Structure

Message ID:	16 bit word 238 (0x00EE)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 2.
Type:	1 byte: AssignmentType ActionType1 = 1 All other types reserved for future use.
Count:	16-bit Word, number of key assignments in message
Port Number:	16-bit Word, Port Number of target panel for key assignment
Endpoint Type:	16 bits. This is the endpoint type associated with the port. Note, this is only set to a non-zero value when the port is a role. This therefore indicates which endpoint the role is currently logged into.
Action Type 1	
Region:	1 byte, region of panel where key to be assigned is located
Page:	1 byte, page of panel where key to be assigned is located
Key:	1 byte, key position number of key to be assigned
Reserved:	1 byte, set to 0
Entity Type:	2 bytes, NULL = 0x0000 remove assignment from key

	PORT = 0x0001 for add port assignment to key CONF = 0x0002 for add of conf etc... GROUP = 0x0003 IFB = 0x0004 GPIO = 0x0005 CONTROL = 0x0006
Entity Reference:	4 bytes, data defining entity of given type for assignment on the key Byte 1 = System number of entity for assignment on the. Byte 2 = reserved, set to 0 16 Bit Word = Specific reference of type e.g. port number
Key Activation:	1 Byte, 1 = TalkOnly, 2 = ListenOnly 3 = TalkAndListen 4 = Dual Talk And Listen. 5 = Talk And Forced Listen.
Latch Mode:	1 Byte, Not currently supported, set to 0

4.63.3 Usage

The Count field contains the number of Status Data fields in the message.

The Entity type field indicates the type of assignment of the key, if the field is set to EN_NULL then the action has been deleted. This will only occur as assignments are being changed.

Note: In response to a Request Remote Key Action Status message, this message will only report assignments which are currently present, keys that are deleted will not be reported.

4.63.4 Scope

CSU to host only.

4.63.5 Performance

4.63.6 Example

DualTalkListen assignment to EHX Port 3 on panel connected to EHX Port 1 in Region 2 on Key 01 Page 0

00 EE 80 AB BA CE DE 01 01 00 01 00 00 02 00 01 00 00 01 01 00 00 02 04 00

4.63.7 Related messages

Request Remote Key Actions, Request Remote Key Action Status.

4.63.8 See also

4.64 Request VoIP Status

4.64.1 Description

This message is used by the host to request VoIP Status for all IVC32 channels on all IVC32 and E-IPA cards.

On the IVC32/E-IPA card a channel equates to a port.

Supported VoIP ports are IVP Panels, Agent-IC, LQ, Trunks, directs and hosted panel directs.

4.64.2 Scope

Host to CSU

4.64.3 Structure

Message ID:	16 bit word, 248 (0x00F8)
Flags:	1 byte, see FLAGS on page 59.
Protocol tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Sub Message ID:	Set to 0x16

4.64.4 Usage

4.64.5 Scope

4.64.6 Performance

4.64.7 Example

00 F8 30 AB BA CE DE 01 16

4.64.8 Related messages

Reply VOIP Status

4.64.9 See also

4.65 Reply VOIP Status

4.65.1 Description

This message is used in response to a Request VOIP Status. It can also be sent as an un-solicited message when VoIP status' change.

4.65.2 Structure

Message ID:	2 bytes, 248 (0x00F8)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Sub Message ID:	Set to 0x17
Number of Status':	2 bytes. The number of VoIP status' within this message.
The following fields are repeated for each VoIP status.	
Slot Number:	1 byte. Slot number of the IVC32/E-IPA reporting the start with first CPU slot being 'Slot 1'.
Channel:	1 byte. The VoIP channel i.e. port offset into card.
Port Number:	2 bytes. The absolute physical port number assigned in the EHX map for this channel.
Status:	1 byte. The new status of the VoIP channel; 0 = Unknown, 1 = Up, 2 = Down
Down Reason:	1 byte. See Status Enums below, set to 27 when channel status is NOT down.
Blocked Flag:	1 byte. Indicates whether the call establishment has been blocked due to repeated remote refusals of call request; 0 = True, 1 = False

4.65.3 Additional Information

4.65.4 Status Enums

```
CALL_END_REASON_BY_LOCAL_USR = 0,  
CALL_END_REASON_NO_ACCEPT = 1,  
CALL_END_REASON_ANSWER_DENIED = 2,  
CALL_END_REASON_REMOTE_USER = 3,  
CALL_END_REASON_REFUSAL = 4,  
CALL_END_REASON_NO_ANSWER = 5,  
CALL_END_REASON_CALLER_ABORT = 6,  
CALL_END_REASON_TRANSPORT_FAIL = 7,  
CALL_END_REASON_CONNECT_FAIL = 8,  
CALL_END_REASON_GATEKEEPER = 9,  
CALL_END_REASON_NO_USER = 10,  
CALL_END_REASON_NO_BANDWIDTH = 11,  
CALL_END_REASON_CAPABILITY_EXCHANGE = 12,  
CALL_END_REASON_CALL_FORWARDED = 13,  
CALL_END_REASON_SECURITY_DENIAL = 14,  
CALL_END_REASON_LOCAL_BUSY = 15,  
CALL_END_REASON_LOCAL_CONGESTION = 16,  
CALL_END_REASON_REMOTE_BUSY = 17,  
CALL_END_REASON_REMOTE_CONGESTION = 18,  
CALL_END_REASON_UNREACHABLE = 19,  
CALL_END_REASON_NO_END_POINT = 20,  
CALL_END_REASON_REMOTE_HOST_OFFLINE = 21,  
CALL_END_REASON_TEMP_FAILURE = 22,  
CALL_END_REASON_Q931_CAUSE = 23,  
CALL_END_REASON_DURATION_LIMIT = 24,  
CALL_END_REASON_INVALID_CONFERENCE_ID = 25,  
CALL_END_REASON_CONN_TIMEOUT = 26,  
CALL_END_REASON_NOT_SET = 27
```

4.65.5 Usage

4.65.6 Scope

CSU to Host

4.65.7 Performance

4.65.8 Example

Port 64, the 2nd port on the IVC32 card in slot 3 is reporting down with an error of TRANSPORT FAIL.

00 F8 30 AB BA CE DE 01 17 00 01 03 01 40 02 07 00

4.65.9 Related messages

Request VOIP Status

4.65.10 See also

4.66 Request to Change Telephone Hook State

4.66.1 Description

This message is used to toggle the on / off (line release) hook of a Clear-Com telephone hybrid (like TEL-14 or LQ-SIP). It includes killing all talks to the telephone hybrid and putting the telephone hybrid either on hook or off hook.

4.66.2 Structure

Message ID:	2 bytes, 269, 0x0010D
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Telephone port number:	2 bytes, 0 to 495
Hook State:	1 byte, 0 = on hook, remote telephone line release; 1 = off hook

4.66.3 Usage

4.66.4 Scope

Host to CSU.

4.66.5 Performance

4.66.6 Example

Remote line release a telephone hybrid on port 32

- Putting the telephone hybrid ON hook

00 01 0D 08 AB BA CE DE 01 00 1F 00

To take a telephone OFF hook on Port 32:

00 01 0D 08 AB BA CE DE 01 00 1F 01

4.66.7 Related messages

4.66.8 See also

4.66.9 Additional Information

4.67 Request Set System Time

4.67.1 Description

A 32 bit time_t UTC statement the current system time is the payload to this message. This system time will be applied to the config card.

Only one of these messages will be accepted per config card boot. This limitation is applied to stop multiple HCI clients on different PCs repeatedly setting the system time to different values.

There is no reply sent to this message.

4.67.2 Structure

Message ID:	2 bytes, 68 (0x0044)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
UTC (time_t) system time:	4 bytes, 32bit time_t representation of current system time.

4.67.3 Usage

4.67.4 Scope

Host to CSU

4.67.5 Performance

4.67.6 Example

Setting the current time to ~~0000-1st of Jan 1970~~.

00 44 30 AB BA CE DE 01 00 00 00 00

4.67.7 Related messages

4.67.8 See also

4.67.9 Additional Information

This message will only be processed once every 24 hours by the matrix, please also not that EHX will send this message to the matrix if it is connected. The matrix will accept a new set time message when it passes 2am local time.

4.68 Request Peripheral Info

4.68.1 Description

This message is used to request software and hardware version information for FreeSpeak 2 Antenna, Splitters and Beltpacks and other as yet undefined peripherals

4.68.2 Structure

Message ID:	2 bytes, 247 (0x00F7)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Sub Message ID:	1 byte, set to 0x14
Slot ID:	1 byte. The slot of interest. For wireless devices set this field to 0xFF 4.4 Slot numbering scheme

4.68.3 Usage

4.68.4 Scope

Host to CSU

4.68.5 Performance

4.68.6 Example

Request the hardware and firmware information for all the beltpacks in the matrix.

00 F7 30 AB BA CE DE 01 FF

4.68.7 Related messages

Reply Peripheral Info

4.68.8 See also

4.69 Reply Peripheral Info

4.69.1 Description

This message is used in response to Request
Request Peripheral Inf and is never unsolicited.

4.69.2 Structure

Message ID:	2 bytes, 123 (0x007B)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Sub type:	1 byte, 0x15
Slot number:	1 byte. This is the slot number that was in the request
Count:	2 bytes. The number of repeatable fields, if applicable.
The following fields are repeated X times, dependant on above count field.	
Slot number:	2 bytes. Slot number of port. Note 0xFFFF in this field for entries that do not have a slot e.g. beltpack roles
Port number:	2 bytes. Port or role number
Panel type:	2 bytes
Panel operation:	1 bytes. Bit 0 == Online, Bit 1 == sub type, Bit 2..7 == type
Talk and listen label (Unicode):	20 bytes. 10 wchar_t characters

Talk and listen alias (Unicode):	20 bytes. 10 wchar_t characters
Number of keys:	1 byte
Answerback timeout:	1 byte
Number of expansion panels:	1 byte
Region number of first expansion panel:	1 byte
PMID:	4 bytes. Beltpack unique identifier
Number of versions strings in message:	1 byte
Major:	2 bytes
Minor:	2 bytes
Firmware ID:	2 bytes
Revision:	2 bytes
Version string:	20 bytes

4.69.3 Usage

4.69.4 Scope

CSU to Host

4.69.5 Performance

4.69.6 Related messages

Request Peripheral Info

4.69.7 Additional Information

In the panel operation field above the sub type bit is used to indicate that the type field is in use in the same byte.

4.69.8 See also

4.70 Request IFB Status

4.70.1 Description

This message is used by a ~~HCI client~~ host to request any of the attributes of the specified IFB.

If the message is sent to a matrix that doesn't own the IFB (meaning the IFB was created on another matrix), IFB Attribute Type must be set to IFB_POTENTIAL_CALLERS, otherwise no answer will be returned.

The IFB attribute information is returned to the requesting host in the IFB Reply Status message.

4.70.2 Structure

Message ID:	2 bytes, 61 (0x003D)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
IFB Attribute Type:	1 byte IFB_INT_LEVEL == 0 IFB_DIM_LEVEL == 1 IFB_PRIORITY == 2 IFB_ACTIVE_CALLERS == 3 IFB_SOURCES == 4 IFB_DESTINATION == 5 IFB_RETURNS == 6 IFB_POTENTIAL_CALLERS == 7 IFB_TYPES_ALL == 255 If IFB_TYPES_ALL is requested then each set of attributes for the specified IFB is output.
Matrix Identifier:	1 byte. Matrix identifier.

IFB Identifier:	2 bytes. Unique IFB identifier within the context of this frame i.e. IFB identifiers are unique addressable across a linked by prefixing the matrix identifier. For example, if this request was for IFB 23 on matrix 5 this field would be set to 23 and the message would be sent to system 5.
-----------------	--

4.70.3 Additional Information

If the IFB_TYPE_ALL attribute type is requested, then the reply message will contain each attribute of the IFB.

The matrix identifier field is typically set to zero with the request being sent to the matrix that is the owner of the IFB. One exception however is the potential callers attribute request. This effectively searches for keys to the IFB and this message can be sent to matrices that are not the resident matrix of the IFB.

4.70.4 Usage

4.70.5 Scope

Host to CSU

4.70.6 Performance

4.70.7 Example

Request all attributes of IFB 1.4.8

00 3D 30 AB BA CE DE 01 FF 01 08

4.70.8 Related messages

Reply IFB Status

4.70.9 See also

4.71 Reply IFB Status

4.71.1 Description

This message is used to carry the response to the IFB Request Status message.

4.71.2 Structure

Message ID:	2 bytes, 62 (0x003E)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	2 bytes. The number of repeatable fields, if applicable.
The following fields are repeated X times, dependant on above Count field.	
IFB Attribute Type:	1 byte IFB_INT_LEVEL == 0 IFB_DIM_LEVEL == 1 IFB_PRIORITY == 2 IFB_ACTIVE_CALLERS == 3 IFB_SOURCES == 4 IFB_DESTINATION == 5 IFB_RETURNS == 6 IFB_POTENTIAL_CALLERS == 7 IFB_RETURN_LISTENS == 8 IFB_DESTINATION_LISTENS == 9
Matrix Identifier:	1 byte. The identifier of the matrix the IFB belongs to.

IFB Identifier:	2 bytes. Unique IFB identifier within the context of this frame i.e. IFB identifiers are unique addressable across a linked by prefixing the matrix identifier.
Is absolute state:	1 byte. Indicates whether the values for this attribute represent the complete known state of an IFB.
Element count:	2 bytes. Number of elements in this property type list e.g. number of sources, number of active callers etc.
The following fields are repeated X times, dependant on above Element count field.	
Entry Action:	1 byte 0 = DELETED, 1 = ADDED, 2 = ADDED_PENDING, 3 = PRESENT/NO CHANGE, 4 = EDITED. The message is sent in response to add or delete request. It is also sent as an unsolicited message when the attribute is edited etc.

Value:	1, 4 or 5 bytes Value field length is different depending on the attribute type. Length and value field detail shown below. IFB_INT_LEVEL : 1 byte (Non-Zero means the IFB has been triggered i.e. being dimmed) IFB_DIM_LEVEL : 1 byte – These are as follows: 0 = 0db, 1 = -3db, 2 = -6db, 3 = -9db, 4 = -12db, 5 = -15db, 6 = -18db, 7 = -21db, 8 = -24db, 9 = -27db, 10 = -30db, 15 = -72db (i.e. off) IFB_PRIORITY : 5 bytes– (Dial code : 4, Priority : 5) IFB_ACTIVE_CALLERS : 4 bytes – (Dial code : 4) IFB_SOURCES : 4 bytes – (Dial code : 4) IFB_DESTINATION : 4 bytes – (Dial code : 4) IFB_RETURNS : 4 bytes – (Dial code : 4) IFB_POTENTIAL_CALLERS : 4 bytes – (Dial code : 4)
--------	--

4.71.3 Additional Information

The element count will vary depending on the attribute type being requested. The number of callers, sources and returns returned will vary depending on the number in place.

There will only ever be one value each for interrupt level, dim level and priority.

IFB_POTENTIAL_CALLERS will not include panels with unassigned key group keys. These panels will only become potential callers when the key group key has been assigned.

4.71.4 Usage

4.71.5 Scope

Matrix to Client

4.71.6 Performance

4.71.7 Example

4.71.8 Related messages

Request IFB Status, Request IFB Set

4.71.9 See also

4.72 Request IFB Set

4.72.1 Description

This message is used to request an IFB property edit, add or delete. The Matrix will respond with an IFB Reply Status containing only the attributes that have been deleted, added or edited.

4.72.2 Structure

Message ID:	2 bytes, 63 (0x003F)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Number of X i.e. count:	2 bytes. The number of repeatable fields, if applicable.
The following fields are repeated X times, dependant on above count field.	

IFB Attribute Type:	1 byte IFB_INT_LEVEL = 0 IFB_DIM_LEVEL = 1 IFB_PRIORITY = 2 IFB_ACTIVE_CALLERS = 3 IFB_SOURCES = 4 IFB_DESTINATION = 5 IFB_RETURNS = 6 IFB_POTENTIAL_CALLERS = 7 IFB_RETURN_LISTENS = 8 IFB_DESTINATION_LISTENS = 9 IFB_TYPES_ALL = 255 If IFB_TYPES_ALL is requested then each set of attributes for the specified IFB is output.
IFB Instance:	2 bytes. Unique IFB identifier within the context of this frame i.e. IFB identifiers are unique addressable across a linked by prefixing the matrix identifier.
Element count:	2 bytes. Number of element in this property type list e.g. number of sources, number of active callers etc.
The following fields are repeated N times, dependant on above Element count field.	
Entry Action:	1 byte 0 = DELETED, 1 = ADDED, 2 = ADDED_PENDING, 3 = PRESENT/NO CHANGE, 4 = EDITED.

Value:	1, 4 or 5 bytes Value field length is different depending on the attribute type. Length and value field detail shown below. IFB_INT_LEVEL : 1 byte (Non-Zero means the IFB has been triggered i.e. being dimmed) IFB_DIM_LEVEL : 1 byte – These are as follows: 0 = 0db, 1 = -3db, 2 = -6db, 3 = -9db, 4 = -12db, 5 = -15db, 6 = -18db, 7 = -21db, 8 = -24db, 9 = -27db, 10 = -30db, 15 = -72db (i.e. off), IFB_PRIORITY : 5 bytes – (Dial code : 4, Priority : 5) IFB_ACTIVE_CALLERS : 4 bytes – (Dial code : 4) IFB_SOURCES : 4 bytes – (Dial code : 4) IFB_DESTINATION : 4 bytes – (Dial code : 4) IFB_RETURNS : 4 bytes – (Dial code : 4) IFB_POTENTIAL_CALLERS : 4 bytes – (Dial code : 4)
--------	--

4.72.3 Additional Information

The element count will vary depending on the attribute type being requested. The number of callers, sources and returns returned will vary depending on the number in place.

There will only ever be one value each for interrupt level, dim level and priority.

4.72.4 Usage

4.72.5 Scope

Client to Matrix

4.72.6 Performance

4.72.7 Example

4.72.8 Related messages

Reply IFB Status

4.72.9 See also

4.73 Request Entity Info

This message is used to request entity instance information from the target matrix frame

4.73.1 Description

The information returned indicates the existence of an entity and its label in UTF16 Unicode format.

A specified entity type can be requested or instead all supported entities types can be returned.

The entity type currently supported by this message are;

- CONFERENCES
- GROUPS
- IFBS

Note – Port instance information should be requested from the matrix using the message Request Port Info (ID 183).

4.73.2 Structure

Message ID:	2 bytes, 175 (0x00AF)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Entity type:	1 byte. 0 == All (i.e. Conferences, Groups and IFBs), 2 == Conferences, 3 == Groups, 4 == IFBs

4.73.3 Usage

4.73.4 Scope

Host to CSU

4.73.5 Performance

4.73.6 Example

Request all virtual IFBs in the current matrix

00 B0 30 AB BA CE DE 01 04

4.73.7 Related messages

Reply Entity Info

4.73.8 See also

4.74 Reply Entity Info

This message is used to reply to the request entity ~~instance~~ information message (ID-175). It delivers (over multiple messages if required) the requested entity instance information for the supported entity types in the current matrix frame configuration.

4.74.1 Description

Multiple replies may be sent to a single request depending on the number of entities that need to be reported on.

The client can request a single entity type or 'all'. All means all conferences, groups and IFBs.

4.74.2 Structure

Message ID:	2 bytes, 176 (0x00B0)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Request Entity Type:	1 byte 0 == All, 2 == Conferences Only, 3 == Groups Only, 4 == IFBs Only
Number of instances in this message: count	2 bytes. Entries in this message. If more messages are to follow the continuation bit will be set in the flags (above) of this message and these number of instances of entities to follow will not be included in this count.
The following fields are repeated X times, dependant on above count field.	
Dial-code of entity instance:	4 bytes total. System Number (8 bits) Entity type (8 bits) Instance of entity (16 bits)

Name Length:	2 bytes. Unicode string buffer length – States number of bytes of Unicode string to follow.
Name:	Variable. Number of bytes as stated in the preceding field.

4.74.3 Additional Information

Only user level conferences and groups will be reported in the entity info reply. For example, the internal group and conference elements created to support route monitors, stacked keys, PL turnarounds etc. will not be reported.

4.74.4 Usage

4.74.5 Scope

CSU to Host

4.74.6 Performance

4.74.7 Example

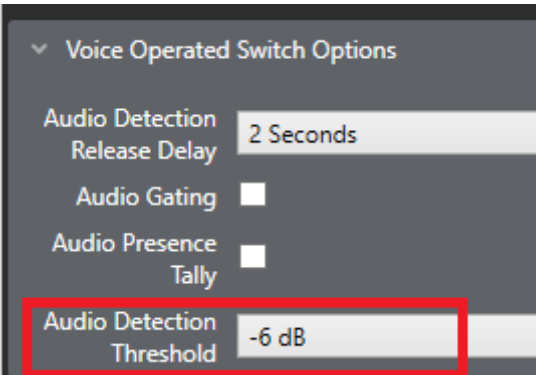
4.74.8 Related messages

Request Entity Info

4.74.9 See also

4.75 Request VOX Threshold Status

This message is used to request the current non default VOX threshold status set in the current matrix frame.



4.75.1 Description

The default VOX threshold value is 20. Any port with a non-default threshold level will be reported on in the resulting Reply Vox Threshold Status msg.

4.75.2 Structure

Message ID:	2 bytes, 74 (0x004A)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.

4.75.3 Usage

4.75.4 Scope

Host to CSU

4.75.5 Performance

4.75.6 Example

00 4A 30 AB BA CE DE 01

4.75.7 Related messages

Reply Vox Threshold Status

4.75.8 See also

4.76 Reply Vox Threshold Status

This message is used to either acknowledge a VOX level set or to reply to a Request VOX status message.

4.76.1 Description

If this message is sent in response to a Request VOX Threshold Set then the contents will be the entries that received in the request and have been successfully handled.

If this message is sent in response to a Request VOX Threshold Status then the message will contain all port threshold levels in the matrix frame where the threshold is set to a non-default value.

The default VOX threshold value is 20.

4.76.2 Structure

Message ID:	2 bytes, 74 (0x004A)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count-	2 bytes. Entries in this message.
The following fields are repeated X times, dependant on above count field.	
Port:	2 bytes. Zero based port number to receive the update
Level:	2 bytes. New VOX threshold level (value between 0 and 40)

4.76.3 Additional Information

Level parameter to dB mapping as follows.

0 == -40 dB

1 == -39 dB

2 == -38 dB

...

40 == 0 dB

4.76.4 Usage

4.76.5 Scope

CSU to Host

4.76.6 Performance

4.76.7 Example

4.76.8 Related messages

Request Vox Threshold Status

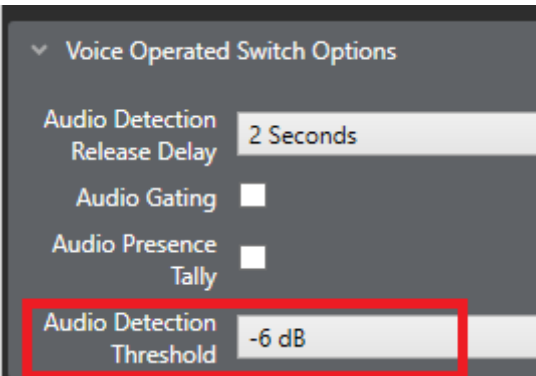
4.76.9 See also

4.77 Request Vox Threshold set

The message may contain s 1 or more port VOX threshold level updates.

4.77.1 Description

This feature will not enable audio gating or audio presence tally. It will only update the currently active VOX threshold level.



4.77.2 Structure

Message ID:	2 bytes, 72 (0x0048)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Number of instances in this message: Count	2 bytes. Entries in this message.
The following fields are repeated X times, dependant on above count field.	
Port:	2 bytes. Zero based port number to receive the update
Level:	2 bytes. New VOX threshold level (value between 0 and 40)

4.77.3 Additional Information

Level parameter to dB mapping as follows.

0 == -40 dB

1 == -39 dB

2 == -38 dB

...

40 == 0 dB

4.77.4 Usage

4.77.5 Scope

CSU to Host

4.77.6 Performance

4.77.7 Example

Set port 1 Vox threshold to -1 dB

00 48 AB BA CE DE 01 00 01 00 01 00 01

4.77.8 Related messages

Reply Vox Threshold Set

4.77.9 See also

4.78 Beltpack Status Reply

This message is sent to the Host when a beltpack status changes.

4.78.1 Description

Changes to Online/Offline status, Antenna Port & Role number will cause this message to be sent.

4.78.2 Structure

Message ID:	2 bytes, 76 (0x004C)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Number of instances in this message: Count	2 bytes. Entries in this message.
The following fields are repeated X times, dependant on above count field.	
PMID	20 Bits. Beltpack unique identifier
Status	1 Bit. 0 = Offline, 1 = Online
Frequency Type	2 Bits. 0 = Not Set, 1 = 1.9Ghz, 2 = 2.4Ghz
Wireless Mode	1 Bit. 0 = FS1, 1 = FS2
Unused	8 bits.
Role Number	2 bytes. The role number in use by this beltpack.
Antenna Port	2 bytes. The Port number of the Antenna the Beltpack is connected to.

4.78.3 Usage

4.78.4 Scope

CSU to Host

4.78.5 Performance

4.78.6 Example

4.78.7 Related messages

4.78.8 See also

4.79 Reply Panel Status

This message is sent to the Host when a panel status changes.

4.79.1 Description

Changes to Online/Offline status will cause this message to be sent.

4.79.2 Structure

Message ID:	2 bytes, 76 (0x004C)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Number of instances in this message: Count-	2 bytes. Entries in this message.
The following fields are repeated X times, dependant on above count field.	
Panel Num LSB:	1 byte. The least significant 8 bits of the panel number (0 index)
Panel Type:	1 byte. See available panel types below.
Panel Condition:	1 byte. See available panel conditions below.
Panel Num MSB:	1 byte. The most significant 8 bits of the panel number (0 index)

4.79.3 Additional Information

Panel Types:

Unknown = 0

Generic = 1

FS 1 Antenna 1.9Ghz = 2

FS 2 Antenna 1.9Ghz = 3

FS 2 Antenna 2.4Ghz = 4

Antenna Type Unknown = 5

Panel Condition:

Unknown = 0

Good = 1

Absent = 2

Faulty = 3

Disconnect = 4

4.79.4 Usage**4.79.5 Scope**

CSU to Host

4.79.6 Performance**4.79.7 Example****4.79.8 Related messages****4.79.9 See also**

4.80 Request SRecord Transmission Initiation

4.80.1 Description

This message is used to initiate an srecord transfer to a matrix.

The type of srecord (and therefore the subsequent handling in the matrix) is specified inside the srecord file transmitted.

Currently the matrix will accept a configuration (or map) and matrix firmware as follows.

If the srecord is a matrix configuration, then the parameters in the initiation message allow the following to be specified:

- Reset type – Red (Apply with reset), Black (Apply with reset and NV clear) or None (Apply changes)
- Partial NV Clear Options

4.80.2 Structure

Message ID:	2 bytes, 69 (0x0045)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Length	2 bytes. Payload length of this message (), that is length in bytes from the start of the length field.
Sub Message ID	1 byte. 5.
Apply Type	1 byte. 1 – Apply with Reset (Red reset) 2 – Apply Changes (no reset) 9 – Apply with reset and NVRAM clear (Black reset)
Reserved	9 bytes. Reserved for future use.

Partial NV Clear Options	16 bits Bit field, OR the required options as follows 15 – Clear locally assigned keys 14 – Clear locally assigned (online) forced listens 13 – Clear locally made level changes 12 – Clear locally made fixed group changes 11 – Clear locally made conference changes 10 – Clear locally held over the air registration (FS II) 9 .. 0 - Unused
--------------------------	---

4.80.3 Additional information

4.80.4 Usage

4.80.5 Scope

Client to Matrix.

4.80.6 Performance

4.80.7 Example

4.80.8 Related messages

4.80.9 See also

4.81 SRecord Transmission

4.81.1 Description

This message is used to send a single srecord to the matrix. (This protocol reference document will not describe the contents of the Motorola srecord format).

EHX supports (from version 8.8) the export of a matrix downloadable configuration in the form of an SRecord.

Each line of the export file is a separate srecord. Once the download session has been initiated using the SRecord Transmission Initiation Request (specified in the last section), this srecord format file simply needs to be sent down to the matrix one srecord (line) at a time. When the final srecord has been received by the matrix it will automatically process the download as directed in the initiation message.

Note: The Sub ID 83 field is included in the srecord (83 being the ascii code for 'S').

4.81.2 Structure

Message ID:	2 bytes, 69 (0x0045)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Length	2 bytes. Payload length of this message (), that is the length in bytes from the start of the length field.
Srecord	2 bytes. One srecord.

4.81.3 Additional information

4.81.4 Usage

4.81.5 Scope

CSU to Host .

4.81.6 Performance

4.81.7 Example

4.81.8 Related messages

4.81.9 See also

4.82 Request Rack Properties: Active Configuration Bank

4.82.1 Description

This message is sent into the matrix to request info on what is the currently active configuration bank.

4.82.2 Structure

Message ID:	2 bytes, 44 (0x002C)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Sub Msg ID	2 bytes. 4 == Config Bank Request.

4.82.3 Additional information

The matrix replies to the client using the Rack Properties Configuration Bank Reply.

4.82.4 Usage

4.82.5 Scope

Host to CSU .

4.82.6 Performance

4.82.7 Example

4.82.8 Related messages

4.82.9 See also

4.83 Reply Rack Properties: Active Configuration Bank

4.83.1 Description

This message is sent from the matrix to the Host in reply to a request currently active configuration bank

4.83.2 Structure

Message ID:	2 bytes, 44 (0x002C)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Sub Msg ID	2 bytes. 5 == Config Bank Request.
Success	1 byte. 1 – success.
Bank Number	1 byte. Bank number from 0 to 4. 0 indicates that the embedded default map is currently in use.

4.83.3 Additional information

4.83.4 Usage

4.83.5 Scope

CSU to Host .

4.83.6 Performance

4.83.7 Example

4.83.8 Related messages

4.83.9 See also

4.84 Request Rack Properties: Rack State Get

4.84.1 Description

This message is sent into the matrix to request the current rack running state. The type of state reported allows a connected Host to establish if a sent configuration has been accepted, and so on.

4.84.2 Structure

Message ID:	2 bytes, 44 (0x002C)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Sub Msg ID	2 bytes. 8 == Rack State Get Request.

4.84.3 Additional information

4.84.4 Usage

4.84.5 Scope

Client to Matrix.

4.84.6 Performance

4.84.7 Example

4.84.8 Related messages

4.84.9 See also

4.85 Reply Rack Properties: Rack State Get

4.85.1 Description

This message is sent to a Host in response to a Rack Properties Rack State Get Request.

4.85.2 Structure

Message ID:	2 bytes, 44 (0x002C)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Sub Msg ID	2 bytes. 9 == Rack State Get Reply.
Current State	2 bytes. 0 == UNKNOWN, 1 == READY, 2 == RESETTING, 3 == DOWNLOAD, 4 == DOWNLOAD COMPLETE, 5 == DOWNLOAD_FAILED
Current State Additional Info	2 bytes. 0 == NOT_SET, 1 == NID_NOT_POSSIBLE, 2 == HW_DIFFERENT, 3 == TRUNKS_DIFFERENT, 4 == CSUM_FAILED, 5 == TIMEOUT_FAIL, 6 == FORMAT_FAIL
Previous State	2 bytes. 0 == UNKNOWN, 1 == READY, 2 == RESETTING, 3 == DOWNLOAD, 4 == DOWNLOAD COMPLETE, 5 == DOWNLOAD_FAILED
Previous State Additional Info	2 bytes. 0 == NOT_SET, 1 == NID_NOT_POSSIBLE, 2 == HW_DIFFERENT, 3 == TRUNKS_DIFFERENT, 4 == CSUM_FAILED, 5 == TIMEOUT_FAIL, 6 == FORMAT_FAIL

4.85.3 Additional information

The current previous rack states are included in the reply. This allows the HCI client to differentiate between, for example, the case where a live apply changes was successfully performed, and one where the matrix reset during the apply changes.

4.85.4 Usage

4.85.5 Scope

CSU to Host

4.85.6 Performance

4.85.7 Example

4.85.8 Related messages

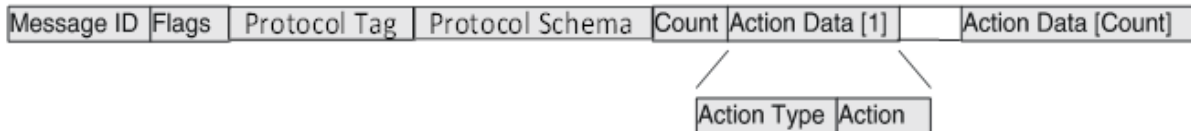
4.85.9 See also

4.86 Request Audio Monitor Actions

4.86.1 Description

This message is used by the Host to add or remove port monitors in the CSU. A port monitor is another port that receives the same audio mix as the monitored port. This feature would typically be used by a recording or supervision feature.

4.86.2 Structure



Message ID:	16 bit word, 17 (0x0011)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16 bit word
Action Data:	
Action Type:	16 bit word, set to 0x0100
Action:	4 x 16 bit words:
Word 0:	
bit 0	direction bit, 0 = delete, 1 = add
bits 1 - 15	set to 0
Word 1:	
bits 0 - 10	Monitor port number i.e. port to which xpts to target port are mirrored. (Zero based port numbering)
bits 11 - 15	Set to 0
Word 2:	

bits 0 - 10	Target port i.e. the number of the port being monitored. (zero based port numbering)
Bits 11 - 15	Set to 0
Word 3:	
bits 0 - 15	set to 0

4.86.3 Usage

This message allows multiple monitor and target pairs to be added /removed from the current matrix monitoring component. When any port audio monitor is in place, any source routes to the target port will be mirrored to the associated monitor port.

Each Request Audio Monitor Action will be acknowledged by the transmission of an Reply Audio Monitor Action.

The monitor port has to be on the same matrix as the target port.

All audio sources (local and remote) will be mirrored to the monitor port.

4.86.4 Scope

Host to CSU only.

4.86.5 Performance

4.86.5.1 Example

The following example is requesting a single monitor of the audio route out to port 0 to be mirrored out to port 1.

00 11 08 AB BA CE DE 01 00 01 01 00 00 01 00 01 00 00 00 00

4.86.5.2 Special notes

None.

4.86.5.3 Related messages

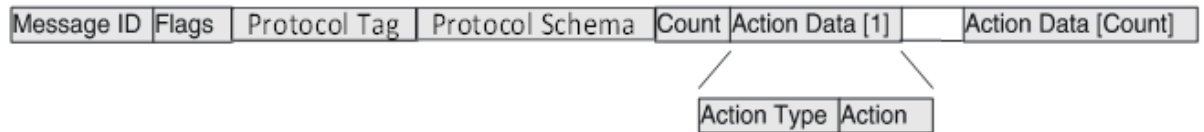
Reply Audio Monitor Action.

4.87 Reply Audio Monitor Actions

4.87.1 Description

This message is used in reply to a request an audio monitor

4.87.2 Structure



Message ID:	16 bit word, 16 (0x0010)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16 bit word
Client IP Address	4 bytes, the source IP address of the client issuing the HCI request
Action Data:	
Action Type:	16 bit word, set to 0x0100
Action:	4 x 16 bit words:
Word 0:	
bit 0	direction bit, 0 = delete, 1 = add
bits 1 - 15	set to 0
Word 1:	
bits 0 - 10	Monitor port number i.e. port to which xpts to target port are mirrored. (Zero based port numbering)
bits 11 - 15	Set to 0
Word 2:	

bits 0 - 10	Target port i.e. the number of the port being monitored. (zero based port numbering)
Bits 11 - 15	Set to 0
Word 3:	
bits 0 - 15	set to 0

4.87.3 Additional information

This message acknowledges the receipt of the Request Action Monitor Request.
The parameters of the message are those of the request received.

4.87.4 Scope

Host to CSU only.

4.87.5 Performance

4.87.6 Example

4.87.7 Related messages

Request Audio Monitor Action.

4.88 Request Telephony Client Get State

4.88.1 Description

This message is used to request the state of the specified, or all Telephony Client ports in the matrix. Telephony client ports supported include TEL-14 and SIP client types.

4.88.2 Structure

Message ID:	16 bit word, 285 (0x011D)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client
Count:	16 bit word
Port	2 bytes. Valid port number (0 to 495) to get the state for a specified port. Set to 0xFFFF (65535) to request all Telephony port states. This may involve multiple reply messages
Server external IP Address	4 bytes. IPV4 IP address of Server. Set to 0xFFFFFFFF to receive all client states. A monitoring application may use this option, whereas a Telephony Server would typically only request that state of clients that are configured to connect to it.

4.88.3 Additional information

Sent in response to the Telephony Client Get State Request, additionally will be sent unsolicited when there is a telephone client status update.

4.88.4 Scope

Host to CSU.

4.88.5 Performance

4.88.6 Example

4.88.7 Related messages

4.89 Reply Telephony Client Get State

4.89.1 Description

This message is used to reply to the Telephony Client Get State Request. It contains the Telephony client status for the specified port or all ports. Telephony client ports supported include TEL-14 and SIP client types.

4.89.2 Structure

Message ID:	16 bit word, 286 (0x011E)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	16 bit word
Number of X, that is Count	2 bytes. The number of entries as of the structure below
The following fields are repeated X times, dependant on above count field.	
Port	2 bytes. Valid port number (0 to 495) to get the state for a specified port. Set to 0xFFFF (65535) to request all Telephony port states. This may involve multiple reply messages
Status	1 byte. 0 == ON_HOOK, 1 == ON_HOOK_ALLOCATED, 2 == CONNECTING_OUT, 3 == CONNECTED_OUT, 4 == CONNECTING_IN, 5 == CONNECTED_IN

Call Info	32 bytes. ASCII string, Dial digits for outgoing calls, CLI information for incoming calls.
-----------	---

4.89.3 Additional information

Sent in response to the Telephony Client Get State Request, additionally will be sent unsolicited when there is a telephone client status update.

4.89.4 Scope

CSU to Host

4.89.5 Performance

4.89.6 Example

4.89.7 Related messages

4.90 Request Telephony Client Disconnect Incoming

4.90.1 Description

This message is used to disconnect a call that is currently connected. The call may have been an incoming or outgoing call.

4.90.2 Structure

Message ID:	16 bit word, 291 (0x0123)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Port:	2 bytes. Valid port number (0 to 495). This is the port number on the matrix that has an associated Telephony client on the Telephony server.
Reason	1 byte. 0 == REASON_NOT_SET 1 == TERMINATED_BY_USER 2 == TERMINATED_BY_SERVER

4.90.3 Additional information

The matrix acknowledges this message with the Telephony Client Disconnect Incoming Reply.

4.90.4 Scope

Host to CSU.

4.90.5 Performance

4.90.6 Example

4.91 Reply Telephony Client Disconnect Incoming

4.91.1 Description

This message is used to acknowledge the incoming disconnect request from the associated Telephony Server.

4.91.2 Structure

Message ID:	16 bit word, 291 (0x0123)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Port:	2 bytes. Valid port number (0 to 495). This is the port number on the matrix that has an associated Telephony client on the Telephony server.
Success	1 byte. 0 == false, 1 == true

4.91.3 Additional information

This message is sent in response to the Telephony Client Connect Incoming Request. The success field being true indicates that the incoming call request was valid.

4.91.4 Scope

CSU to Host.

4.91.5 Performance

4.91.6 Example

4.92 Request Telephony Client Disconnect Outgoing

4.92.1 Description

This message is used to request by the matrix to request the disconnection of a Telephony call that is currently connected via a third part SIP server switch.

4.92.2 Structure

Message ID:	16 bit word, 293 (0x0125)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Port:	2 bytes. Valid port number (0 to 495). This is the port number on the matrix that has an associated Telephony client on the Telephony server.
Reason	1 byte. 0 == REASON_NOT_SET 1 == TERMINATED_BY_USER 2 == TERMINATED_BY_MATRIX

4.92.3 Additional information

This message is used to disconnect both incoming and outgoing calls.

4.92.4 Scope

Matrix to SIP Server application.

4.92.5 Performance

Example

4.93 Reply Telephony Client Disconnect Outgoing

4.93.1 Description

This message is used to acknowledge the disconnection of the call associated with the specified port

4.93.2 Structure

Message ID:	16 bit word, 294 (0x0126)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Port:	2 bytes. Valid port number (0 to 495). This is the port number on the matrix that has an associated Telephony client on the Telephony server.
Success	1 byte. 0 == false, 1 == true

4.93.3 Additional information

This message is sent in response to the Telephony Client Disconnect Outgoing Request.

4.93.4 Scope

SIP server application to matrix.

4.93.5 Performance

4.93.6 Example

4.94 Request Telephony Client Dial Info Outgoing

4.94.1 Description

This message is used to send dial information from the Matrix to the specified SIP server application.

4.94.2 Structure

Message ID:	16 bit word, 295 (0x0127)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Port:	2 bytes. Valid port number (0 to 495). This is the port number on the matrix that has an associated Telephony client on the Telephony server.
Dial Buffer	32 bytes. ASCII dial buffer

4.94.3 Additional information

These digits may either be sent before or after the call has been connected.

Note: There is no reply to this message

4.94.4 Scope

Matrix to SIP server application.

4.94.5 Performance

4.95 Request Telephony Client Dial Info Incoming

4.95.1 Description

This message is used to send dial information from the SIP call server to the Matrix.

4.95.2 Structure

Message ID:	16 bit word, 296 (0x0128)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Port:	2 bytes. Valid port number (0 to 495). This is the port number on the matrix that has an associated Telephony client on the Telephony server.
Dial Buffer	32 bytes. ASCII dial buffer

4.95.3 Additional information

This message can be used for prepared dialling i.e. call initiation, or for in call progress digit passing e.g. to traverse an audible menu structure.

4.95.4 Scope

CSU to Host

4.95.5 Performance

4.96 Request Get Proxy Indication State

4.96.1 Description

This message is used to request the state of the proxy key LED indications for the specified panel, or all panels. These keys are 3rd party system proxy keys i.e. keys that are driven by a third party system via HCI.

Note: This data is stored in volatile matrix storage. If connection is lost with the matrix, the Host should request the display data for all keys driven by the client in case the matrix has been reset.

4.96.2 Structure

Message ID:	16 bit word, 310 (0x0136)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Panel Number:	2 bytes. All panels == 65535, otherwise a valid panel number

4.96.3 Additional information

The Get Proxy Indication State Reply is sent in response to this message.

If all panels are requested only those keys configured in EHX as third-party keys will be sent to Host, that is not all keys in the system.

4.96.4 Scope

Client to matrix.

4.96.5 Performance

4.97 Reply Get Proxy Indication State

4.97.1 Description

This message contains the current state of the proxy key indications of one or more panel in the local matrix. These keys are third-party system proxy keys, that is keys that are driven by a third-party system using HCI.

4.97.2 Structure

Message ID:	16 bit word, 311 (0x0137)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Number of Entries: Count_	2 bytes. Number of keys entries in this message.
The following fields are repeated X times, dependant on above count field.	
Port Number:	2 bytes, Panel port number
Region:	1 byte, Region number of key
Page:	1 byte, Page number of key
Key:	1 byte, Key number
Colour/Brightness:	1 byte, 0x1 == Bright, Red == 0x2, Green == 0x4, Blue == 0x8. Note, when bright is not set the LED brightness is dim. The off mode is set via the rate field below
Rate:	1 byte, 0 == Off, 1 == 1Hz, 2 == 2 Hz, 3 == 4Hz, 15 == always on.

Auto MIC on Enabled:	1 byte, 0 == Off, 1 == On
----------------------	---------------------------

4.97.3 Additional information

This message is sent in response to a Get Proxy Indication States Request.

4.97.4 Scope

CSU to Host.

4.97.5 Performance

4.98 Request Set Proxy Indication State

4.98.1 Description

This message sets the current state of the key indication of one or more panel key indications in the local matrix. These keys are 3rd party system proxy keys i.e. keys that are driven by a third party system via HCI.

Note: This data is stored in volatile matrix storage. If connection is lost with the matrix, the client should request the display data for all keys driven by the client in case the matrix has been reset.

4.98.2 Structure

Message ID:	16 bit word, 312 (0x0138)
Flags:	1 byte, see on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Number of Entries: Count	2 bytes. Number of keys entries in this message.
The following fields are repeated X times, dependant on above count field.	
Port Number:	2 bytes, Panel port number
Region:	1 byte, Region number of key
Page:	1 byte, Page number of key
Key:	1 byte, Key number
Colour/Brightness:	1 byte, 0x1 == Bright, Red == 0x2, Green == 0x4, Blue == 0x8. Note, when bright is not set the LED brightness is dim. The off mode is set via the rate field below

Rate:	1 byte, 0 == Off, 1 == 1Hz, 2 == 2 Hz, 3 == 4Hz, 15 == always on.
Auto MIC on Enabled:	1 byte, 0 == Off, 1 == On

4.98.3 Additional information

The Set Proxy Indication State Reply is sent in reply to this message.

Only the LED associated with the talk key on the rotary V-Series is RGB capable.

Lever key panels only have a Red LED on the TALK key and a Green LED on the listen key

Push button panels can only show a mix of Red & Green

4.98.4 Scope

Host to CSU

4.98.5 Performance

4.99 Reply Set Proxy Indication State

4.99.1 Description

This message details the resultant state of a third-party proxy key indication after a Set Proxy Keys State Request. These keys are third-party system proxy keys, that is keys that are driven by a third party system using HCI.

4.99.2 Structure

Message ID:	16 bit word, 313 (0x0139)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Number of Entries:	2 bytes. Number of keys entries in this message.
The following fields are repeated X times, dependant on above field.	
Port Number:	2 bytes, Panel port number
Region:	1 byte, Region number of key
Page:	1 byte, Page number of key
Key:	1 byte, Key number
Colour/Brightness:	1 byte, 0x1 == Bright, Red == 0x2, Green == 0x4, Blue == 0x8. Note, when bright is not set the LED brightness is dim. The off mode is set via the rate field below
Rate:	1 byte , 0 == Off, 1 == 1Hz, 2 == 2 Hz, 3 == 4Hz, 15 == always on.

Auto MIC on Enabled:	1 byte, 0 == Off, 1 == On
----------------------	---------------------------

4.99.3 Additional information

This message is sent in response to a Set Proxy Indication State Request.

4.99.4 Scope

CSU to Host .

4.99.5 Performance

4.100 Request Get Proxy Display Data

4.100.1 Description

This message requests the display data currently set for the specified third-party proxy displays. These keys are third-party system proxy keys, that is keys that are driven by a third-party system via HCI.

Display data is the information that is displayed on the physical display physically associated with the key i.e. OLED display in the case of the V-Series panel range.

Note: This data is stored in volatile matrix storage. If connection is lost with the matrix, the client should request the display data for all keys driven by the client in case the matrix has been reset.

4.100.2 Structure

Message ID:	16 bit word, 314 (0x013A)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Panel Number:	2 bytes. All panels == 65535, otherwise a valid panel number

4.100.3 Additional information

The Get Proxy Display Data Reply is sent by the matrix in reply to this message.

4.100.4 Scope

Host to CSU

4.100.5 Performance

4.101 Reply Get Proxy Display Data

4.101.1 Description

This message details the display data currently set for the specified third-party proxy displays. These displays are third-party system proxy displays, that is the display data is driven by a third-party system using HCI.

Display data is the information that is displayed on the physical display physically associated with the key i.e. OLED display in the case of the V-Series panel range.

4.101.2 Structure

Message ID:	16 bit word, 315 (0x013B)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Number of Entries: Count	2 bytes. Number of Display entries in this message
The following fields are repeated X times, dependant on above count field.	
Port Number	2 bytes. Panel port number
Region	1 byte. Region number of key
Page	1 byte. Page number of key
Key (associated)	1 byte. This is the key number of the talk key associated with this display
Display Text	20 bytes. Primary display text (wide character format)-unicode (UTF-16)

Alt Display Text	20 bytes. Alternative display text (wide character format) unicode (UTF-16) – For use with the EHX 'Alt text' feature.
Gain Level Indicator	1 byte. 0 == Min gain, 15 == Max gain, Intermediate values == Intermediate gains. 255 == Hide gain bar.
Icons	2 bytes. 0x01 == Vox, 0x02 == Monitored, 0x04 == Telos/SIP, 0x08 == Relay, 0x10 == Up arrow, 0x11 == Down arrow.

4.101.3 Additional information

This message is sent in response to a Get Proxy Display Data Reply Request.

4.101.4 Scope

CSU to Host.

4.101.5 Performance

4.102 Request Set Proxy Display Data

4.102.1 Description

This message sets the display data currently used for the specified third-party proxy display. These displays are third-party system proxy displays, that is the display data is driven by a third-party system using HCI.

Display data is the information that is displayed on the physical display physically associated with the panel keys, that is the OLED display in the case of the V-Series panel range.

Note – This data is stored in volatile matrix storage. If connection is lost with the matrix, the client should request the display data for all keys driven by the client in case the matrix has been reset.

4.102.2 Structure

Message ID:	16 bit word, 316 (0x013C)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Number of Entries: Count	2 bytes. Number of Display entries in this message
The following fields are repeated X times, dependant on above count field.	
Port Number	2 bytes. Panel port number
Region	1 byte. Region number of key
Page	1 byte. Page number of key
Key (associated)	1 byte. This is the key number of the talk key associated with this display
Display Text	20 bytes. Primary display text (wide character format) unicode (UTF-16)

Alt Display Text	20 bytes. Alternative display text (wide character format) unicode (UTF-16) – For use with the EHX 'Alt text' feature.
Gain Level Indicator	1 byte. 0 == Min gain, 15 == Max gain, Intermediate values == Intermediate gains. 255 == Hide gain bar.
Icons	2 bytes. 0x01 == Vox, 0x02 == Monitored, 0x04 == Telos/SIP, 0x08 == Relay, 0x10 == Up arrow, 0x11 == Down arrow.

4.102.3 Additional information

The Set Proxy Display Data Reply is sent in response to this message.

4.102.4 Scope

Host to CSU.

4.102.5 Performance

4.103 Reply Set Proxy Display Data

4.103.1 Description

This message specifies the display data currently used for the specified third-party proxy display as a result of the processing of a Set Proxy Display Data Request. These displays are third-party system proxy displays, that is the display data is driven by a third-party system using HCI.

Display data is the information that is displayed on the physical display physically associated with the panel keys i.e. OLED display in the case of the V-Series panel range.

4.103.2 Structure

Message ID:	16 bit word, 317 (0x013D)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Number of Entries: Count	2 bytes. Number of Display entries in this message
The following fields are repeated X times, dependant on above count field.	
Port Number	2 bytes. Panel port number
Region	1 byte. Region number of key
Page	1 byte. Page number of key
Key (associated)	1 byte. This is the key number of the talk key associated with this display
Display Text	20 bytes. Primary display text (wide character format) unicode (UTF-16)

Alt Display Text	20 bytes. Alternative display text (wide character format) unicode (UTF-16) – For use with the EHX 'Alt text' feature.
Gain Level Indicator	1 byte. 0 == Min gain, 15 == Max gain, Intermediate values == Intermediate gains. 255 == Hide gain bar.
Icons	2 bytes. 0x01 == Vox, 0x02 == Monitored, 0x04 == Telos/SIP, 0x08 == Relay, 0x10 == Up arrow, 0x11 == Down arrow.

4.103.3 Additional information

Sent in response to a Set Proxy Display Data Request.

4.103.4 Scope

CSU to Host.

4.103.5 Performance

4.104 Request Panel Keys Public Get State

4.104.1 Description

The message is used to get the current state (latched stated) of keys on the specified panel. It is either sent in response to a Panel Keys Public State Request or it can be sent out unsolicited if auto updates have been enabled for a pane

The message is used to request the state of the keys on the specified panel.

4.104.2 Structure

Message ID:	16 bit word, 320 (0x0140)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Port Number:	2 bytes. All panels == 65535, otherwise a valid panel number

4.104.3 Additional information

The Panel Keys Get Status Reply is sent in reply to this message.

4.104.4 Scope

Host to CSU.

4.104.5 Performance

4.105 Reply Panel Keys Public Get State

4.105.1 Description

The message is either sent in response to a Panel Keys Public State Request or it can be sent out unsolicited if auto updates have been enabled for a panel.

4.105.2 Structure

Message ID:	16 bit word, 321 (0x0141)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Key Count:	1 byte. Number of keys records in this message
The following fields are repeated X times, dependant on above count field.	
Port Number:	2 bytes. Panel port number
Region:	1 byte. Panel region of key
Page:	1 byte. Page number of key
Key:	1 byte. Key number
Pressed State:	1 byte. 0 == Key Off, 1 == Key On

4.105.3 Additional information

Sent in response to Panel Keys Public Get Status Reply. Also, if unsolicited Key updates have also been enabled on a port using the Panel Key Status Auto Updates Request, then unsolicited messages of this type will be sent to connected Host applications when the status changes.

4.105.4 Scope

CSU to Host.

4.105.5 Performance

4.106 Request Panel Keys Status Auto Updates

4.106.1 Description

The message is used to request the unsolicited transmission of Panel Key Status Reply messages when a key status updates.

4.106.2 Structure

Message ID:	16 bit word, 318 (0x013E)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Port Number Start:	First Port number to apply setting to
Port Number End:	Last port number to apply setting to
Enabled/Disabled:	0 == Disabled, 1 == Enabled.

4.106.3 Additional information

The Panel Keys Status Auto Updates Reply is sent in reply to this message.

4.106.4 Scope

Host to CSU

4.106.5 Performance

4.107 Reply Panel Keys Status Auto Updates

4.107.1 Description

The message is used to acknowledge the Host application of the received Panel Keys Status Auto Updates Request message.

4.107.2 Structure

Message ID:	16 bit word, 319 (0x013F)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Port Number Start:	First Port number to apply setting to
Port Number End:	Last port number to apply setting to
Enabled/Disabled:	0 == Disabled, 1 == Enabled.

4.107.3 Additional information

This message is sent in response to the Panel Key Status Auto Updates Request.

4.107.4 Scope

CSU to Host

4.107.5 Performance

4.108 Request Panel Keys Public Set State

4.108.1 Description

The message is used to request the change of state of the keys for a specific panel or all panels

4.108.2 Structure

Message ID:	16 bit word, 341 (0x0155)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count	1 byte, number of entries in message
The fields below are repeated count (above) times	
Port Number:	2 bytes. All panels == 65535, otherwise a valid panel number
Region	1 byte, Region Number of key
Key	1 byte, Key ID
Key Event	1 byte 0 = KEY_PRESS 1 = KEY_RELEASE 255 = KEY_PRESS_RELEASE (i.e. latch if possible)

4.108.3 Additional information

The Panel Keys Public Set Status Reply is sent in reply to this message.

4.108.4 Scope

Host to CSU

4.108.5 Performance

4.109 Reply Panel Keys Public Set State

4.109.1 Description

The message is used to reply to the Public Set State Request

4.109.2 Structure

Message ID:	16 bit word, 342 (0x0156)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count	1 byte, number of entries in message
The fields below are repeated count (above) times	
Port Number:	2 bytes. All panels == 65535, otherwise a valid panel number
Region	1 byte, Region Number of key
Key	1 byte, Key ID
Key Event	1 byte 0 = KEY_PRESS 1 = KEY_RELEASE 255 = KEY_PRESS_RELEASE (i.e. latch if possible)

4.109.3 Additional information

The Panel Keys Public Set Status Request is sent to request key state updates. This message is sent in reply to this.

4.109.4 Scope

CSU to Host.

4.109.5 Performance

4.110 Reply Telephony Key Status

4.110.1 Description

This message is used to send unsolicited key pressed state information to connected Host applications. This message is only transmitted from the matrix hosting the panel if telephony key state transitions have been enabled using the Request Telephony Key Status Enable message.

4.110.2 Structure

Message ID:	16 bit word, 348 (0x015C)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Panel Number:	2 bytes. Port number of panel hosting the key (Zero based port number)
Page:	1 byte. Key page number.
Region:	1byte. Key region number.
Key:	1 byte Key ID.
Key Text:	40 bytes. Unicode text buffer associated with the key
State:	1 byte. KEY_OFF = 0, ///< Key is released, and not latched. KEY_OFF_NOW_PRESSED = 1, ///< Key was off, has just been pressed. KEY_OFF_NOW_HELD = 2, ///< Key was off, has now been held. KEY_ON_NOW_PRESSED = 3, ///< Key was on, has just been pressed.

	KEY_ON_NOW_HELD = 4, ///< Key was on, has now been held. KEY_ON2_NOW_PRESSED = 5, ///< Key was on (event 2), has just been pressed. KEY_ON2 = 6, ///< Key is latched (event 2), and not currently pressed. KEY_ON = 7, ///< Key is latched, and not currently pressed. KEY_ON_MIRRORED_HELD = 8, ///< this is a mirrored key which is mirroring a key that is currently held. KEY_OFF_MIRRORED_HELD = 9, ///< Usually the same as the above Except for Dual Talk Listen. KEY_OFF_UNKNOWN = 10
--	--

4.110.3 Additional information

This message is only generated on telephony related key pressed when the Telephony Key Status Enable Request message received enabling the feature.

4.110.4 Scope

CSU to Host

4.110.5 Performance

4.111 Set Panel Audio Front End State Request

4.111.1 Description

This message is used to set the HCI requested state of the panel MIC, speaker and sidetone state.

When enabled, the state of the panel is exactly as though a talk key has been pressed. This HCI request simply acts as though a virtual talk key is pressed.

For example, if a panel talk key is already pressed and the HCI request is received then the panel audio hardware state does not change. However, if the panel talk key were now released the panel mic, sidetone etc would stay in the same state i.e. as though a talk key is still pressed, until the HCI disable equivalent of this message is received by the matrix.

This message is useful when e.g. a 3rd party application is using the HX panel of routing to other types of device than the HX matrix.

4.111.2 Scope

Client to Matrix.

4.111.3 Structure

Field	Size (Bytes)	Description / Value
Message ID	2	353 (0x161)
Flags	1	0x08
Magic Number	4	Set to 0xABBACEDE (Used to indicate HCI 2 protocol)
Schema Number	1	Set to 0x1
Panel Number	2	All panels == 65535, otherwise a valid panel number
Enabled State	1	0 == disabled, 1 == enabled

4.111.4 Additional Information

The set panel audio front end state reply is sent in response to this message.

4.112 Set Panel Audio Front End State Reply

4.112.1 Description

This message is sent in response to the Set Panel Audio Front End State Request acknowledging the successful receipt and processing of the request.

See the request message description for usage.

4.112.2 Scope

Matrix to Client.

4.112.3 Structure

Field	Size (Bytes)	Description / Value
Message ID	2	354 (0x162)
Flags	1	See HCI Flags section
Magic Number	4	Set to 0xABBACEDE (Used to indicate HCI 2 protocol)
Schema Number	1	Set to 0x1
Port number	2	Panel port number
Enabled State	1	0 == disabled, 1 == enabled

4.112.4 Additional Information

This message is sent in response to a set panel audio front end state request.

4.113 Get Panel Audio Front End State Request

4.113.1 Description

This message is used to get the HCI requested state of the panel MIC, speaker and sidetone state.

When enabled, the state of the panel is exactly as though a talk key has been pressed. This HCI request simply acts as though a virtual talk key is pressed.

For example, if a panel talk key is already pressed and the HCI request is received then the panel audio hardware state does not change. However, if the panel talk key were now released the panel mic, sidetone etc would stay in the same state i.e. as though a talk key is still pressed, until the HCI disable equivalent of this message is received by the matrix.

This message is useful when e.g. a 3rd party application is using the HX panel of routing to other types of device than the HX matrix.

4.113.2 Scope

Client to Matrix.

4.113.3 Structure

Field	Size (Bytes)	Description / Value
Message ID	2	355 (0x163)
Flags	1	0x08
Magic Number	4	Set to 0xABBACEDE (Used to indicate HCI 2 protocol)
Schema Number	1	Set to 0x1
Panel Number	2	All panels == 65535, otherwise a valid panel number

4.113.4 Additional Information

The get panel audio front end state reply is sent in response to this message.

4.114 Get Panel Audio Front End State Reply

4.114.1 Description

This message is sent in response to the Set Panel Audio Front End State Request acknowledging the successful receipt and processing of the request. The message contains the current state of the HCI requested panel audio front end of the specified panel.

See the request message description for usage.

4.114.2 Scope

Matrix to Client.

4.114.3 Structure

Field	Size (Bytes)	Description / Value
Message ID	2	356 (0x164)
Flags	1	See HCI Flags section
Magic Number	4	Set to 0xABBACEDE (Used to indicate HCI 2 protocol)
Schema Number	1	Set to 0x1
Port number	2	Panel port number
Enabled State	1	0 == disabled, 1 == enabled

4.114.4 Additional Information

This message is sent in response to a get panel audio front end state request.

4.115 Request Telephony Key Status Enable

4.115.1 Description

This message enables/disables the transmission of key status messages for telephony related keys.

4.115.2 Structure

Message ID:	16 bit word, 346 (0x015A)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Enabled:	1 byte. 1 == Enabled, 0 == Disabled.

4.115.3 Additional information

The Telephony Key Status Enable Reply is sent in response to this message. Telephony Key Status Reply messages are sent from this matrix once the feature has been enabled using this enable message.

4.115.4 Scope

Host to CSU.

4.115.5 Performance

4.116 Reply Telephony Key Status Enable

4.116.1 Description

This message is used to reply to the Telephony Key Status Enable Request.

4.116.2 Structure

Message ID:	16 bit word, 347 (0x015B)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Enabled:	1 byte. 1 == Enabled, 0 == Disabled.

4.116.3 Additional information

The Telephony Key Status Enable Reply is sent in response to this message. Telephony Key Status Reply messages are sent from this matrix once the feature has been enabled using this enable message.

4.116.4 Scope

CSU to Host

4.116.5 Performance

4.117 Request System Crosspoint

4.117.1 Description

This message is used to request the state of linked matrices point to point routes (cross-points).

The sources and destinations involved in these messages are user endpoint ports, that is the endpoint to trunk cross-points (intelligent linking hops) of the end to end routing are not included.

4.117.2 Structure

Message ID:	16 bit word, 201 (0x0C9)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.

4.117.3 Additional information

One or more System Crosspoint Replies are sent in response to this message. All linked matrices level crosspoint to or from this matrix will be included in the reply/replies to this message.

4.117.4 Scope

Host to CSU

4.117.5 Performance

4.118 Reply System Crosspoint

4.118.1 Description

This message is used to either reply to the system Crosspoint Request (and detail all remote routes to or from this matrix), or will be sent unsolicited from the matrix for an individual route when a route is made or broken.

The unsolicited route transition version of this message will be sent from both matrices that are at the end of the intelligent linking routing.

4.118.2 Structure

Message ID:	16 bit word, 200 (0x0C8)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Number of Entries: Count	2 bytes. Number of keys entries in this message.
The following fields are repeated X times, dependant on above count field.	
Port Info:	2 bytes. Bit 0 .. 10 – Port number Bit 11 .. 12 - Reserved Bit 13 – Is audio source Bit 14 – Is audio destination Bit 15 – Is MONITORED port (specified once at the beginning of list of port info elements to which it is being talked or listened to.
System Number:	1 byte. System number associated with the preceding port info.

	0 == local matrix, otherwise linked set matrix ID.
--	--

4.118.3 Additional information

One or more System Crosspoint Replies are sent in response to this message. All linked matrix level crosspoint to or from this matrix will be included in the reply/replies to this message.

4.118.4 Scope

CSU to Host.

4.118.5 Performance

4.119 Request Panel Keys Unlatch All

4.119.1 Description

This message is used to request the unlatching of all keys associated with the specified panel.

4.119.2 Structure

Message ID:	16 bit word, 334 (0x014E).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Card Slot:	1 byte. Card slot number 4.4 Slot numbering scheme
Port offset on card:	1 byte. Zero based offset into panel card.

4.119.3 Additional information

The Panel Keys Unlatch All Reply is sent in response to this message.

4.119.4 Scope

Host to CSU.

4.119.5 Performance

4.120 Reply Panel Keys Unlatch All

4.120.1 Description

This message is used to reply to the Panel Keys Unlatch All Request.

4.120.2 Structure

Message ID:	16 bit word, 335 (0x014F)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Card Slot:	1 byte. Card slot number 4.4 Slot numbering scheme
Port offset on card	1 byte. Zero based offset into panel card

4.120.3 Additional information

The Panel Keys Unlatch All Reply is sent in response to the Request Panel Keys Unlatch All message.

4.120.4 Scope

CSU to Host

4.120.5 Performance

4.121 Request Panel Discovery

4.121.1 Description

The message is used to request the discovery of any IP panels connected to the current LAN and populate a panel information cache in the matrix ready for retrieval.

4.121.2 Structure

Message ID:	16 bit word, 247 (0x0F7)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Sub ID:	1 byte. Set to 0 (Discovery Request)

4.121.3 Additional information

This IP Panel Discovery Reply is sent in response to this.

The Request IP Panel List message is used to retrieve the resulting cache.

4.121.4 Scope

Host to CSU.

4.121.5 Performance

4.122 Reply Panel Discovery

4.122.1 Description

The message is reply to the IP panel discovery type request.

4.122.2 Structure

Message ID:	16 bit word, 248 (0x0F8)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Sub ID:	1 byte. Set to 1 (Discovery Reply)

4.122.3 Additional information

This message is sent is response to the IP Panel Discovery Reply.

4.122.4 Scope

CSU to Host

4.122.5 Performance

4.123 Request IP Panel List

4.123.1 Description

The message is to request the content of the discovered IP panels cache stored in the matrix. This information can be used to display panels discovered on the network, but also to request the configuration / connected of such panels to the matrix.

4.123.2 Structure

Message ID:	16 bit word, 247 (0x0F7)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Sub ID:	1 byte. Set to 8 (Panel Cache Request)

4.123.3 Additional information

.

4.123.4 Scope

Host to CSU

4.123.5 Performance

4.124 Reply IP Panel List

4.124.1 Description

The message is reply to Request IP Panel List message. It contains the contents of the discovered IP panel cache.

4.124.2 Structure

Message ID:	16 bit word, 248 (0x0F8)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Sub ID:	1 byte. Set to 9 (Panel Cache Request)
Total Entries: Count	2 bytes. Total number of entries in the current discovery cache – Note if this number of entries cannot fit in one message then multiple messages will be sent. Each message will state the total number of entries (spanning all messages).
The following fields are repeated X times, dependant on above count field.	
Entry Length:	1 byte. Data length of entry (55 bytes)
Username:	10 bytes. Login Username (ASCII)
Panel IP Address:	4 bytes. Panel IP Address
Server Port	2 bytes. Login server port number
Server IP Address	4 bytes. Login server IP address
Network Mask	4 bytes. Panel Subnet mask

Network Gateway	4 bytes. External Login server IP address
DNS Server Address	4 bytes. Panel DNS server IP address
MAC Address	6 bytes. Panel MAC address
Net Options	1 byte.
Connection Type	1 byte. LAN, WAN or Internet VOIP connection type
Login Status	2 bytes.
Transaction Number	2 bytes.
Panel Type	2 bytes. V-Series Panel Type
Panel Up Time	4 bytes. Number of seconds since last boot
Connection Up Time	4 bytes. Number of seconds that current connection has been up

4.124.3 Additional information

This message is sent in response to the IP Panel Discovery Request.

If additional messages are to follow to complete the cache transfer then the continuation bit will be set in the message flags.

4.124.4 Scope

CSU to Host .

4.124.5 Performance

4.125 Request IP Panel Settings Assign

4.125.1 Description

The message is sent to apply IP settings to a specified IP V-Series panel. Note, this does not work across different networks, unless the panel being configured is already connected to the matrix i.e. it is a re-assignment.

These are IVC type IP panel connections, not AES67.

4.125.2 Structure

Message ID:	16 bit word, 247 (0x0F7)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Sub ID:	1 byte. Set to 2 (Panel Configuration Request)
Entry Length:	1 byte. Entry Data length (set to 54)
Setting Mask:	1 byte. Set to 1
Card Number:	1 byte. Ignored when setting mask == 1
Card Port Offset:	1 byte. Ignored when setting mask == 1
Username:	10 bytes. Login Username (ASCII)
Password:	10 bytes. Login password (ASCII)
Server IP Address:	4 bytes. Login server IP address
Server Port:	2 bytes. Login server port number
Connection Type:	1 byte. 0 = LAN, 1 = WAN, 2 = Internet, 3 = Default i.e. type in MAP. Note Default should be used here when possible, otherwise panel may

	need to connect twice to apply non map default type.
Network Options:	1 bytes. Set to 1
Panel IP Address:	4 bytes. Panel IP Address
Network Mask:	4 bytes. Panel Subnet mask
Network Gateway:	4 bytes. External Login server IP address
DNS Server Address:	4 bytes. Panel DNS server IP address
MAC Address:	6 bytes. Panel MAC address

4.125.3 Additional information

This message is sent in response to the IP Panel Discovery Request.

If additional messages are to follow to complete the cache transfer then the continuation bit will be set in the message flags.

4.125.4 Scope

CSU to Host.

4.125.5 Performance

4.126 Reply IP Panel Settings Assign

4.126.1 Description

The message is sent in reply to the IP Panel Settings Apply Request.

4.126.2 Structure

Message ID:	16 bit word, 248 (0x0F8)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Sub ID:	1 byte. Set to 3 (Panel Configuration Reply)
Status:	1 byte. 0 = Ok, -1 and -2 = length/unpack error
MAC Address:	6 bytes. Panel MAC address

4.126.3 Additional information

This message is sent in response to the IP Panel Settings Apply Request.

A successful response only indicates that the request is valid and has therefore been sent onto the panel.

4.126.4 Scope

CSU to Host

4.126.5 Performance

4.127 Request Panel Shift Page Action

4.127.1 Description

This message is used to request the current page of a specified panel, or to request the change of current page of a specified panel.

4.127.2 Structure

Message ID:	16 bit word, 339 (0x0153).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Action:	1 byte. 0 = Get Current Page, 1 = Set Current Page
Count:	2 bytes, number page set actions in message
All fields below repeated count field (see above) times	
Card Slot:	1 byte. Card slot number. For beltpack roles card slot 0xFF. 4.4 Slot numbering scheme
Port offset on card:	1 byte. Zero based offset into panel card. For beltpack roles this field is the offset from the first role ID (600)
Page Number:	1 byte. Only included if the Action field above is set to 1. The page number to make the currently selected page on this panel. 0 == main page, 1 == first shift page etc.

4.127.3 Additional information

The Panel Page Action Reply is sent in response to this message.

This feature is dependent on the Request Panel Keys Status Auto Request being sent into the matrix (i.e. enabled) for this panel.

4.127.4 Scope

Host to CSU.

4.127.5 Performance

4.128 Reply Panel Shift Page Action

4.128.1 Description

This message is used to respond to the Panel Page Action Reply. It contains the current active page of the specified.

4.128.2 Structure

Message ID:	16 bit word, 340 (0x0154).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Action:	1 byte. 0 = Get Current Page, 1 = Set Current Page
Card Slot:	1 byte. Card slot number. For beltpack roles card slot 0xFF. 4.4 Slot numbering scheme
Port offset on card:	1 byte. Zero based offset into panel card. For beltpack roles this field is the offset from the first role ID (600)
Page Number:	1 byte. The page number to make the currently selected page on this panel.

4.128.3 Additional information

The Reply Panel Shift Page Action is sent in response to the Request Panel Shift Page Action.

4.128.4 Scope

CSU to Host.

4.128.5 Performance

4.129 Request Key Group Action

4.129.1 Description

This message is used to request the association of an key entity (port, IFB instance etc) to a key group present in the matrix configuration. The matrix replies with a Reply Key Group Status message.

4.129.2 Structure

Message ID:	2 bytes, 251 (0x00FB)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Sub Message ID:	1 byte, set to 0x1
Key Group System Number	1 byte, set to system number of the key group
Key Group ID	2 bytes, set to instance number of the key group
Target Entity System ID	1 byte, system number of entity (port, IFB etc)
Entity Type	1 byte, type ID of entity
Entity Instance	2 bytes, instance number of entity

4.129.3 Usage

Used to request the assignment of the specified intercom entity to the specified key group.

If the message is being sent to the owning system of the key group then key group system 0 can be used i.e. local.

The request key group action can be sent to any matrix in the linked set. The matrices will propagate the action to the other matrices in the linked set.

4.129.4 Scope

Host to CSU

4.129.5 Performance

4.129.6 Example

Request the assignment of system 3, port 5 to key group 7 created on system 10

00 FB 08 AB BA CE DE 01 01 0A 00 07 03 01 00 05

4.129.7 Related messages

Reply Key Group Status

4.129.8 See also

4.130 Request Key Group Status

4.130.1 Description

This message is used to request the current assignment state of the specified key group.

4.130.2 Structure

Message ID:	2 bytes, 251 (0x00FB)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Sub Message ID:	1 byte, set to 0x3
Key Group System Number	1 byte, set to system number of the key group
Key Group ID	2 bytes, set to instance number of the key group

4.130.3 Usage

Used to request the assignment status of the specified key group.

If the message is being sent to the owning system of the key group then key group system 0 can be used i.e. local.

4.130.4 Scope

Host to CSU

4.130.5 Performance

4.130.6 Example

Request the assignment of system 3, port 5 to key group 7 created on system 10

00 FB 08 AB BA CE DE 01 03 0A 00 07

4.130.7 Related messages

Reply Key Group Status, Request Key Group Action, Reply Key Group Action

4.130.8 See also

4.131 Reply Key Group Status

4.131.1 Description

This message is used to reply to the request key group status message.

4.131.2 Structure

Message ID:	2 bytes, 251 (0x00FB)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Repeated Elements Below, the number of elements contains need to be established from the HCI message length field i.e. de-serialise the fields below until the message length is consumed.	
Key Group System Number	1 byte, System number of the key group
Key Group ID	2 bytes, Instance number of the key group
Target Entity System ID	1 byte, System number of entity (port, IFB etc)
Entity Type	1 byte, Type ID of entity
Entity Instance	2 bytes, instance number of entity

4.131.3 Usage

This message replies to the Request Key Group Status message.

4.131.4 Scope

CSU to Host

4.131.5 Performance

4.131.6 Example

Reply to the status request for key group 7 where this key group is currently assigned to system 3, port 5.

00 FB 08 AB BA CE DE 01 0A 00 07 03 01 00 05

4.131.7 Related messages

Request Key Group Status, Request Key Group Action, Reply Key Group Action

4.131.8 See also

4.132 Request All Key Group Statuses

4.132.1 Description

This message is used to request the key entity associated for all key groups on the current matrix.

4.132.2 Structure

Message ID:	2 bytes, 251 (0x00FB)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Sub Message ID:	1 byte, set to 0x2

4.132.3 Usage

Used to request the assignment status of all key groups on the current matrix.

4.132.4 Scope

Host to CSU

4.132.5 Performance

4.132.6 Example

Request the assignment of system 3, port 5 to key group 7 created on system 10

00 FB 08 AB BA CE DE 01 02

4.132.7 Related messages

Request Key Group Status, Reply Key Group Status, Request Key Group Action, Reply Key Group Action

4.132.8 See also

4.133 Request Frame Status

4.133.1 Description

This message is used to request the core matrix frame status i.e. not interface cards.

4.133.2 Structure

Message ID:	2 bytes, 97 (0x0061)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.

4.133.3 Usage

Used to request the matrix frame status from the matrix.

This information includes PSU alarm status and CPU temperature.

Note: The HX Delta matrix supports more PSU alarms than the Omega or Median.

4.133.4 Scope

Host to CSU

4.133.5 Performance

4.133.6 Example

Request the matrix frame status.

00 61 08 AB BA CE DE 01

4.133.7 Related messages

Reply Frame Status

4.133.8 See also

4.134 Reply Frame Status

4.134.1 Description

This message is used to deliver the current matrix frame status to the host(s). It can be requested, but is also generated automatically as PSU alarm states transation.

4.134.2 Structure

Message ID:	2 bytes, 98 (0x0062)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
PSU Status	2 bytes, Only functional on the HX Delta Ext Psu 1 fail - 0x0001 Ext Psu 2 fail - 0x0002 Int Psu 1 fail - 0x0004 Int Psu 2 fail - 0x0008 Fan 1 fail - 0x0010 Fan 2 fail - 0x0020 Config fail - 0x0040 // Fired by CPU App e.g. when master slave switchover occurs Ext. Alarm- 0x0080 // alarm GPO state i.e. activated when core alarms bits are high Overtemp - 0x0100
CPU Card Temperature	2 bytes, signed temperature measured at the CPU card in degrees centigrade. See below for usage.

4.134.3 Usage

Used to request the matrix frame status from the matrix.

This information includes PSU alarm status and CPU temperature.

Note: The HX Delta matrix supports more PSU alarms than the Omega or Median.

4.134.4 Scope

Host to CSU

4.134.5 Performance

4.134.6 Example

Request the matrix frame status.

00 62 08 AB BA CE DE 01 0C 00 00 15

Sample code for display of CPU temperature

```
u8 point5 = (temp & 0x0080) ? 5 : 0;
```

```
u16 temp >>= 8;
```

```
if(temp & 0x0080)
```

```
    temp |= 0xFF00;    /* Sign extend data */
```

```
else
```

```
    temp &= 0x00FF;    /* Clear sign bit & any rubbish */
```

```
printf("Temp %d.%d deg C", temp, point5);
```

4.134.7 Related messages

Reply Frame Status

4.134.8 See also

4.135 Request Xpt and Level Status

4.135.1 Description

Requests the broadcast of all xpts currently in the matrix along with their current level.

4.135.2 Structure

Field	Description
Message ID:	16 bit word 150 (0x0096).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Version	2 bytes, set to 1
Count	2 bytes, number of ports for which xpts being requested
Fields below repeated <i>Count</i> (field above) times.	
Port Number	2 bytes, port number for which xpts being requested

4.135.3 Usage

This message triggers a background matrix service to send out the xpt source, destination, and level for all crosspoints in the system.

4.135.4 Scope

Host to CSU only.

4.135.5 Performance

4.135.6 Example

Request all xpt and level information for one source port, port 5.

00 96 08 AB BA CE DE 01 0 01 00 01 00 05

4.135.7 Related messages

Reply Xpt and level Status

4.135.8 See also

4.136 Reply Xpt and Level Status

4.136.1 Description

Delivers the source, destination, and level information for xpts currently present in the system.

This can be requested by the host (all ports only) or is sent unsolicited from the matrix when a xpt is made, broken or level adjusted.

4.136.2 Structure

Field	Description
Message ID:	16 bit word 151 (0x0097).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count	2 bytes, number of xpts and level entries in messages (see below)
Entry below is repeated count (field above) times	
Monitored	1 bit, if set this is the header port entry for subsequent entries that are in relation to this port
Talk Xpt	1 bit, Monitor port is the xpt source
Listen Xpt	1 bit, Monitor port is the xpt destination
Reserved	3 bits, not currently used
Port	10 bits, Port number
Level	2 bytes, Crosspoint level

4.136.3 Usage

Xpt source, destination, and level information for all xpts in the matrix.

4.136.4 Scope

CSU to Host only.

4.136.5 Performance

4.136.6 Example

Xpt and level information for xpt between port 5 (source) and port 7 (destination) at 0 dB (i.e. level 0xCC).

00 97 08 AB BA CE DE 01 00 01 80 05 40 07

4.136.7 Related messages

Request Xpt and level Status

4.136.8 See also

4.137 Request Trunk Usage Statistics

4.137.1 Description

Requests a report on the current trunk statistics held by the matrix. The information contained in this message is the equivalent of that displayed every 10 minutes in the EHX event log.

4.137.2 Structure

Field	Description
Message ID:	16 bit word 368 (0x0170).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.

4.137.3 Usage

This message results in one or more Reply Trunk Usage Statistics messages being received by the Host, this depends on the number and type of trunks configured in the system.

4.137.4 Scope

Host to CSU only.

4.137.5 Performance

4.137.6 Example

Request the current trunk usage statistics from the matrix.

01 70 08 AB BA CE DE 01

4.137.7 Related messages

Reply Trunk Usage Statistics

4.137.8 See also

4.138 Reply Trunk Usage Statistics

4.138.1 Description

This message contains a report on the current trunk statistics held by the matrix. The information contained in this message is the equivalent of that displayed every 10 minutes in the EHX event log.

4.138.2 Structure

Field	Description
Message ID:	16 bit word 369 (0x0171).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count	2 bytes, number of entries (description below) in this message.
The fields below are repeated count (field above) times in the current message (12 bytes per record)	
Matrix ID	1 byte, ID of the trunk associated matrix
Statistics Type	1 byte. Indicates the category of trunk information in this record. The bits are set as follows. Fibre trunks – 0x01 Tx trunks – 0x02 Reserved trunks – 0x04 Fibre Shared Resources – 0x07 (see special fields below for this case).

Fields type 1. Used for all count entries apart from the case where the Fibre Shared Resources (bit 0x07) type info bit is set and the Matrix ID is not the matrix ID of the local matrix.	
In use count	2 bytes, Current trunk usage
10 min peak	2 bytes, Peak usage in the last 10 minutes
10 min mean	2, bytes, Mean usage in the last 10 minutes
Peak (since start)	2 bytes, Peak usage since the last reset of the matrix
Total	2 bytes, Total trunks in this category
Fields type 2. Used only in the case where the Fibre Shared Resources (bit 0x07) type info bit is set and the Matrix ID is the matrix ID of the local matrix.	
Allocated RX resources	2 bytes, Allocated RX port pool count (entire pool)
Peak RX Resources	2 bytes, peak number of Rx fibre port resources used (entire pool) since the last matrix reset.
TDM ports allocated	2 bytes, Current number of TDM ports allocated in the base fibre driver
Peak TDM port allocated	2 bytes, Peak number of TDM ports allocated in the base fibre driver since the last matrix reset
Total TDM ports	2 bytes, Total number of TDM ports available to the fibre driver based on the current configuration.

4.138.3 Usage

This message is sent from the matrix to the host to report the recorded usage of all trunks categories and resources in the system. Depending on the number of matrices in the system and the number of trunk categories in use multiple messages may be required to output the full status. The final message of the status set of messages will have the continuation bit cleared.

4.138.4 Scope

CSU to Host only.

4.138.5 Performance

4.138.6 Example

Reply for the final section of a trunk Usage Statistics reply where the only the local matrix (system 1) fibre shared resources are output. The current state is that 5 fibre RX ports and TDM ports are in use, the peak usage was 6 and the total in this system is 16.

01 71 08 AB BA CE DE 01 00 01 01 08 00 05 00 06 00 05 00 06 10

4.138.7 Related messages

Request Trunk Usage Statistics

4.138.8 See also

4.139 Request Rack Configuration Status

4.139.1 Description

This message is used to request the configuration status from the matrix.

4.139.2 Structure

Field	Description
Message ID:	16 bit word 44 (0x002c).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Sub ID	1 byte, Set to 12.

4.139.3 Usage

The reply to this message includes the matrix and configuration name as well as passcoded feature information.

4.139.4 Scope

CSU to Host only.

4.139.5 Performance

4.139.6 Example

00 2C 08 AB BA CE DE 01 0C

4.139.7 Related messages

Request Rack Configuration Status.

4.139.8 See also

4.140 Reply Rack Configuration Status

4.140.1 Description

This message is used to request the configuration status from the matrix.

4.140.2 Structure

Field	Description
Message ID:	16 bit word 44 (0x002c).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Sub ID	1 byte, Set to 13.
Entry Length	2 bytes, length of entry payload
Matrix ID	1 byte, system number
Download Year	1 byte
Download Month	1 byte
Download Day	1 byte
Download Hour	1 byte
Download Min	1 byte
Download Sec	1 byte
Download msec	1 byte
System Identity	18 bytes

Download Flags	1 byte
Map Format	5 Bytes
Checksum	2 16 bit words
Matrix Description	20 bytes
Hci License	1 byte
Hci Extended License	1 byte
Virtual Client Users	1 byte
Dynam-EC Licenses	1 byte
Delta Lite Extension License	1 byte

4.140.3 Usage

The reply to this message includes the matrix and configuration name as well as passcoded feature information.

4.140.4 Scope

CSU to Host only.

4.140.5 Performance

4.140.6 Example

00 2C 08 AB BA CE DE 01 0C

4.140.7 Related messages

Request Rack Configuration Status.

4.140.8 See also

4.141 Request To Enable/Disable IPA Card Redundancy Switch

4.141.1 Description

This message is used to enable or disable IPA card redundancy switch.

4.141.2 Structure

Field	Description
Message ID:	16 bit word 386 (0x0182).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
IPA Card Slot Number:	1 byte, Set to 0xff for all IPA cards.
Switch State:	1 byte, 1 for disable switching, 0 for enable switching.

4.141.3 Usage

The request can be used to abort card switching and capture a snapshot of the IPA card state for debugging purpose.

4.141.4 Scope

CSU to Matrix only.

4.141.5 Performance

4.141.6 Example: disable all card switching

01 82 08 AB BA CE DE 01 ff 01

4.141.7 Related messages

Request Rack Configuration Status.

4.141.8 See also

4.142 Reply To Enable or Disable IPA Card Redundancy Switch

4.142.1 Description

This message is used to reply the request for enabling or disabling IPA card redundancy switch.

4.142.2 Structure

Field	Description
Message ID:	16 bit word 387 (0x0183).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Number of IPA Card(s):	1 byte, count (n) of reported IPA card(s) in this message.
Switch States:	2*n byte, two bytes per card, first byte is slot number, second byte is switch state (1 for switch disenabled, 0 for switch enabled).

4.142.3 Usage

The reply indicates whether an IPA card redundancy is enable or disabled.

4.142.4 Scope

Matrix to CSU only.

4.142.5 Performance

4.142.6 Example

01 83 10 AB BA CE DE 01 01 00

4.142.7 Related messages

Request to enable/disable IPA redundancy switch.

4.142.8 See also

4.143 Request Macro Panel Keys Public State

4.143.1 Description

The message is used to get the current state (latched stated) of the Dynam-EC macro keys on the specified panel.

4.143.2 Structure

Message ID:	16 bit word, 370 (0x0172)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Port Number:	2 bytes. All panels == 65535, otherwise a valid panel number

4.143.3 Additional information

The Reply Macro Panel Keys Public State is sent in reply to this message.

4.143.4 Scope

Host to CSU.

4.143.5 Performance

4.144 Reply Macro Panel Keys Public State

4.144.1 Description

The message is either sent in response to a Request Macro Panel Keys Public State or it is sent out unsolicited if a panel comes online once the matrix is up and the matrix has received a Request Macro Panel Keys Public State message with the panel field set to indicate 'get all'.

4.144.2 Structure

Message ID:	16 bit word, 371 (0x0173)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Key-Count:	1 byte. Number of keys records in this message
The following fields are repeated X times, dependant on above count field.	
Port Number:	2 bytes. Panel port number
Region:	1 byte. Panel region of key
Page:	1 byte. Page number of key
Key:	1 byte. Key number
Macro Key Group System	1 byte, System number of Macro Key group
Macro Key Group ID	2 bytes, Macro group identifier
Pressed State:	1 byte. 0 == Key Off, 1 == Key On

4.144.3 Additional information

Sent in response to Panel Keys Public Get Status Reply. Also, if unsolicited Key updates have also been enabled on a port using the Panel Key Status Auto Updates Request, then unsolicited messages of this type will be sent to connected Host applications when the status changes.

4.144.4 Scope

CSU to Host.

4.144.5 Performance

4.145 Request Alt Text State

4.145.1 Description

The message is used request the 'Alt Text' display state of a specified panel or all panels.

See the EHX User Guide for a description of the Alt Text feature. This feature is either active or inactive for each panel in the system. The feature can be activated/deactivated via a panel key or via HCI message.

4.145.2 Structure

Message ID:	16 bit word, 376 (0x0178)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Port Number:	2 bytes. All panels == 65535, otherwise a valid panel number

4.145.3 Additional information

The Reply Alt Text Public State is sent in reply to this message.

If a get all (panel == 0xFFFF) is requested only those panels with the Alt Text feature active will be included in the reply. Any panel not included in the reply will be considered to have the Alt Text feature deactivated.

4.145.4 Scope

Host to CSU.

4.145.5 Performance

4.146 Reply Alt Text State

4.146.1 Description

The message is used to report the state of the Alt Text feature of a panel/panels in the local matrix.

See the EHX User Guide for a description of the Alt Text feature. This feature is either active or inactive for each panel in the system. The feature can be activated/deactivated via a panel key or via HCI message.

4.146.2 Structure

Message ID:	16 bit word, 377 (0x0179)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count	2 bytes. Number of entries in the message (see below for payload per entry)
Fields below are repeated 'count' numbers of times in the message	
Port Number	2 bytes. 0 based port number of panel
State	1 byte. 0 == Off, 1 == On.

4.146.3 Additional information

This message is sent in reply to the Request Alt Text Public State.

It is also sent unsolicited from the matrix if the alt text state is toggled at the panel using the assignable ALT TEXT special key.

If this message is sent in response to a get all (panel == 0xFFFF) only those panels with the Alt Text feature active will be included in the reply. Any panel not included in the reply will be considered to have the Alt Text feature deactivated.

4.146.4 Scope

CSU to Host

4.146.5 Performance

4.147 Request Alt Text Set

4.147.1 Description

The message is used to set the state of the Alt Text feature of a panel/panels in the local matrix.

See the EHX User Guide for a description of the Alt Text feature.

4.147.2 Structure

Message ID:	16 bit word, 378 (0x0180)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Port Number	2 bytes. 0 based port number of panel
State	1 byte. 0 == Off, 1 == On.

4.147.3 Additional information

This message is sent to request the activation or deactivation of the alt text state of the specified panel.

4.147.4 Scope

Host to CSU.

4.147.5 Performance

4.148 Reply Alt Text Set

4.148.1 Description

The message is used to reply to the Request Alt Text Set message.

See the EHX User Guide for a description of the Alt Text feature.

4.148.2 Structure

Message ID:	16 bit word, 379 (0x0181)
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Port Number	2 bytes. 0 based port number of panel
State	1 byte. 0 == Off, 1 == On.

4.148.3 Additional information

This message is sent to confirm the activation or deactivation of the alt text state of the specified panel.

4.148.4 Scope

Host to CSU.

4.148.5 Performance

4.149 Request Assigned Keys (With Labels)

4.149.1 Description

Requests all assigned key configuration for selected panel. These assignments are the net result of the map configuration downloaded baseline plus any HCI, online, panel based assignments on top of this.

The message is the same as the Request Assigned Keys (ID 231), however each entry in the reply includes the label shown on the key.

4.149.2 Structure

Message ID	Flags	Protocol Tag	Protocol Schema	Slot Number	Port
------------	-------	--------------	-----------------	-------------	------

Field	Description
Message ID:	16 bit word 380 (0x017C).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Slot:	Card Slot Number 1 byte 4.4 Slot numbering scheme
Port:	Port offset from first port of the card 1 byte

4.149.3 Usage

Can be used to retrieve assigned keys for a particular panel. These are the currently active keys for a panel, whether they be from the map or some form of online assignment.

4.149.4 Scope

Host to CSU only.

4.149.5 Performance

4.149.6 Example

Request Information for Slot 2 Port Offset 1

01 7C 08 AB BA CE DE 01 02 01

4.149.7 Related messages

4.149.8 See also

4.150 Reply Assigned Keys (With Labels)

4.150.1 Description

Returns the configuration of all assigned keys for selected panel. These assignments are the net result of the map configuration downloaded baseline plus any HCI, online, panel-based assignments on top of this.

The message is the same as the Reply Assigned Keys (ID 232), however each entry in the reply includes the label shown on the key.

4.150.2 Structure

Message ID	Flags	Slot	Port	Count	KeyData[0]	KeyData[1]	KeyData[2]		KeyData[Count - 1]
------------	-------	------	------	-------	--------------	--------------	--------------	--	----------------------

Region	Key	Page	Entity	Key Status	Key Operation	Key Config	Key Action	Label
--------	-----	------	--------	------------	---------------	------------	------------	-------

Field	Description	Size
Message ID:	16 bit word 381, (0x017D)	
Flags:	1 byte, see FLAGS on page 59.	
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.	
Protocol Schema:	1 byte. Schema 1 and 2 currently supported. The request format is the same for both schemas, but more information is returned in the reply when schema 2 is requested.	
Slot:	Card Slot Number 1 byte	

	4.4 Slot numbering scheme	
Port:	Port offset from first port of the card 1 byte	
Endpoint Type:	2 bytes. Only includes this field if schema 2 was requested. The field is only used when the slot and offset fields above target a role. If this is not a role target then this field is set to 0. This field states the endpoint type currently used by the role e.g. 0x8200 would signify that the FS II beltpack is currently in use and therefore that these are FS II BP keys.	
Count:	Number of locally assigned keys on this panel. 1 byte	
Region:	Region this key is on 1 byte	
Key:	This keys Id 1 byte	
Page:	Page this key is on 1 byte	
Entity:	e.g EN_PORT, EN_GROUP 16 bit word	
Key Status:		
KeyState	Bits 6 to 7	
ListenMode	Bit 5	
PotAssigned	Bit 4	
PotState	Bits 0 to 3	
PotNumber	16 bit word	
Key Operation: 4 bytes		
Unpaged	0 paged, 1 unpaged Bit 8	
TextMode	0 not displayed, displayed Bit 7	
Dual	0 normal, 1 double width Bit 6	
Dial	0 dial mode off, 1 dial mode on Bit 5	
LatchMode	0 – LATCH_NON_LATCH	

Bits 0 – 3	1 – LATCH 2 – NON_LATCH 3 – RETURN_AFTER_HOLD 4 – TRI_ROT 5 – TRI_TOG 6 – RETURN_AFTER_LATCH 7 – DEL_LATCH 8 – HKS_LATCH	
Group Bits 4 – 7	If non-zero, other keys on the same panel with the same group number will be deselected if this key is selected	
Deactivating	1 all interlock group keys may be off Bit 3	
Make/Break	1 interlock group make before break Bit 2	
Cross Page	1 keys interlocked across pages in same region Bit 1	
CMAPSi Sp1	CMAPSi flag special 1 Bit 0	
CMAPSi Sp2	CMAPSi flag special 2 Bit 7	
Region Value Bit 4 – 6	Key region number	
Stacked group	If set indicates that the key is a stacked group Bit 3	
unused	Bit 0 – 2	
Page Value	If the key is a page key, this value is the page it makes a switch to when pressed, 1 byte	
Key Config: 26 bytes		
System Number	System Number 1 byte	
Unused	Spare 1 byte	

Specific	Use depends on entity 16 Bit Word	
Secondary	Dcc/DialCode(speed_dial) 16 Bit word	
KeyAction: 16 Bit word		
Force Listen	0 disable, 1 enable Bit 15	
Talk	0 disable, 1 enable Bit 14	
Listen	0 disable, 1 enable Bit 13	
Hold to Talk	0 disable, 1 enable Bit 12	
Initial State	0 not selected, 1 selected Bit 11	
Assign Locally	0 disable, 1 enable Bit 10	
Assign Remotely	0 disable, 1 enable Bit 9	
Locally Assigned	0 un-assigned, 1 assigned Bit 8	
Reserved	Unused bits 0 – 7	
GUID	Unique ECS GUID Number 16 Bytes	
Label	20 bytes, 10 Unicode characters	

4.150.3 Usage

Used to return locally assigned key information for selected panel.

Missing byte info

4.150.4 Scope

CSU to Host only.

4.150.5 Special notes

Note: Many messages may be required to be transmitted by the CSU to the host as it is expected the local key information data may be larger than the maximum data size for a message, in that case the flags will be used as in section 4.2 of the HCI document.

4.150.6 Performance

4.150.7 Example

4.150.8 Related messages
Request Locally Assigned Keys.

4.150.9 See also

4.151 Request Role State

4.151.1 Description

This message is used to request the current status of a matrix role.

4.151.2 Structure

Message ID:	16 bit word, 388 (0x0184).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Role:	2 bytes, Role number, or use 0xFFFF to request all roles.

4.151.3 Additional information

The Reply Role State message is sent in response to this.

4.151.4 Scope

Host to CSU.

4.151.5 Performance

4.152 Reply Role State

4.152.1 Description

This message is used to send the current role status for one or more roles to the host. This message can be requested or sent unsolicited on a state transition.

4.152.2 Structure

Message ID:	16 bit word, 389 (0x0185).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	2 bytes
Fields below repeated <count> times	
Role Number:	2 bytes, Role Id e.g. 600, 601.
Current in use status:	1 byte, 0 == Free, 1 == Allocated From Pool, 2 == Allocated (Fixed association)
Physical port:	2 bytes, Port number of hardware to which the current role is allocated. This will be set to 0xFFFF when the role is not currently allocated.
Configured Endpoint type:	2 bytes, Panel type associated with the key configuration in use with this role. This may not match the physical endpoint type below when e.g. a 1 RU Rotary panel used to access a role that is only configured for use with a V-Series 2U Lever pane. This will be set to 0 when the role is not currently allocated.

Physical Endpoint type:	2 bytes, Physical panel type of the endpoint currently using this role. This will be set to 0 when the role is not currently allocated.
-------------------------	---

4.153 Request Role State Set

4.153.1 Description

This message is used to request a change to the current role assignment state.

4.153.2 Structure

Message ID:	16 bit word, 390 (0x0186).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Role:	2 bytes, Role number
New Status:	1 byte, 0 == Free, 1 == Allocated From Pool
Physical port:	2 bytes, Port number of hardware to which the current role is allocated. This will be set to 0xFFFF if not in use e.g. if the role is being allocated to a physical port then the status will be set to 1 and the physical port will be set to a valid port number.

4.153.3 Additional information

The Reply Role State Set message is sent in response to this.

4.153.4 Scope

Host to CSU.

4.153.5 Performance

4.154 Reply Role State Set

4.154.1 Description

This message is used to reply to Request Role State Set message.

4.154.2 Structure

Message ID:	16 bit word, 391 (0x0187).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Role:	2 bytes, Role number.
Current in use status:	1 byte, 0 == Free, 1 == Allocated From Pool
Physical port:	2 bytes, Port number of hardware to which the current role is allocated. This will be set to 0xFFFF if not in use e.g. if the role is being allocated to a physical port then the status will be set to 1 and the physical port will be set to a valid port number.
Success:	1 byte, 1 == Success, 0 == Failure.

4.154.3 Additional information

Sent in response to a Request Role State Set message.

4.154.4 Scope

CSU to Host.

4.154.5 Performance

4.155 Request Network Redundancy Endpoint Status

4.155.1 Description

This message is used to request a change to the current role assignment state.

4.155.2 Structure

Message ID:	16 bit word, 392 (0x0188).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Port:	2 bytes, Endpoint port number. Use 0xFFFF to get the status for all endpoints.

4.155.3 Additional information

The Reply Network Redundancy Endpoint Status message is sent in response to this.

4.155.4 Scope

Host to CSU.

4.155.5 Performance

4.156 Reply Network Redundancy Endpoint Status

4.156.1 Description

This message is used to reply to Request Network Redundancy Endpoint Status message.

4.156.2 Structure

Message ID:	16 bit word, 393 (0x0189).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	2 bytes, number of entries in this message.
Fields below repeated <count> times	
Physical port:	2 bytes, Port number associated with endpoint
Active Lan Port:	1 byte, 0 == UNKNOWN, 1 == MAIN, 2 == BACKUP.
Switch Count	2 bytes, number of times that the endpoint reports that it has switched since boot up.

4.156.3 Additional information

Sent in response to a Request Network Redundancy Endpoint Status message.

When a get all request is responded to only those ports for which network redundancy is configured will be include in the response.

4.156.4 Scope

CSU to Host.

4.156.5 Performance

4.157 Request Network Redundancy Card Status

4.157.1 Description

This message is used to request the main/standby state of a card.

4.157.2 Structure

Message ID:	16 bit word, 394 (0x018A).
Flags:	1 byte, see FLAGS on page 57.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Slot:	1 byte, Actual slot number as per the silk screen on the physical rack. Use 0xFF to request all E-IPA slots

4.157.3 Additional information

The Reply Network Redundancy Card Status message is sent in response to this.

4.157.4 Scope

Host to CSU.

4.157.5 Performance

4.158 Reply Network Redundancy Card Status

4.158.1 Description

This message is used to reply to Request Network Redundancy Card Status message.

4.158.2 Structure

Message ID:	16 bit word, 395 (0x018B).
Flags:	1 byte, see FLAGS on page 57.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Count:	1 bytes, number of entries in this message.
Fields below repeated <count> times	
Physical slot:	1 bytes, Slot number of the card
IVC State:	1 byte, 0 = Main, 1 = Standby
IVC Switchover Count	2 bytes, number of times that the card has switched active IVC lan port since boot up.
AoIP State:	1 byte, 0 = Main, 1 = Standby
AoIP Switchover Count	2 bytes, number of times that the card has switched active AoIP lan port since boot up.

4.158.3 Additional information

Sent in response to a Request Network Redundancy Card Status message.

4.158.4 Scope

CSU to Host.

4.158.5 Performance

4.159 Request Panel Connection Management Action

4.159.1 Description

This message is used to request actions against a matrix panel connection.

4.159.2 Structure

Message ID:	16 bit word, 398 (0x018E).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Action Type:	1 byte. Disconnect Session == 1.
Port:	2 bytes, Port number of the physical panel port

4.159.3 Additional information

The Reply Panel Connection Management Action message is sent in response to this.

Additionally, as the requested end point is disconnected (for disconnect requested) the standard messaging normally associated with this will also be received.

4.159.4 Scope

Host to CSU.

4.159.5 Performance

4.159.6 Example

The following payload would be sent to request the disconnection of client port 10.

01 8F 10 AB BA CE DE 01 01 00 0A

4.160 Reply Panel Connection Management Action

4.160.1 Description

This message is used to reply to the Request Panel Connection Management Action message.

4.160.2 Structure

Message ID:	16 bit word, 399 (0x018F).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Action Type:	1 byte. Action 1 (Disconnect) is the only currently supported action.
Port:	2 bytes. Port number in the request.
Success:	1 byte, 1 == Success, 0 == Failure.

4.160.3 Additional information

Sent in response to a Request Panel Connection Management Action message.

4.160.4 Scope

CSU to Host.

4.160.5 Performance

4.160.6 Example

The following payload would be sent for a successful client disconnection of port 10.

01 8F 10 AB BA CE DE 01 01 00 0A 01

4.161 Request Beltpack Information

4.161.1 Description

This message is used to request all beltpack data records from the matrix. This information record (one per beltpack) includes information on the beltpack hardware, registration and configuration.

4.161.2 Structure

Message ID:	16 bit word, 257 (0x0101).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte. If this is set to 1 then there are no additional parameters in this request. If this is set to 2 then the request type attribute is additionally provided (see below).
Request Type	1 byte. 1 == Get all entries added via map or OTA i.e. not HCI added entries. 2 == Only entries than have been added via HCI API.

4.161.3 Additional information

The Reply Beltpack Information message is sent is response to this.

4.161.4 Scope

Host to CSU.

4.161.5 Performance

4.161.6 Example

The following payload would be sent to request all beltpack records.

01 01 10 AB BA CE DE 01

4.162 Reply Beltpack Information

4.162.1 Description

This message is used to reply to the Request Beltpack Information message. The matrix always responds with all beltpack records, spanning multiple HCI messages if required.

4.162.2 Structure

Message ID:	16 bit word, 258 (0x0102).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte. Schema number of the request message.
Count	2 bytes. Number of beltpack records in this message. If multiple messages are required to transmit the full response, then the continuation flag will signal this.
The following fields are repeated i.e. 1 set of attributes per beltpack of 'count' beltpacks.	
Serial number	4 bytes

PMID	4 bytes
Default Role	2 bytes
Mode	1 byte. 0 = Pool, 1 = Preferred Team, 2 = Preferred Role, 3 = Preferred Role and Team, 4 = Fixed Role, 225 = Not Set.

4.162.3 Additional information

The Reply Beltpack Information message is sent in response to the Request Beltpack Information message.

4.162.4 Scope

CSU to Host.

4.162.5 Performance

4.162.6 Example

The following payload would be sent where there is only one beltpack. Serial number 2, PMID 1000, No default role and Pool mode of operation.

01 02 10 AB BA CE DE 01 00 01 00 00 00 02 00 00 03 E8 FF FF 00

4.163 Request Beltpack Add

4.163.1 Description

This message is used to request the addition of a wireless beltpack to the matrix. This effectively adds a beltpack registration to the matrix with several properties e.g. the beltpack roll association mode and details.

4.163.2 Structure

Message ID:	16 bit word, 403 (0x0193).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.

Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Serial number	4 bytes
PMID	4 bytes
Name	15 bytes. 15-character name of the beltpack.
Role	2 bytes. Associated role (if applicable to mode). Set to 65535 if not used i.e. the null port number.
Mode	1 byte. 0 = Pool, 1 = Preferred Team, 2 = Preferred Role, 3 = Preferred Role and Team, 4 = Fixed Role, 225 = Not Set.
Role Team	1 byte. Associated role team (if applicable to mode). Set to 255 if not used i.e. the null role team ID.

4.163.3 Additional information

The Reply Registration Edit message is sent in response to this message.

4.163.4 Scope

Host to CSU.

4.163.5 Performance

4.163.6 Example

4.164 Reply Beltpack Add

4.164.1 Description

This message is used to reply the addition of a wireless beltpack to the matrix.

4.164.2 Structure

Message ID:	16 bit word, 404 (0x0194).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Serial number	4 bytes
Reserved	1 byte
PMID	3 bytes
Success	1 byte. 0 == Success, 1 == Store Full, 2 == Invalid Request

4.164.3 Additional information

The Reply Beltpack Add message is used to reply to the Request Beltpack Add message.

4.164.4 Scope

CSU to Host.

4.164.5 Performance

4.164.6 Example

4.165 Request Beltpack Delete

4.165.1 Description

This message is used to request the deletion of a wireless beltpack from the matrix. If the associated beltpack is connected it will be disconnected.

4.165.2 Structure

Message ID:	16 bit word, 405 (0x0195).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Reserved	0 bytes
PMID	3 bytes

4.165.3 Additional information

The Reply Beltpack Delete message is sent in response to this message.

4.165.4 Scope

Host to CSU.

4.165.5 Performance

4.165.6 Example

4.166 Reply Beltpack Delete

4.166.1 Description

This message is used to reply to the Request Beltpack Delete message.

4.166.2 Structure

Message ID:	16 bit word, 406 (0x0196).
Flags:	1 byte, see FLAGS on page 59.
Protocol Tag:	4 bytes, always set to 0xABBACEDE. This code is used by the host to establish that the rest of the message is in the HCI 2.0 format.
Protocol Schema:	1 byte, set to 1. If the payload of this message changes format in future releases this schema number will be used to establish the payload structure being passed from this client.
Reserved	1 byte
Success	0 == Success, 1 == Entry Not Found

4.166.3 Additional information

The Reply Beltpack Delete message is used to reply to the Request Beltpack Delete message.

4.166.4 Scope

CSU to Host.

4.166.5 Performance

4.166.6 Example

5 APPENDIX A - GAIN LEVEL TRANSLATION TABLE

Level ref.	Level ref.	Input
0	0x0000	Cut
1	0x0001	-72
18	0x0012	-66
35	0x0023	-60
52	0x0034	-54
69	0x0045	-48
86	0x0056	-42
103	0x0067	-36
119	0x0077	-30
136	0x0088	-24
153	0x0099	-18
170	0x00AA	-12
179	0x00B3	-9
187	0x00BB	-6
196	0x00C4	-3
204	0x00CC	0
212	0x00D4	+3
221	0x00DD	+6
229	0x00E5	+9
238	0x00EE	+12
246	0x00F6	+15
255	0x00FF	+18
263	0x0107	+21
272	0x0110	+24
280	0x0118	+27
287	0x011F	+29

The gain is calculated as:

$$\text{gain (dB)} = (\text{level value} - 204) * 0.355$$

6 **APPENDIX B - GLOSSARY OF TERMS**

ACK	Acknowledge packet.
CPU	Central processing unit
CSU	Central switching unit. Made up of one or more racks, but forming one system.
GPI	General purpose input.
GPIO	General purpose input/output. This feature is provided by using PDE6619 cards.
GPO	General purpose output.
NACK	Not acknowledge packet.
RCU	Rear Connector Unit.
SFO	Special function output. This feature is named Controls in EHX.
Port	The number of the physical connection on the CSU that is used to connect panels and analogue inputs and outputs.
Panel	A Clear-Com intercom system control panel that is connected to the CSU via a Panel Interface
TCP/IP	Transport Control Protocol / Internet Protocol

7 GLOSSARY

Analog Port

Any of the Eclipse matrix's analog input/output RJ-45 connectors that are used to connect cable from the matrix to panels and interfaces. Each "port" connects to a separate audio channel in the matrix intercom system.

Bus

A bus is the channel or path between the components in the matrix along which electrical signals flow to carry information from one component to the next.

Call Signal

A call signal is an electronic signal sent from one panel or interface to another. A call signal can be audible and/or visual.

Typically a call signal is sent to get the attention of a panel operator who may have turned down their intercom speaker's volume or removed their headset. It can also be sent to activate an electronic relay.

Category-5 cable

EIA/TIA 568 category specification relating to network cabling. Shielded category-5 cabling is required for Eclipse matrix wiring.

CellCom

Digital wireless communications product. Sold under the CellCom name in USA and as FreeSpeak in Europe and Asia.

Central Matrix

The term "central matrix" is used to differentiate the central hardware and software of the intercom system from the connected audio devices. The central matrix consists of:

- The metal housing for the circuit cards and power supplies.
- The circuit cards.
- The power supplies.
- The rear panel connectors which connect the matrix's hardware to panels and interfaces.

Destination

A device such as an intercom panel, beltpack, or interface to which audio signals are sent. The device from which audio signals are sent is called a "source".

Duplex

All real-time communication between individuals talking face to face is full duplex, meaning that they can both talk and listen simultaneously. The Eclipse matrices provide full-duplex audio.

EHX

Eclipse HX Configuration System. Software program that guides the operation of the central matrix circuit cards and connected panels.

Ethernet

International standard which describes how information is transmitted across a network. Provides for the efficient organization of network components.

Fiber-optic Cable

A fiber-optic cable consists of a glass core covered with a reflective material called "cladding" and several layers of buffer coating to protect the cable from the environment. A laser sends light pulses through the glass core to the other end of the cable.

FreeSpeak

Digital wireless communications product. Sold under the FreeSpeak name in Europe and Asia and CellCom in USA.

FreeSpeak II

Enhanced Digital wireless communications product.

Full Duplex

Refers to transmission of signals in two directions simultaneously.

IFB

Interruptible Foldback. The term "foldback" refers to sending "program" audio, or some other audio mix, back to announcers while they are on the air. Doing so allows announcers to monitor themselves, other announcers, videotapes of commercials, or some mix of sources, while they on the air. This is typically found in television news and live broadcast events.

Announcers typically wear a small ear piece so they can hear the selected foldback audio mix. When a director wants to give directions to an announcer on air, or to announce changes in the program, the director must "interrupt" the foldback. To do this, the director uses a channel specifically set up to interrupt the foldback audio.

Interface Module

A piece of electronic hardware designed to convert the 4-wire signals of a central matrix port to some other form of communication, such as 2-wire party line, telephone, etc. The interface module is connected to a central matrix port. The external non-4-wire device is then connected to the interface module.

ISO

The ISO function, short for "panel ISOLation", allows a panel operator to call a destination and interrupt all of that destination's other audio paths and establish a private conversation. When the call is completed the destination's audio pathways are restored to their original state before the interruption.

IV-R

Instant Voice Router. Software that routes digital audio data between Concert users and between Concert users and Eclipse systems.

Label

A label is an alphanumeric name of up to five characters that identifies a source, destination, or control function accessed by an intercom panel. Labels

appear in the displays of the intercom panel. Labels can identify panels, ports interfaced to other external equipment, fixed groups, party lines, and special control functions.

Mode

A term used to describe a light path through a fiber as in multimode or single mode.

Multimode Fiber-optic Cable

The glass core of a multimode fiber is larger than the core of a single mode fiber, which causes the transmitted light beam to disperse as it travels through the core. Single mode fiber, with its smaller core, concentrates the light beam so that it carries signals further. Multimode fiber was the first type of fiber offered by manufacturers. Single-mode fiber evolved as production methods improved.

Multiplexing

The process by which two or more signals are transmitted over a single communications channel. Examples include time division and wavelength division multiplexing.

Nanometer (nm)

Common unit of measure for wavelength. One billionth of a meter.

Non-volatile Memory

Data stored in the CPU's firmware (ROM) that is not lost when the power is turned off.

Optical Signal

A laser at one end of a fiber-optic cable pulses on or off to send a light signal through the glass core of the cable to the other end of the cable. Because the light signals are binary (on or off), the signal is digital.

Panel

Also referred to as "station" in some cases (usually older manuals). Any intelligent intercom device connected to the rear-panel analog ports of the central matrix. This term does not refer to devices connected through interface modules.

Port

Any of the input/output connections (RJ-45 connectors) on the back panel of the central matrix. These connectors and the attached cables connect the central matrix to remote intercom devices. The term "port" emphasizes that the connection is a "portal" between the central matrix and the remote intercom devices.

Program

Any separate audio source that is fed into the intercom channels. In television applications, for example, "program" audio is the audio that is broadcast on air.

Rack Unit or RU

Standardized unit of mounting space on a rack panel. Each rack unit is 1.75 inches (44.45 mm) of vertical mounting space. Therefore 1 RU is 1.75 inches (44.45 mm) of vertical mounting space, 2 RU is 3.5 inches (88.9 mm), 3 RU is 5.25 inches (133.35 mm), and so on.

Remote Panel

Any intelligent intercom device connected to the back-panel ports of the central matrix. This term does not refer to devices connected through interfaces.

Sidetone

The sound of the panel operator's own voice heard in their own earphone as they speak.

Single-mode Fiber-optic Cable

The glass core of a single-mode fiber is smaller in diameter than the core of a multimode fiber, so that the light signal transmitted over the core is more concentrated than with multimode fiber, which allows the signal to travel further. Single-mode fiber evolved from multimode fiber as production methods improved.

Source

In this manual, the term "source" refers to a device—such as an intercom panel, interface, or beltpack—that sends audio into the matrix. The device to which audio is sent is called a "destination".

VOX

In the Eclipse system, when audio at a panel exceeds a threshold, a light switches on at the panel's port card to visually cue the operator. The threshold level is set in the Eclipse Configuration Software.

V-Series

Communications panels used with Eclipse systems providing advanced facilities. Available in rack mount and desktop formats.

Wavelength-division Multiplexing (WDM)

A method of multiplexing optical signals developed for use on fiber-optic cable. Each signal is assigned a particular wavelength on the light spectrum and therefore many signals can be transmitted simultaneously without interfering with each other.